

Agentic Exploration of Physics Models

Maximilian Nägele^{✉*} and Florian Marquardt[✉]

*Max Planck Institute for the Science of Light, Staudtstraße 2, 91058 Erlangen, Germany and
Department of Physics, Friedrich-Alexander Universität Erlangen-Nürnberg, Staudtstraße 5, 91058 Erlangen, Germany*

The process of scientific discovery relies on an interplay of observations, analysis, and hypothesis generation. Machine learning is increasingly being adopted to address individual aspects of this process. However, it remains an open challenge to fully automate the heuristic, iterative loop required to discover the laws of an unknown system by exploring it through experiments and analysis, without tailoring the approach to the specifics of a given task. Here, we introduce SciExplorer, an agent that leverages large language model tool-use capabilities to enable exploration of systems without any domain-specific blueprints, and apply it to physical systems that are initially unknown to the agent. We test SciExplorer on a broad set of models spanning mechanical dynamical systems, wave evolution, and quantum many-body physics. Despite using a minimal set of tools, primarily based on code execution, we observe impressive performance on tasks such as recovering equations of motion from observed dynamics and inferring Hamiltonians from expectation values. The demonstrated effectiveness of this setup opens the door towards similar scientific exploration in other domains, without the need for finetuning or task-specific instructions.

I. INTRODUCTION

In recent years, specialized machine learning techniques have been increasingly employed to address various individual aspects of the scientific discovery process, for example, by suggesting new experiments [1–3], predicting or analyzing experimental results [4–6], learning meaningful representations of data [7, 8], or recovering governing equations as symbolic expressions [9]. However, these tools are typically designed to solve a specific task in a specific setting. Large-language models (LLMs) exceed the capabilities of the usual machine learning approaches in several ways: Famously, they can work on a large variety of tasks without any specialized training data set, often in a zero-shot manner, i.e. without even providing a few exemplary solution templates [10]. They work well with arbitrarily structured input, including multimodal data mixing text and images [11] and can reason, producing a textual record of their arguments. As a result, LLMs can work well in situations where heuristic approaches and some level of creativity are required, all the time relying on their general factual knowledge garnered during training. For the tasks we will consider here, this includes in particular math and physics knowledge, supplemented by outstanding abilities in coding [12, 13]. At the same time, the greatest weakness of LLMs is their propensity to hallucinate. One approach to overcome this to some extent, also relied on in the present work, is to focus on scenarios where some level of self-correction is possible, with independent ways to check and repudiate results [14].

In the past few years, the power of LLMs has been boosted by agentic behavior (tool use). By allowing the LLM to call up external software tools, one can combine the reliability of those tools with the heuristics and general knowledge of the LLM [15]. First examples of agentic

behavior have recently been introduced into several scientific domains. In computer science, LLMs conduct complex coding tasks [16], and can, to some extent, also suggest new research ideas and summarize their findings [17]. In chemistry, they combine web search, specialized simulation tools, and access to laboratory interfaces to, for example, optimize chemical reactions [18], and synthesize molecules [19, 20]. In biology, agentic LLMs are employed to find gene perturbations resulting in specific phenotypes [21], and design gene-editing experiments [22].

In physics, non-agentic LLMs have been used to solve well-defined tasks such as calculating Hartree-Fock Hamiltonians [23] or answering questions about astrophysics [24]. Additionally, their performance has been tested on a suite of benchmarks containing typical exam questions ranging from high school to Olympiad-level difficulties [25–27], as well as on typical scientific coding tasks [28].

While exploring the capabilities of agentic LLMs in physics has been called for [29, 30], previous work focuses on building specialist agents with specific tools for tasks that require adaptability but typically follow a structured blueprint. Examples include density functional calculations [31], cosmology calculations [32, 33], calculating properties of topological materials [34], inverse design for meta-materials [35], designing integrated photonic circuits [36], cooling of trapped atom experiments [37], and calibrating superconducting quantum processors [38].

In this work, we focus on exploring the capabilities of a general AI physicist in a verifiable setting requiring heuristic, open exploration. Crucially, and in contrast to previous work, the artificial physicist is given only minimal task specific instructions. Therefore, if it performs well on the benchmark problems, we can expect good performance also on other tasks. Specifically, we will explore how tool use can enable an LLM to automate the heuristic loop of scientific discovery, including selecting experiments, analyzing results, and forming hypotheses when exploring an unknown system.

* maximilian.naegele@mpl.mpg.de

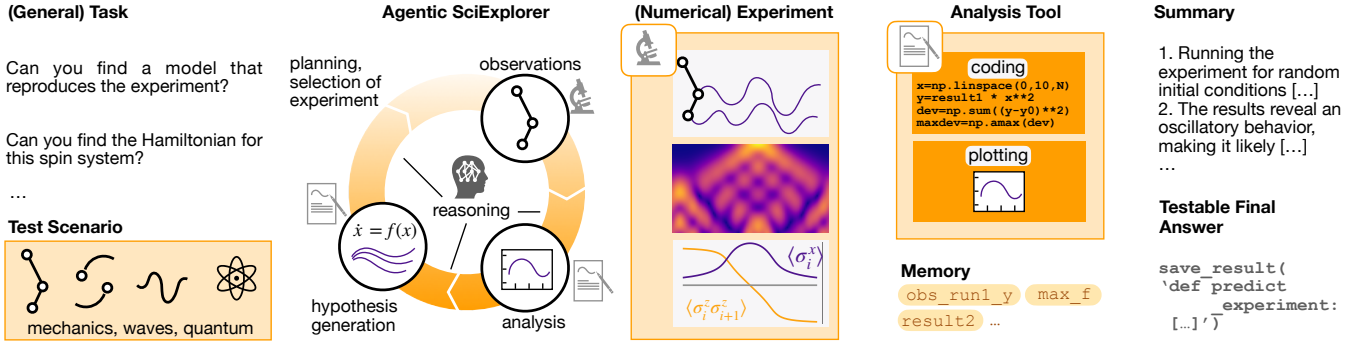


FIG. 1. **Agentic SciExplorer**. The explorer is given a general task, which is then applied to a specific physics scenario. During the scientific exploration cycle, LLM-based reasoning is employed to select a (numerical) experiment to be carried out, calling the appropriate tool. The resulting observations are analyzed using one or several calls to analysis tools, often involving both the coding and the multimodal capabilities of the LLM. This cycle repeats until the LLM agent decides it has acquired sufficient information. Finally, a summary of the reasoning chain is produced for the human user. For benchmarking, the agent is asked to provide a testable final answer, e.g. in the form of code that can be run and evaluated.

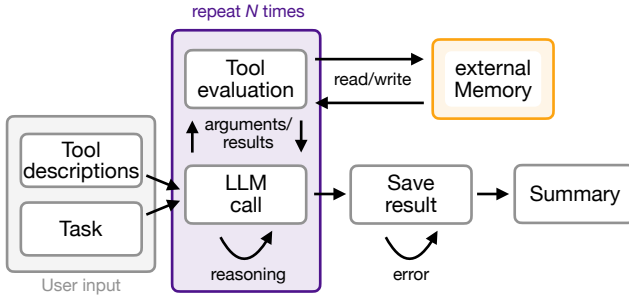


FIG. 2. **SciExplorer scheme**. Given task and tool descriptions supplied by the user, SciExplorer executes up to N (in our case 100) steps fully autonomously. In each step, SciExplorer can reason verbally and specify arguments for tools it wants to evaluate. The tools are then executed, reading and writing to an external memory containing all previous tool results. The results are typically returned to SciExplorer in plain text. However, for large numerical arrays, SciExplorer receives a description of their type and shape. When SciExplorer calls the `save_result` tool, the exploration stops. In case of a syntax error, SciExplorer can try again, otherwise it is asked to summarize the exploration.

and a simulator useful for a wide range of tasks (e.g. a differential equation solver) The tools we consider include running experiments (here, numerical experiments), performing data analysis through a general coding tool, and visualizing results. Based on these tools, the LLM can use its reasoning to drive an active learning process, deciding which analysis or experiment to carry out next based on previous results. We will explore in prototypical physics scenarios, drawn from fields like dynamical systems, wave dynamics, and quantum many-body systems, how this approach can successfully solve research-oriented tasks requiring heuristics by mimicking essential aspects of the scientific process. Tasks we will consider include discovering the equations of motion of a system

by observing its dynamics and discovering Hamiltonians of quantum many-body systems from measurements.

II. AGENTIC SCIEXPLORER

The Agentic SciExplorer is based on an LLM agent that autonomously selects and analyzes experiments (see Figure 1), where ‘experimental’ results here are obtained via numerical simulations. In a typical scenario, the agent specifies initial conditions, observes the time evolution of a system, and adaptively selects subsequent experiments based on its previous analysis. To support this process, the agent can access two generic analysis tools. Rather than providing narrowly defined, user-specified tools, we leverage the advanced coding capabilities of modern LLMs[28], giving the agent access to a coding tool that executes arbitrary Python code. This choice maximizes flexibility and enables dynamic tool creation by the agent during runtime. Additionally, the agent can access a code-based plotting tool, which, paired with the agent’s multimodal capabilities, enables the agent to quickly grasp qualitative insights about the system under study by visual inspection. We integrate an external memory that is accessible through the analysis tools, to offer reliable persistence of experimental and analysis results in the form of arrays of numerical values. When addressing a user-defined task, the agent proceeds in multiple steps. Each step involves verbal reasoning and may include multiple tool calls for controlling the experiment and analyzing results. The tool results are then communicated to the agent and added to the memory (see Figure 2 and Supplement A for a more complete description)

To explore the intrinsic capabilities of state-of-the-art LLMs in successful agentic exploration, we deliberately keep any domain-specific input to a minimum. The relatively short system prompt only provides general ad-

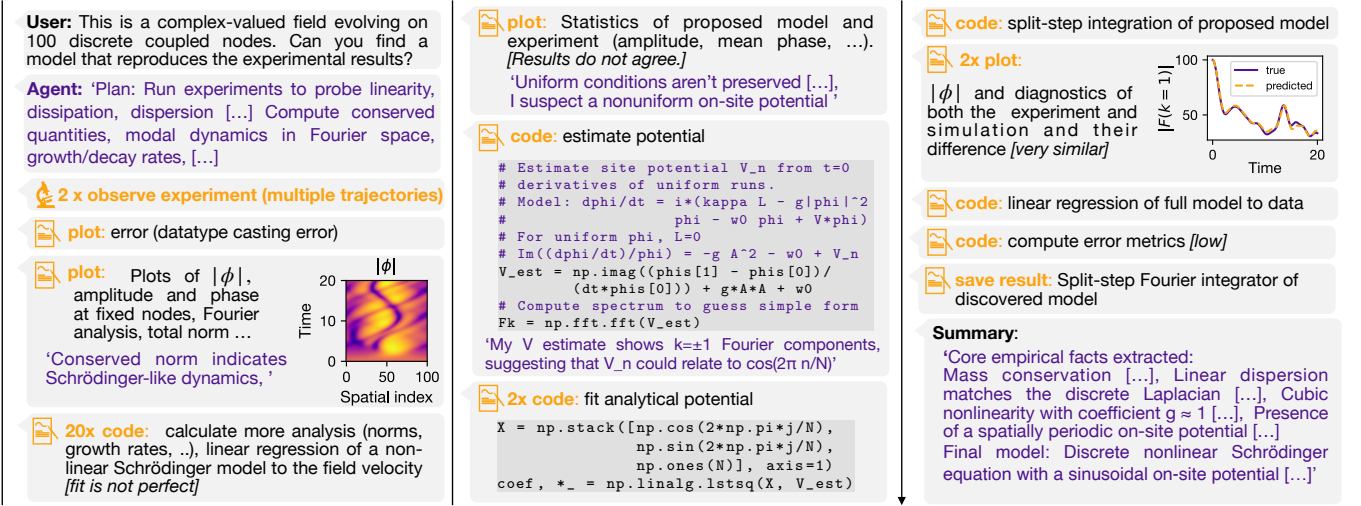


FIG. 3. **Example exploration.** The agent is tasked with discovering the unknown model of a complex field system by observing and analyzing its dynamics. In this figure, we sketch in a compressed fashion some of the steps in the resulting extended autonomous exploration. The agent first runs several experiments with varying initial conditions. Then, it infers the qualitative model (here, a nonlinear Schrödinger equation with external potential) from visualizations and analysis based on the experimental results. It subsequently estimates the external potential and discovers its analytical cosine structure. To validate the proposed model, the agent simulates time evolution using a (self-coded) split-step integrator and compares the results to experimental data. Finally, it saves its result as a Python function that reproduces the experimental results and contains both a simulator of the system and the underlying partial differential equation. The sequence of steps depends entirely on the choices of the agent, which are adapted to the given problem, reacting to its own observations and conclusions.

vice for scientific exploration (like systematically formulating hypotheses) but does not contain task-specific or even domain-specific information and is applied across all domains explored in the following (see Supplement F). When providing information to the agent through the system description or the documentation of the tools executing experimental runs, we follow a simple philosophy: We only provide information that an experimentalist investigating such a system would have access to. This includes, for example, the dimensionality, topology (e.g. 1d vs. 2d), and constituents (e.g. particles vs. spins) of the system but not the general shape of its governing laws. Therefore, we do not even tell the agent that mechanical systems are governed by ordinary differential equations or wave systems by partial differential equations.

We evaluate SciExplorer on two qualitatively distinct classes of tasks. In the first scenario, the agent is asked to reproduce the experimental results by implementing a Python function. This setting involves not only model discovery but is closer to the even more challenging task of program discovery, as the solution space consists of the vast set of all valid functions. In this case, correct solutions typically include both a simulator of the system and a description of the system’s governing laws. In the second scenario, the agent is asked to identify the model of the system within a broad but predefined model class. The advantage of this approach is that the resulting models tend to be more interpretable. However, it requires some limited prior knowledge about the class of admissible models. To evaluate SciExplorer on both task types,

we consider the first scenario for mechanical and field systems, and the second scenario for quantum systems, where the agent is asked to determine the Hamiltonian governing the experiment.

If not otherwise specified, we use GPT 5 [39] as the state-of-the-art LLM backbone of the SciExplorer.

III. RESULTS

SciExplorer is able to recover accurate models even when starting from minimal information about the system under consideration. Below, we explicitly present all information provided for a representative example: the damped double pendulum (see Supplemental Material [40] for details on all systems). The system and task description reads: ‘You are investigating a dynamical physical system. Can you find a model that reproduces the **observe_experiment** function? After your exploration, save it using the **save_result** function’. The **observe_experiment** tool provides only the additional information ‘The system has 2 generalized coordinates’. It also includes a description of the format of its arguments, namely the initial generalized coordinates, initial generalized velocities, the start and end times of the experiment, and the time resolution.

A. Mechanical Systems

Mechanical systems provide an ideal testbed for the SciExplorer due to their wide diversity spanning conservative, dissipative, and driven regimes, and the wide range of accessible complexity levels, selected via choosing the equations of motion or their dimensionality. An example of an illustrative exploration process is summarized in Figure 3.

For test purposes, we considered generic dynamical systems with up to three generalized coordinates and systems with particles moving in 2d (with up to ten particles) subject to fixed potentials or pairwise interactions (for the full equations of motion see Supplement, Table S1 [40]). Figure 4 summarizes the agent’s performance across these systems when asked to infer their equations of motion. The agent receives information only about the number of generalized coordinates (or number of particles moving in 2d) and is asked to produce a Python function that reproduces the experimental results. To quantify the agent’s performance, we compute the coefficient of determination R^2 between the velocities and accelerations predicted by the agent and the ground-truth dynamics.

As an expanded challenge, we also consider dynamical systems where the agent is only allowed to observe a subset of particles, and is not even told the number of unobserved particles. In those cases, we additionally require the agent to infer the initial conditions of the hidden particles. We then report the mean R^2 value across trajectories with random initial conditions of the observable particle (see Supplement B for details).

For a large subset of systems, the agent recovers the governing model both qualitatively and quantitatively, often achieving perfect fits ($R^2 \approx 1$) using only observations of the system’s time evolution under initial conditions selected by the agent. Performance degrades for more complex or atypical systems, and the agent only sometimes recovers the correct model. Systems that are not close to any known models (e.g. the ‘arbitrary 2d potential’) are particularly challenging, indicating that the LLM agent relies on its broad knowledge base to be successful for other systems. To identify systems correctly in most cases, the agent autonomously implements a broad repertoire of analysis techniques by generating and executing Python code, without requiring specific tools. Typically, the agent first extracts qualitative signatures from visualizations. These include, among others, linearity, mirror symmetry, limit cycles, attractors, and decay. It then constructs candidate basis terms for the governing differential equations and fits their coefficients to the observed accelerations. When applicable, it uses sparse linear regression [41]. Otherwise, it fits numerical solutions of the proposed differential equations directly to the experimental data. Depending on the task, the agent uses a large set of additional strategies, such as locating centers of attraction in 2d by maximizing the radial acceleration with respect to a candidate center, testing

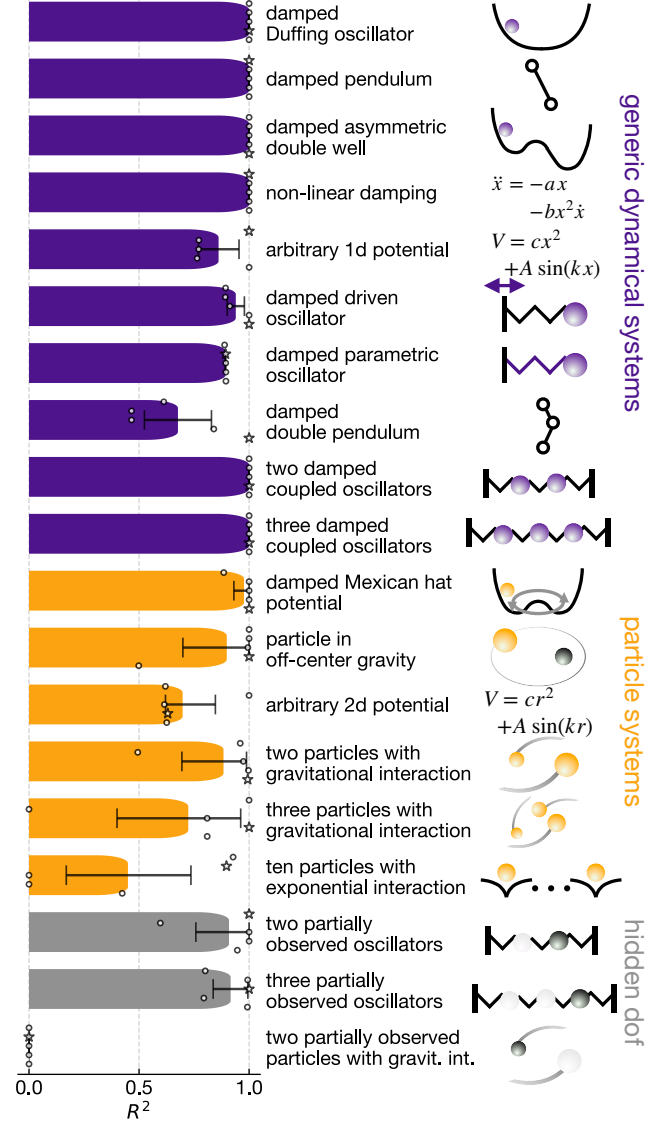


FIG. 4. **Mechanical systems.** The agent discovers the equations of motion of black-box physical systems by specifying initial conditions, observing dynamics, and analyzing the results. We consider generic dynamical systems with one to three generalized coordinates, particles moving in 2d, and systems where an observable particle interacts with an unknown number of hidden degrees of freedom (dof). We run 5 independent attempts per system and show the mean coefficient of determination R^2 of the agent’s proposed model with the true system with 95% bootstrap confidence intervals. The agent can recover the true model in a large subset of systems. Stars indicate the conversation the agent considers best when asked to rank all attempts.

for angular momentum and energy conservation, and using Fourier transforms to find initial parameter guesses for oscillator systems or to infer the number of hidden particles in partially observed systems (for example explorations, see Supplement S3A and S3B, for the models discovered by the agent, see Supplement Table S2 [40]).

Common failure modes of the agent include premature commitment to a set of models without reconsidering when fits are poor and overlooking qualitative cues visible in plots, such as periodic oscillations in acceleration plots (for an example, see Supplement S3C [40]).

Furthermore, we test the agent’s ability to self-critique by asking it, for each mechanical system, to identify in which of multiple conversations it performed best (stars in Figure 4). When the correct model was found at least once, the agent reliably selects that conversation. However, when none of the proposed models was exactly correct, the agent often does not choose the conversation corresponding to the highest R^2 value. Instead, it tends to prioritize qualitative agreement, such as the existence of bound and unbound trajectories, over purely quantitative fit quality.

It is important to note that even in a real experimental setting, one can verify the quality of the solution discovered by the agent without knowing the ground truth by comparing the agent’s theory with new experimental data. Therefore, one can boost the effective success rate of the agent by running several explorations and picking the best-performing model (in all but three of the scenarios of Figure 4, this leads to the correct solution, as it is among the five runs).

To understand which components of the SciExplorer are essential, we perform ablation studies. Depending on the physical system, both the plotting and coding tools are crucial for consistent performance. To test the importance of tool access, we compare a given exploration run of our agent against a conversation where an LLM without access to tools is initially provided with visualizations for the same number of experiments, but now with randomly chosen initial conditions. In this case, the LLM cannot recover the correct model. Interestingly, we also find that GPT 5 [39] performs much better than Gemini 2.5 pro [42] (for details see Supplement E).

The superior performance of LLM agents based on state-of-the-art reasoning models relies on significant runtime investment. For the systems studied here, a single exploration ranges from a few minutes to over 1.5 hours, with the bottleneck being the LLM’s response time, not the numerical simulations of the system. We also observe that the agent generally requires more time to solve more complex systems than simpler ones (see Figure A2). Nevertheless, based on our experience as working theoretical physicists, we estimate that the overall runtime for a single task is already lower than what even an expert human would typically need for such an initially completely open exploration of an unknown system.

B. Dynamics of Waves and Fields

A fundamental cornerstone of the physical description of nature is the notion of dynamics of many degrees of freedom arranged in an extended space, with locality and

translational invariance as guiding principles. These lead naturally to the evolution equations of fields and waves, which also form the basis of the most fundamental theories we have. We ask the LLM agent to explore this domain by providing access to ‘experimental’ runs where the agent can set up an arbitrary initial configuration of a field ϕ and observe the subsequent dynamics $\phi(x, t)$ that is governed by a classical evolution equation unknown to the agent. The agent is provided only with information about the system’s topology: *‘This physical system consists of a complex-valued field evolving on 100 discrete coupled nodes. The nodes are arranged in 1D with periodic boundary conditions, i.e. the first and last node are neighbors.’* As before, the agent is asked to implement a function that reproduces the experimental observations. The agent may aid its analysis by running simulations for arbitrary hypothesized field equations. In the present incarnation of this scenario, we specialized to 1d field equations (with correspondingly lower numerical effort for a single run) for a complex field ϕ . Examples that we tested the agent on include the nonlinear Schrödinger equation, describing the dynamics of matter waves of interacting atoms or light waves subject to nonlinearities, and the time-dependent Ginzburg-Landau equation, describing the evolution of an order parameter field, e.g. in a superconductor, relaxing to its thermodynamic equilibrium configuration. In each case, there can be variations like the presence of an arbitrary external potential or of longer-range coupling terms on the underlying lattice. The SciExplorer can discover the true model for any of these variations when given multiple attempts (see Figure 5). However, it struggles with an artificial model describing relaxation into a sinusoidal potential for the field.

In all field and wave scenarios, we provide the same simulation tool to the agent, which requires the agent to spell out its hypothesized evolution equation in Python code. This code defines the local evolution in x -space (including any nonlinear and potential terms) and the evolution in k -space (the kinetic part, including wave propagation or diffusion). These components can be directly used within a split-step integrator. The agent performs very well in translating back and forth between the evolution equation itself (e.g. $\partial_t \phi = -i\Delta\phi - ig|\phi|^2\phi$), which it mentions in its analysis, and the corresponding Python specifications that also account for lattice dispersion.

In our studies, we observe that the agent often lays out a detailed exploration plan in advance, mentioning the most important field theories as plausible options, sometimes including expected signatures, such as Bragg scattering for linear waves in periodic potentials or soliton-like features for Kerr-type and other nonlinearities. It tends to start with a Gaussian wave packet as its choice of initial condition in the first experiment. If the results of this experiment indicate wave-like dynamics, the agent often adopts a principled approach where it first tries to extract the dispersion relation. One of its strategies is to launch several plane waves of varying wavenumber

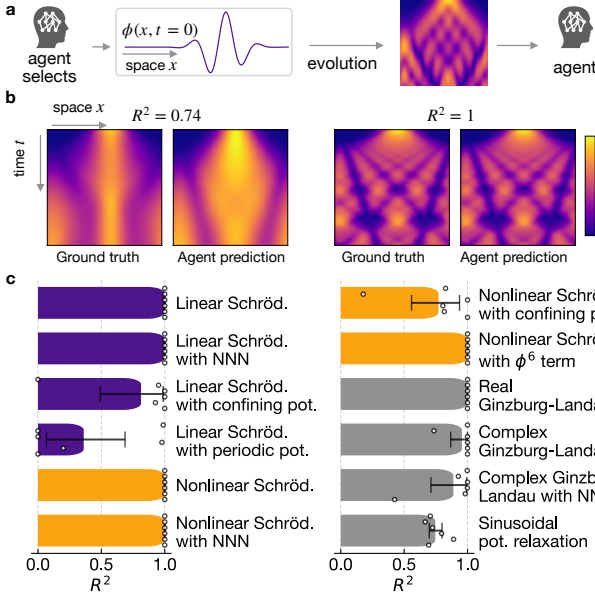


FIG. 5. Waves and fields. **a** In its exploration of waves and fields, the agent can select the initial field configuration for each experimental run. It can also define and simulate field equations for comparison. **b** Evolution of the absolute square $|\phi|^2$ of a Gaussian wave packet for the true model and the agent’s discovered model (announced by the agent at the end of the exploration). The true model on the left is a linear Schrödinger equation with confining potential (we do not show the most accurate model discovered by the agent in its multiple runs). On the right, the true model is a complex Ginzburg-Landau equation with next-nearest-neighbor hopping on a tight-binding lattice. The R^2 value is calculated between the evolution equations for the true and the predicted model, for multiple reasonable initial conditions (see Supplement C for details). **c** Statistics for various scenarios. We run six independent explorations, and the agent can recover the true model ($R^2 \approx 1$) in all but one scenario in at least one attempt.

and low amplitude to avoid nonlinearities, and to analyze the evolution of the phase of the field ϕ . Once that is settled, it tries to determine any nonlinearities by checking the amplitude dependence of the evolution. In linear systems, the agent often fits a correct 100×100 Hamiltonian to the data but cannot save it, since it is only stored in the external memory not accessible by the `save_result` tool (counted as $R^2 = 0$). If the initial dynamics are not wave-like but contractive, with dissipation leading to some steady state, it will adapt its strategy accordingly. In both cases, the agent often visualizes the field $\phi(x, t)$ using space-time density plots, e.g. with the declared aim to ‘spot qualitative structures (breathers, oscillations, solitons)’. To compare its proposed model with observational data, the agent typically implements a split-step integrator or, in linear systems, diagonalizes the Hamiltonian to simulate the proposed field equation. Using these strategies, we find it to quickly make systematic progress (see Supplement Table S4 for the mod-

els discovered by the agent and Supplement S3D for an example exploration [40]).

C. Quantum Many-Body Physics

In domains ranging from condensed matter physics to particle physics, the fundamental language is that of quantum many-body systems. The essence of this domain, with complexities like the exponential growth of the Hilbert space with particle number and the presence of entanglement, is captured well by models of interacting spins or qubits. We therefore let the agent explore and analyze spin Hamiltonians, a task with wide-ranging applications from solid-state physics to quantum technologies [43, 44]. In terms of experiments, we test two variations: (i) In one setting, the agent can initialize the system in a product state and then observe the temporal evolution of the expectation values of single-spin operators. (ii) In another setting, the agent can construct arbitrary observables (Hermitian operators) and obtain their expectation values in the ground state of the unknown Hamiltonian (see Figure 6). In both settings, the agent is provided with information that the system has a 1d or 2d configuration and that it consists of spins.

As usual, the agent is also provided with a tool that can set up a freely chosen many-spin Hamiltonian and either simulate its time dynamics or get its ground state expectation values, respectively. The agent is told that it may specify the Hamiltonian or the operators via Python code that is given access to the predefined spin operators.

We find that the agent has an active working knowledge of quantum many-body physics and spin models in particular, including standard models like transverse Ising, XXZ, and Heisenberg, but also e.g. Dzyaloshinskii-Moriya (spin-orbit induced) chiral couplings. It can deploy that knowledge reliably in this challenge.

For the dynamical setting, where the time traces of spin expectation values can look very complex, the agent can nevertheless quickly rule out some hypotheses. For example, it may run short-time dynamics for multiple initial states, which allows it to first check for conserved quantities such as the global magnetization, compute dominant frequency components using Fourier transforms, and discriminate local and many-body interactions. It then often obtains fits to hypothesized families of Hamiltonians (for an example, see Supplement S3E [40]). We also introduced an even more challenging scenario, where the agent can only observe and control a subset of spins. As expected, this appears to be one of the most complex tasks, and the agent only sometimes makes good progress. For example, in one attempt to discover a ten-spin Heisenberg model by observing only two spins, the agent correctly identifies a tunable field in the X direction but mistakes the coupling to be of XY- instead of Heisenberg-type.

In the ground state setting, the agent typically first probes the symmetry and structure of the Hamiltonian

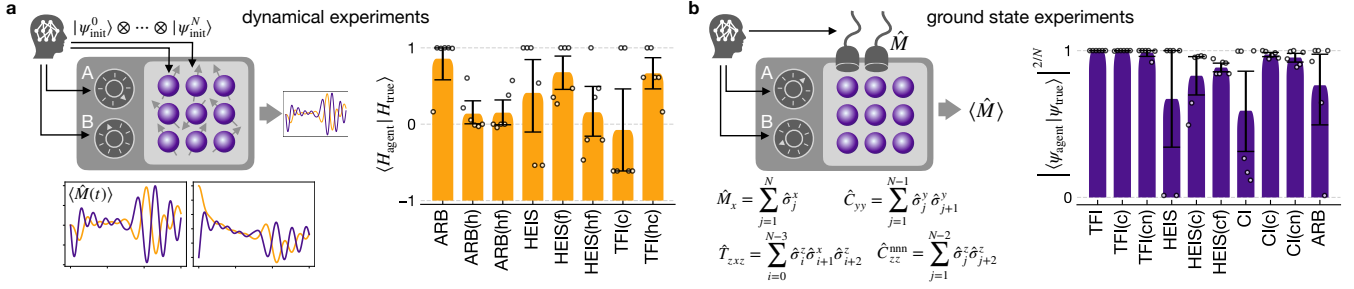


FIG. 6. **Quantum many-body physics.** **a** Discovering the Hamiltonian of a system of multiple spin-1/2 particles, based on observing the dynamics for initial conditions selected by the agent. Each experimental run produces the evolution of the single-spin expectation values $\langle \hat{M}(t) \rangle$ (with $\hat{M} = \hat{\sigma}_j^x$, $\hat{\sigma}_j^y$, and $\hat{\sigma}_j^z$), for an initial (product) state selected by the agent. In some experiments, we ask the agent to discover a whole class of Hamiltonians by allowing it to control experimental parameters, whose meaning it is not aware of ('A' and 'B'). The bottom left shows an example – the complex dynamics of two spins of a Heisenberg model. Right: Performance for various scenarios. Normalized scalar product between the true Hamiltonian and the Hamiltonian proposed by the agent (1 means perfect match, for details see Supplement D). We consider the following systems: *ARB*: Arbitrarily chosen Hamiltonian acting on 3 spins, *HEIS*: 1d Heisenberg model with 10 spins, *TFI*: 1d transverse field Ising model with 10 spins. The letters in brackets denote whether the agent can vary the value of a field parameter (f), of a coupling parameter (c), or observe only two spins of a larger chain (h). **b** Hamiltonian discovery by measuring the ground state expectation values of spin operators $\hat{M}$ defined by the agent. Some examples are shown on the bottom left. Right: Fidelity scaled by the number of spins N between the ground state of the Hamiltonian predicted by the agent after the exploration and the true ground state. We consider the following systems: *TFI*: 1d transverse field Ising model with 10 spins, *HEIS*: 2d Heisenberg model with 9 spins, *CI*: 1d cluster Ising model with three-body interactions and 10 spins, *ARB*: Arbitrarily chosen Hamiltonian acting on 3 spins. Letters denote the same as in part a. Additionally, the agent can sometimes vary the number of spins in the chain, as denoted by (n).

by selecting a limited set of intelligently chosen operators, obtaining single-spin expectation values as well as nearest-neighbor correlators but also more subtle quantities suited e.g. for exploring potential chirality, like $\langle \hat{S}_x^j \hat{S}_y^{j+1} - \hat{S}_y^j \hat{S}_x^{j+1} \rangle$. It quickly deduces aspects like open boundary conditions, translational invariance, and the likely absence of some families of spin-coupling terms in the Hamiltonian. It then selects candidate spin Hamiltonians, conjectures values of their parameters, and runs the simulation tool to confirm or reject these hypotheses. The agent is also cautious enough to keep an open mind (as per its instructions) and lists possibilities that have not yet been refuted, like more subtle, longer-range couplings.

In an extension to both the dynamical and ground-state setting, we let the agent control one or several parameters whose meaning it is unaware of (mimicking closely a potential experimental scenario of exploration), letting it discover a whole family of Hamiltonians. Here, the agent sometimes struggles with specifying the correct scaling or sign of the tunable parameter. For example, in the 2d Heisenberg model with varying coupling and field, it correctly identifies the Hamiltonian structure but misses a factor of two in all six explorations. However, we find that the agent can also successfully solve this extended task for many of the other considered systems by, for example, We find that the agent can also successfully solve this extended task, for example, by creating visualizations of expectation values against the strength of the tunable parameters (for an example, see Supplement

S3F [40]).

Overall, as demonstrated in Figure 6, the agent also performs impressively well in discovering models in quantum many-body physics, again without any fine-tuned instructions or a prescribed exploration loop.

D. Experiments with noise

A common challenge in recovering physical models from experimental data is the presence of measurement errors. For classical systems, measurement errors are often modeled as Gaussian noise, which we add to the mechanical and field systems. In quantum mechanics, experimental observables cannot be measured exactly. Instead, they are estimated from a finite number of single-shot measurements. Therefore, we investigate the agent's performance under a finite number of measurement shots per expectation value. We run additional experiments on one prototypical system from each physical scenario presented above, using two different noise levels. We find that while SciExplorer's predictions become less consistent as noise increases, it can recover highly accurate models even under significant measurement noise for all the considered systems in at least one of six attempts per system (see Figure 7).

E. Common failure modes

As demonstrated above, the general knowledge and

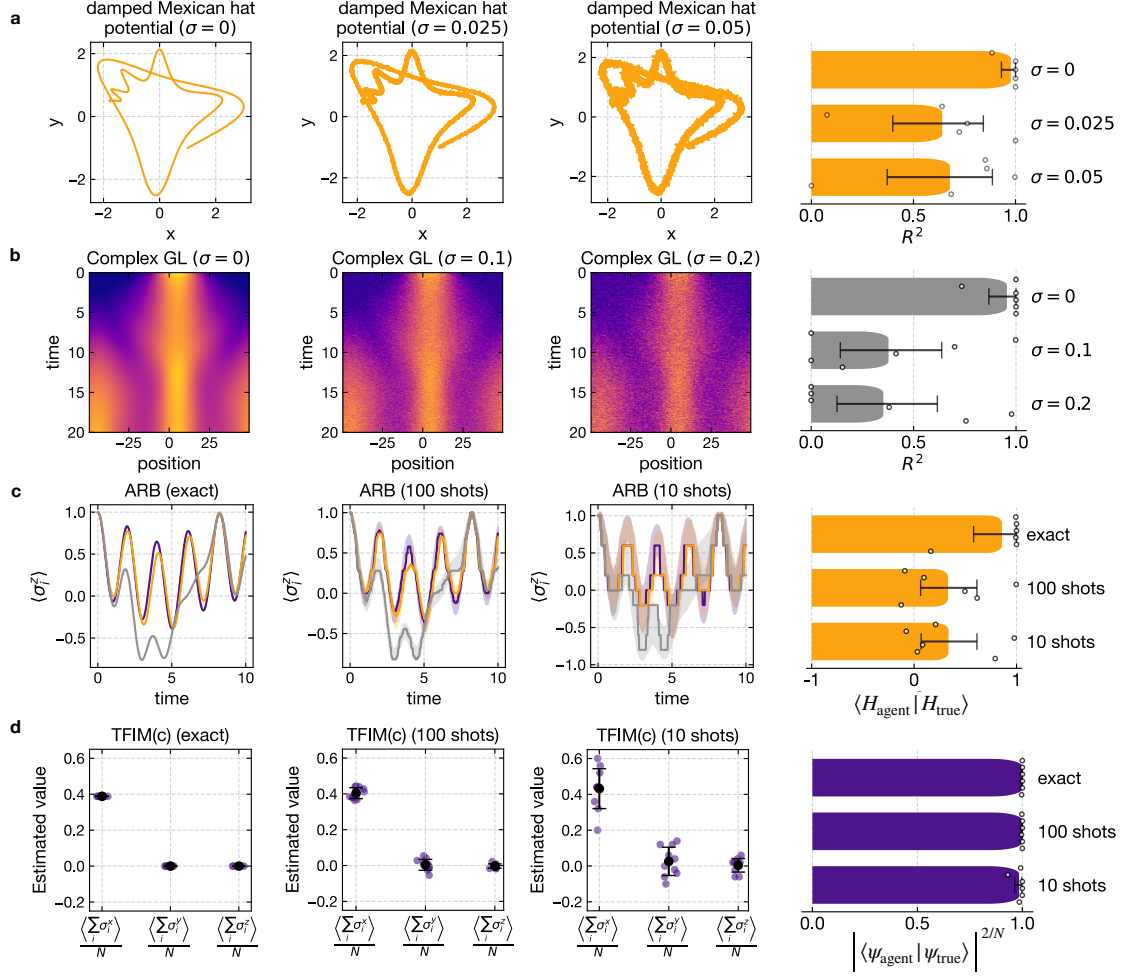


FIG. 7. Experiments with noise. While the predictions of SciExplorer become less consistent for higher measurement noise levels, it successfully recovers a highly accurate model in at least one of six attempts per noise level and scenario. **a** Left: Example trajectories of a particle in the damped Mexican hat potential under varying Gaussian noise levels. Right: Performance of SciExplorer for different noise strengths. **b** Left: Evolution of the absolute square $|\phi|^2$ of a Gaussian wave packet in the complex Ginzburg Landau scenario under varying Gaussian noise levels. Right: Performance of SciExplorer for different noise strengths. **c** Left: Example trajectories of single spin σ^z expectation values under the arbitrary Hamiltonian (ARB) in the quantum dynamics setting. Expectation values are estimated from different numbers of single-shot measurements. Shaded areas indicate standard deviations over multiple runs. Right: Performance of SciExplorer for various numbers of measurement shots. **d** Left: Ground state expectation values of the total spin operators in x, y, and z direction in the transverse field Ising model with variable coupling for different numbers of measurement shots. Dots show ten examples of estimated expectation values and bars denote standard deviation. Right: Performance of SciExplorer for various numbers of measurement shots.

reasoning capabilities of state-of-the-art LLMs are impressive. However, they are by no means perfect and still occasionally hallucinate facts, overlook important clues, or perform superficial analyses.

To potentially improve SciExplorer in the future, a detailed understanding of its failure modes is desirable. Therefore, we manually analyze all failed attempts reported in this manuscript to identify recurring mistakes made by the agent. We categorize these errors into four groups, noting that a single failed run may be assigned to multiple categories (see Figure 8). In particular, in the quantum systems, and occasionally in the mechanical systems, the agent does not perform sufficiently diverse

experiments to uncover all qualitative signatures of the system. For example, in the quantum ground-state setting, it may fail to consider enough expectation values, leading to an incorrect model that nevertheless fits all observed measurement results. Another common problem is that the agent discovers the correct qualitative model structure but fails to infer the correct numerical parameters. In the mechanical and wave settings, this often results from unsuccessful parameter fits. In the quantum setting, most errors arise from simple sign errors or from missing factors of two in some Hamiltonian terms. In such cases, the agent fails to even consider the possibility of an overall scaling factor or sign change.

When analyzing mechanical systems, the agent also frequently overlooks qualitative clues that are, in principle, contained in its analysis results, most often in visualizations. Examples include missing fast oscillations modulating an overall trend or failing to detect the breaking of translational invariance by an external potential in field systems (for an example, see Supplement S3C [40]). Most runs fail not because the agent missed an obvious clue (at least not obvious to the first author of this paper), but because its analysis was not sufficiently exhaustive to identify the correct model, leading the agent to commit prematurely to an incorrect one.

SciExplorer employs heuristics in a similar manner to a human physicist. While this helps SciExplorer navigate the exponentially large search space of possible solutions, it also means that there exists a bias towards ‘likely’ models. However, physically relevant models of an experimental system will typically not contain completely unknown ingredients, but rather combinations of individual, in principle known, parts (e.g. different terms in a wave equation). To test SciExplorer’s abilities on ‘uncommon’ systems, we included examples such as the ‘arbitrary potentials’, ‘sinusoidal potential relaxation’, and ‘arbitrary Hamiltonian’. We find that the agent can still recover the true underlying model in the non-standard mechanical and quantum systems, while it only finds approximate solutions in the wave system containing an artificial ‘ $\sin(0.1|\phi_n|^2)\phi_n$ ’ term in the wave equation.

Many of these failure modes could potentially be mitigated through more sophisticated prompting (e.g. to avoid sign errors in quantum many-body systems). However, we deliberately avoided such interventions in the spirit of not introducing task- or system-specific fine-tuning. Other failure modes may only be alleviated by future LLMs with, for example, improved visual capabilities.

F. Comparison to symbolic regression

In this manuscript, we describe a general AI scientist and benchmark its capabilities in a verifiable setting, namely discovering the model of an unknown physical system. The systems considered here are all governed by analytic expressions. Symbolic regression is a research field concerned with automatically extracting such expressions from data [45]. A human attempting to solve the same task used to benchmark SciExplorer (model discovery) would typically use symbolic regression techniques as part of its discovery process. These techniques can be categorized into three classes: First, standard regression assumes a known analytical expression in which only numerical constants must be adjusted. Second, sparse regression techniques fit a linear combination of Ansatz functions to the data (e.g. SINDy [41, 46] for ordinary differential equations and PDEFIND [47] for partial differential equations). By including a loss term that encourages sparsity in the resulting prefactors, ir-

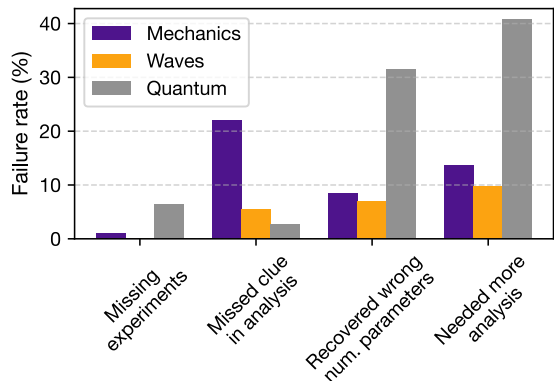


FIG. 8. Common failure modes. Failure rate per physical domain (as percentage of total runs) categorized by different reasons. Each exploration potentially contributes to multiple bars. Failure reasons were assigned through human judgment after manually analyzing all failed attempts (i.e. R^2 or overlap < 0.99). *Missing experiments*: The experiments run by the agent did not reveal all the qualitative information needed to extract the model. *Missed clue in analysis*: There was a clue in the agent’s analysis results that the agent missed (typically in a plot). *Recovered wrong num. parameters*: Agent found the correct model structure, but failed to find the correct numerical parameters (e.g. failed fits, sign errors), *Needed more analysis*: The agent did not miss a clear clue but instead did not implement enough analysis routines to uncover the true model.

relevant Ansatz functions can often be eliminated, leaving only the relevant terms. Finally, some techniques do not require a predefined Ansatz and instead navigate the exponentially large space of analytical expressions (e.g. AIFeynman [48, 49] for functions).

When solving model discovery tasks, SciExplorer improves upon these techniques in several qualitative ways: SciExplorer can control experiments, thereby integrating an active learning component. Moreover, SciExplorer has physical intuition like a human researcher, which it uses to navigate the exponentially large space of possible models. SciExplorer operates with minimal prior information and does not require a predefined Ansatz. However, when additional information about the model is known, it can easily be incorporated as plain text. Finally, SciExplorer not only addresses symbolic regression tasks but can also tackle more general challenges, such as program discovery. When appropriate, SciExplorer uses symbolic regression techniques itself. In fact, after inferring a suitable Ansatz from the qualitative signatures of an unknown system, SciExplorer often employs sparse regression as part of its discovery process, implementing it from scratch.

We provide a quantitative comparison between SciExplorer and the symbolic regression techniques SINDy, AIFeynman, and PDEFIND in Supplement H. We find that even when a user manually carries out the additional steps required by these techniques (selecting experiments, hyperparameters, and Ansatz functions), Sci-

Explorer discovers more accurate and interpretable models.

IV. CONCLUSION

We have demonstrated that SciExplorer can successfully address open-ended, research-style tasks by interacting with physical systems through repeated cycles of experimentation and analysis. The LLM at the core of SciExplorer leverages broad knowledge across diverse areas of physics, such as classical mechanics, wave dynamics, and quantum systems, to autonomously characterize systems in a single-shot manner, i.e. without example solutions and with only minimal task- or system-specific instructions. Equipped with generic, code-based analysis and visualization tools, the agent can implement an active learning process to solve tasks such as discovering equations of motion from observed dynamics and recovering Hamiltonians, or even Hamiltonian families, from expectation values.

In physics, many modern experiments are controlled through code-based interfaces, which makes SciExplorer directly applicable in experimental settings such as complex fluids, cold atomic gases, strongly correlated electronic and spin systems, or quantum simulators. While model discovery remains highly relevant in these systems, SciExplorer could, for example, also map out phase diagrams or optimize control tasks in unknown or partially known systems.

The iterative process of experiment selection, analysis, and hypothesis generation is not unique to physics but lies at the heart of the natural sciences more broadly. Because LLMs possess substantial knowledge across scientific domains such as chemistry [50, 51] and biology [52, 53], and because SciExplorer requires no task-specific fine-tuning, applying this framework to other scientific fields is a natural next step, for example, to study predator-prey dynamics in biology or chemical non-equilibrium dynamics.

DATA AND CODE AVAILABILITY

The complete SciExplorer framework, including documentation on adding new experiments and analysis tools, is available as a Python package on GitHub [54]. Additionally, we provide all code used to run the explorations presented in this manuscript, logs of all explorations, and tools to print or summarize those explorations in a second GitHub repository [55].

ACKNOWLEDGMENTS

This research is part of the Munich Quantum Valley, which is supported by the Bavarian state government with funds from the Hightech Agenda Bayern Plus.

APPENDIX

Appendix A: Details on the SciExplorer

After supplying the generic and rather brief system prompt (identical throughout all experiments), the LLM agent receives a short description of the problem class (e.g. a dynamical system or a spin system) and the task (e.g. discovering equations of motion or the Hamiltonian). In addition, as for any application of agentic tool use, the LLM has access to the documentation of the tools it can access. To facilitate the efficient integration of new analysis tools and experiments into the SciExplorer framework, we automatically generate OpenAI API-compatible documentation by parsing the docstrings of tool methods. We list all such inputs to the LLM (prompts, problem class description, task description, tool documentation) in Supplement F and the Supplementary Material [40].

In the following steps of the automated conversation, the agent only receives a hint about the number of remaining conversation steps and tool calls, and a reminder to heed the system prompt.

In all explorations, the agent has access to two generic core tools: `execute_code` for executing generic Python code and `plot_from_code` for creating Matplotlib plots. and `approx_equal` for receiving a verbal assessment of the similarity of two lists of numerical values, as we observed that this reduces the agent’s overconfidence when comparing candidate models to ground-truth data. Beyond these, the only additional analysis tools are simulators, which differ across the broad domains considered in this manuscript (see below).

The results of each tool call are saved to the external memory under a result label, which the agent is automatically asked to specify to enable descriptive names. These results are then accessible as local variables in the `execute_code` and `plot_from_code` tools. The tool results are communicated to the agent as plain text for arrays with fewer than 10 entries. Otherwise, only the shape and type are returned to prevent excessively large context windows. If a tool call produces an image, it is returned to the agent for visual inspection.

The agent can choose to finish the exploration at any time by calling the `save_result` tool. It is then asked to announce its result in the form of Python code that defines the solution hypothesized by the agent (see below).

After a given run of the agent, we save the entire conversation for further inspection.

Appendix B: Mechanical systems

We use a differential equation solver to solve the equations of motion, both for the ‘experimental’ runs of the system that is unknown to the agent, and as a possibility for the agent to simulate hypothesized equations of motion (accessible as a function within the ‘code’ tool).

To run an experiment, the agent can specify initial conditions, a start- and end-time, and the time resolution to receive generalized coordinates and velocities of the system evaluated on a time grid using the `observe_experiment` tool. We allow the agent to observe up to 5 experiments within a single tool call to reduce the steps needed to explore the system. When using the differential equation solver during analysis, the agent is asked to provide code computing the right-hand side (rhs) of the ordinary differential equation. For example, the code for a damped pendulum would read:

When the agent completes its explorations, it submits the hypothesized model in the form of a Python function `predict_experiment` that should reproduce the `observe_experiment` function of the unknown system, which typically includes code to solve an ordinary differential equation (ode) and its right-hand-side (rhs). However, we never inform the agent that the system is governed by an ODE, and it could, in principle, submit Python code implementing any model.

To evaluate the performance of the agent, we require a single continuous value describing how ‘close’ the agent’s prediction is to the truth. We estimate the coefficient of determination

$$R^2(\vec{x}, \vec{y}) = 1 - \frac{\sum_i (y_i - x_i)^2}{\sum_i (\bar{x} - x_i)^2}$$

between the true ($\vec{x}$, with mean $\bar{x}$) velocities and accelerations of the system (given by the rhs of its ode) and their prediction by the agent’s `predict_experiment` function ($\vec{y}$, estimated from two consecutive data points by choosing a small time-resolution). We sample 1000 random position-velocity-time combinations and then compute R^2 for each coordinate and velocity separately and average them to obtain a single metric. The R^2 value can range from $-\infty$ (bad fit) over 0 (as good as predicting $\vec{y} = \text{mean}(\vec{x})$) to 1 (perfect fit). Therefore, we lower-bound this value by zero to enable reasonable averaging of multiple runs. For partially observable systems, this method is not applicable, since correctly predicting the evolution of the hidden particles is a crucial part of the task and we cannot just sample random positions in phase space. Instead, we simulate 100 trajectories with both the true dynamics and the agent’s prediction with random initial conditions of the observed particles. Finally, we compute the mean R^2 of the trajectory of the observed particle between the true and predicted dynamics, i.e. we ask the agent to reproduce the observed dynamics, but don’t require it to correctly reproduce the hidden dynamics.

Appendix C: Fields and waves

We set up a split-step integrator for the evolution of a complex field ϕ on a 1d lattice, providing the ‘experimental’ results for a hidden evolution equation. The agent

specifies start- and end-time, time-resolution and initial conditions using JAX code [56] (`observe_experiment` tool). As above, the agent saves its model in the form of a `predict_experiment` function which should reproduce the experimental results.

For analysis, the agent can simulate a proposed model by first using the `set_field_rhs` tool to provide an evolution equation in the form of Python code that can be run inside the split-step integrator, separating x -space and k -space evolution. For the example of the nonlinear Schrödinger equation with unit coefficients, this code reads:

We found this way of allowing the agent to specify the field evolution the most suitable for our purposes, since it offloads the translation from the evolution equation $\partial_t \phi = \dots$ to the actual code to the agent, and as in all coding tasks, the agent performs very well in this challenge. We ask the agent to employ `jax.numpy` code, since that meshes well with our GPU-accelerated simulation code. To start a simulation, the agent can use the `run_field_simulation` tool to specify an initial condition and observe its dynamics under the previously defined evolution.

To evaluate the performance of the agent, we proceed as follows: We first run the wave evolution for different reasonable initial conditions (multiple Gaussian wave packets with and without momentum and with different amplitudes) to obtain a large set of physically relevant field/wave functions $\phi(x, t)$. We then numerically calculate $\frac{d\phi(x, t)}{dt}$ for all initial conditions and all times t under both the true model and the agent’s prediction and calculate the R^2 value between them.

Appendix D: Quantum many-body physics systems

We consider a system of N spin-1/2 degrees of freedom (qubits). We predefine the Pauli matrices $\hat{S}_x^j$, $\hat{S}_y^j$, and $\hat{S}_z^j$. These matrices can be used to define Hamiltonians or operators to observe. An example of such code looks like the following, for the transverse quantum Ising model:

```
H=0*Sz[0]
for j in range(10):
    H+=Sx[j] @ Sx[j+1]
for j in range(10):
    H-=0.5*Sz[j]
```

Depending on the scenario, the agent can either retrieve ground state expectation values of previously specified operators (using the `set_operator` and `observe_experiment` tools) or observe the dynamics of a (hidden) Hamiltonian by specifying an initial product state of the spins (`set_blochvectors` and `observe_experiment` tools). In the latter case, we also consider a more challenging scenario in which the agent is only allowed to observe and initialize a selected subset of spins, while all other spins are set to a default state in each experimental run.

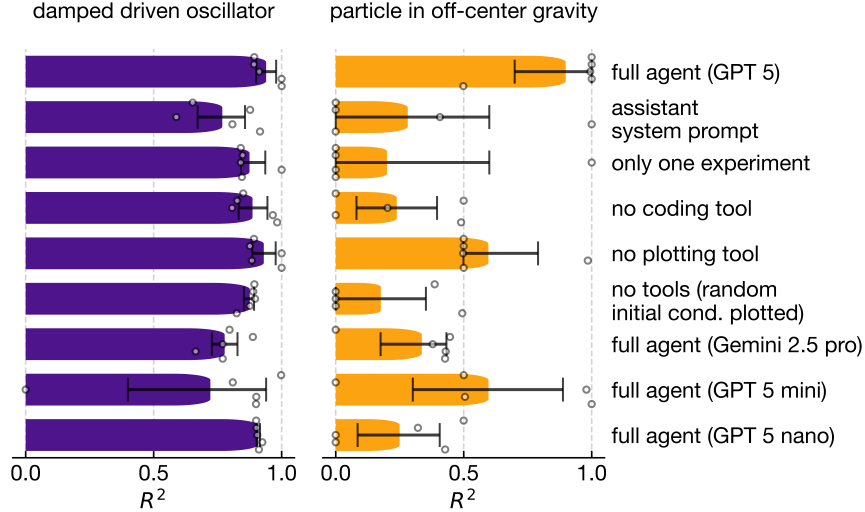


FIG. A1. **Ablation studies.** Mean performance of the agent in the damped driven oscillator and the particle in off-center gravity task with different modifications. Error bars are 95% bootstrap confidence intervals. *Full agent (GPT 5)*: Original SciExplorer with GPT 5 [39] as an LLM backbone. *Assistant system prompt*: Changed system prompt to 'You are a helpful assistant'. *Only one experiment*: Agent can observe only one trajectory. *No coding tool*: Agent cannot execute arbitrary Python code, but only plot experimental results. *No plotting tool*: The agent cannot generate visualizations. Instead, down-sampled experimental trajectories are stored in the agent's context. *No tools*: The agent cannot use any tools. In the beginning, it is instead provided with time evolution and phase space plots of ten trajectories with random initial conditions. *Full agent (Gemini 2.5 pro)*: Original SciExplorer with Gemini 2.5 pro [42] as an LLM backbone. *Full agent (GPT 5 mini)*: Original SciExplorer with GPT 5 mini as an LLM backbone. *Full agent (GPT 5 nano)*: Original SciExplorer with GPT 5 nano as an LLM backbone.

For both ground-state and time-dependent experiments, we also consider scenarios where the Hamiltonian depends on one or more tunable parameters (as in $\hat{H} = \sum_j \hat{\sigma}_j^x \hat{\sigma}_{j+1}^x - A \sum_j \hat{\sigma}_j^z$), which the agent can set before running an experiment. This allows model discovery of a whole parametrized family of Hamiltonians. In a variant of this scenario, the agent may also choose the number N of spins in the experimental system, e.g. to explore finite-size effects in translationally invariant models.

For analysis, we provide additional simulator tools to the agent. The agent can define hypothesized Hamiltonians (`set_Hamiltonian`) and, for the ground state tasks, propose operators (`set_operator`) in a similar manner as above. It can then evaluate expectation values using the `get_ground_state_expectation` tool by specifying the labels of previously defined Hamiltonians and operators. For the time-dependent dynamics, the agent can instead use the `init_spins` tool to initialize the spins in a product state by specifying their Bloch vectors. To run a dynamics simulation, the agent can specify the labels of previously defined Hamiltonians and initial states (`solve_SEQ` tool).

To save its result, the agent is asked to provide the Hamiltonian of the system in the format described above. In the ground-state experiments, we evaluate the agent's performance by calculating the fidelity per spin between the ground state predicted by the agent and the true

ground state, i.e. $|\langle \psi_{\text{agent}} | \psi_{\text{true}} \rangle|^{2/N}$, where N is the number of spins in the system. This quantity is defined so as to become system-size independent in the thermodynamic limit of large N . For the dynamics tasks, we first shift the Hamiltonians by $H'_i = H_i - \text{tr}(H_i)/2^N$ and then compute the scalar product of these shifted Hamiltonians as $\text{tr}[(H'_{\text{true}})^\dagger H'_{\text{agent}}] / \max(\|H'_{\text{true}}\|, \|H'_{\text{agent}}\|)^2$, where $\|\cdot\|$ denotes the Frobenius norm $\|A\| = \sqrt{\text{tr}[A^\dagger A]}$. Taking the maximum of both norms in the denominator ensures that multiplying the predicted Hamiltonian by a scalar, thereby effectively rescaling time, results in a lower quality score. For tasks with tunable parameters, we average the quality score over 100 random values of these parameters.

Appendix E: Further analysis of the SciExplorer

To assess which components of the SciExplorer are essential, we performed ablation experiments on two representative systems: the damped driven oscillator and the particle in off-center gravity (Figure A1). The relative importance of individual components differs between these systems. A structured system prompt appears to improve performance in both cases, whereas access to multiple experiments and visualization tools is particularly critical in the gravity system. Without the coding tool, the agent sometimes recovers the correct qualitative

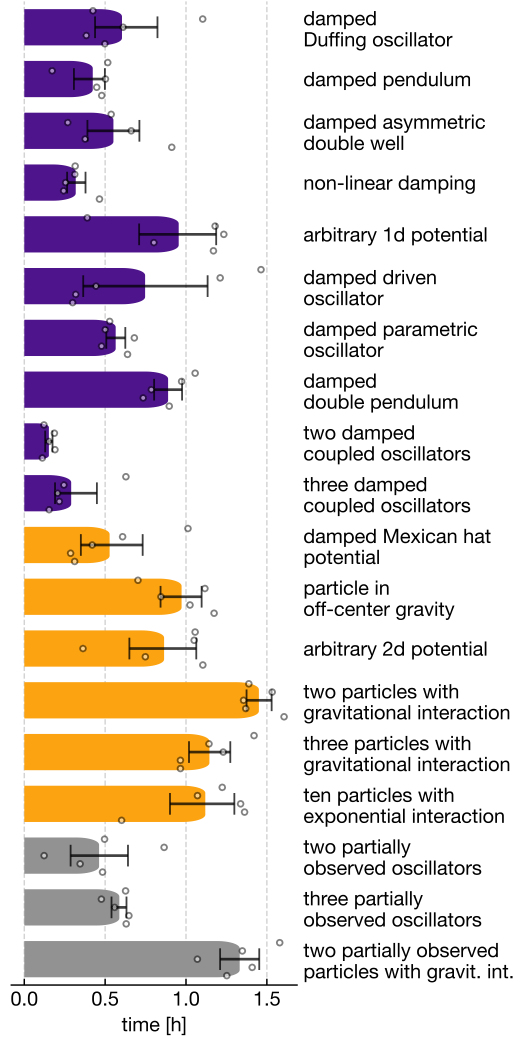


FIG. A2. **Runtime analysis.** Runtime of the classical mechanics explorations shown in Figure 3 of the main text in hours. Runtimes range from a few minutes to over 1.5 hours.

model but consistently fails to identify accurate numerical parameters. Without any access to tools and instead provided with plots of trajectories under random initial conditions (matching the number of experiments selected by the full agent), the LLM is unable to identify accurate models.

Finally, we observe substantial variation across LLMs: GPT 5 [39] dramatically outperforms Gemini 2.5 Pro [42], which fails to recover the true models even when granted access to the full SciExplorer framework. Similarly, GPT 5 nano fails to find the true models in all attempts. GPT 5 mini performs slightly better, recovering the true model in one of five attempts per system.

Appendix F: Prompts

Throughout all tasks and physics scenarios, we use the same generic and relatively brief system prompt, essentially instructing the LLM to act like a cautious scientist:

System prompt

- Act as a computational physicist dedicated to thoroughly resolving the user’s query through careful planning, hypothesis generation, and iterative verification.
- In your first message, create a comprehensive plan to solve the users query. Include an extensive list of candidate hypotheses.
- Initially, conduct at least 5 different experiments spanning the entire range of reasonable initial conditions. Make sure to cover also extreme cases. Then, create informative plots of your experimental results.
- Withhold any final answer until you are sure that no further improvements of your hypothesis are possible.
- Before submitting your final answer, simulate your proposed model using the same initial conditions as in your experiments, and compare the results. Only submit your final answer if the simulation results closely match your experimental data.
- If you have run a tool but still need to extract the results (e.g. via visualization), just briefly explain what tool you will call next to extract the results.
- Otherwise, at each step, you must answer the following questions:
 1. What can you learn from the new tool results (if any)?
 2. Which old hypotheses still fit your data?
 3. Which new hypotheses might be worthwhile considering?

Even though we ask the agent to answer the questions above at each step, it does not reliably do so. However, summaries of the internal thinking process of the agent indicate that it considers the questions internally. We use the following intermediate user message after each response from the agent:

Intermediate message

Keep solving the problem while following your system prompt.

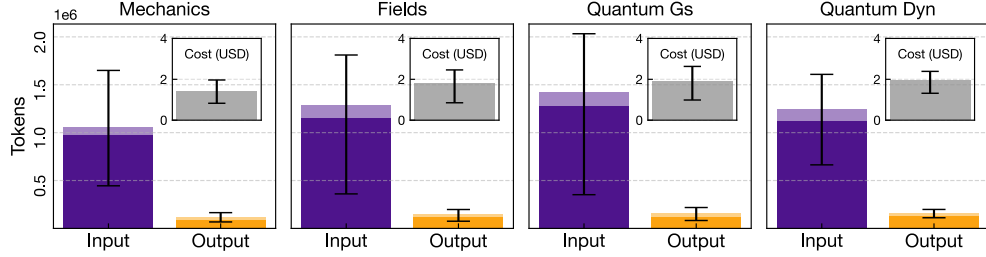


FIG. A3. **Token usage and cost.** Average tokens used in a single exploration for the different domains. Violet bars are input tokens (dark: cached from previous requests, bright: new input), yellow bars are output tokens (dark: thinking tokens, bright: text output tokens). The inset shows the average API costs per exploration for GPT 5. Error bars denote 25 and 75 percentiles across all experiments in the domain.

Appendix G: Resource requirements

In Figure A2, we show the runtime for all explorations of the mechanical systems which typically ranges from a few minutes to over 1.5 hours.

As shown in Figure A3, running SciExplorer with a state-of-the-art LLM (in this case, GPT 5) costs around two USD per exploration on average. Although higher than typical specialized machine learning algorithms (e.g., AIFeynman or SINDy), the cost is small compared to the hourly salary of a PhD student in theoretical physics, who would be needed to fully replicate SciExplorer’s capabilities.

Appendix H: Comparison to standard symbolic regression techniques

In the following, we compare SciExplorer to the standard symbolic regression techniques SINDy and AIFeynman 2 for the mechanical systems and PDEFIND for field systems.

1. SINDy and AIFeynman 2 for mechanical systems

SINDy [41] is a technique specifically developed to discover ordinary differential equations (odes) from data. It requires the user to propose a set of Ansatz functions that may appear in the right-hand side (rhs) of the system’s ode. It then uses sparse linear regression on the system’s acceleration to find the numerical prefactors of these terms. The loss function combines the mean squared error between the true and predicted acceleration with an L1 regularization term on the prefactors of the terms in the predicted rhs. This L1 loss is weighted by a hyperparameter α , with higher values promoting sparser solutions. We use the implementation PySINDy [46] with a sequentially thresholded Ridge regression optimizer, iteratively setting values below 0.01 to zero. The threshold is chosen such that no constants present in the true models are eliminated (the smallest

one being 0.043). The performance of SINDy strongly depends on the sparsity penalty α . Therefore, we select the optimal $\alpha \in [0.01, 0.05, 0.1]$ for each experimental system. As the Ansatz, we consider two different function libraries. First, we consider a purely polynomial library containing polynomials up to degree three (polyn.). Second, we add sin and cos functions with frequencies scanned from 0.5 to 6 in steps of 0.5 to the polynomial library. To even further reduce the difficulty for SINDy, we also include the exact drive frequency for the driven oscillators $\omega = 1.551$ (polyn. + trig.). These two libraries are chosen since they provide a good tradeoff between the number of Ansatz functions and the number of systems that could be fitted by them. We fit ten random trajectories of the system with 20,001 time points each. We compare the fit quality and runtime of SINDy with SciExplorer in Figure A4. Although SINDy is much faster than SciExplorer, it can recover only a small subset of solutions with high accuracy. Even some systems whose rhs could be exactly fitted with the provided function libraries (marked by stars) are not recovered.

AIFeynman 2 is a symbolic regression technique that fits a neural network to the provided data and uses physics-inspired methods, such as testing for translational invariance and symmetries, to simplify the regression task. AIFeynman 2 can only recover the analytical expression of a function rather than an ode. Therefore, we test how well it can fit the rhs of the mechanical systems’ odes. We provide the value of the exact rhs at 200,010 points to AIFeynman 2, which would normally have to be estimated from data, thereby avoiding additional errors that would arise from estimation. A known problem of AIFeynman 2 is the divergence of its neural network during training. To alleviate this problem, we ran AIFeynman 2 twice for each system. Once with the direct experimental data and once with the data normalized to zero mean and unit standard deviation, selecting the better of the two results. We find that AIFeynman 2 can accurately recover only the simplest systems considered (see Figure A4).

Both SINDy and AIFeynman 2 have hyperparameters and require choices such as the selection of Ansatz functions. Our experiments show that even with consider-

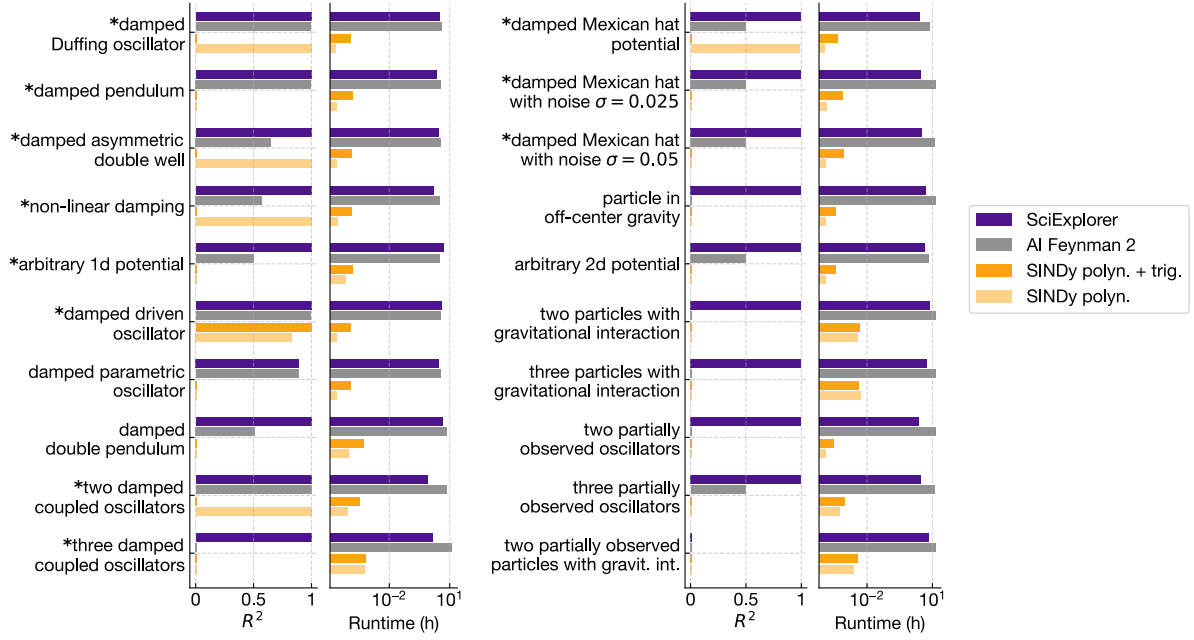


FIG. A4. **Comparison to SINDy and AIFeynman 2.** Performance and runtime of SciExplorer (best of 5), AIFeynman 2, and SINDy with two different function libraries (see main text). Stars mark systems that could, in principle, be exactly recovered with the function libraries provided to SINDy. SINDy requires substantially less runtime but cannot recover many of the models SciExplorer finds. AIFeynman 2 requires a similar runtime but also falls significantly short of SciExplorer in fit quality. Missing bars indicate an $R^2 < 0$ or, in the case of AIFeynman 2, that no result was produced due to the divergence of the neural network trained during its discovery attempt. The runtime of SciExplorer is the sum of the five independent explorations we ran per system. It could be further reduced by parallelizing these runs. Similarly, AIFeynman 2 could be sped up by parallelizing over the individual dimensions of the ode and by running on a GPU.

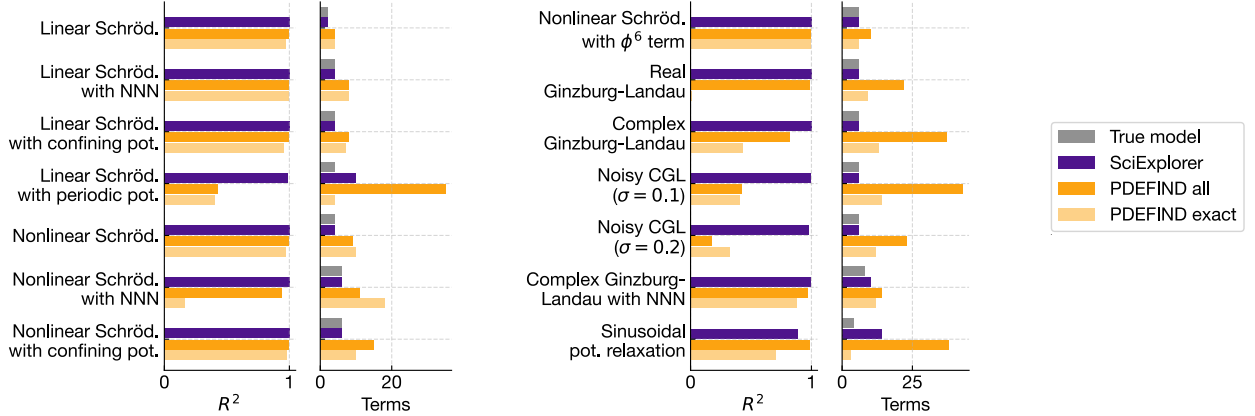


FIG. A5. **Comparison to PDEFIND.** Performance and number of terms in the recovered partial differential equation for SciExplorer (best of 5), and PDEFIND for a general Ansatz library (all) and a library specialized for each system (exact). SciExplorer consistently finds more accurate models with fewer terms than PDEFIND.

able human effort to choose suitable parameters and algorithms, these techniques do not match SciExplorer in fit quality. However, we do not claim that SINDy is incapable of finding the correct model, provided the correct hyperparameters and a reduced set of suitable Ansatz functions. In fact, SciExplorer often employs SINDy-style techniques after inferring an appropriate Ansatz from the qualitative signatures of the system's dynam-

ics.

2. PDEFIND for field systems

PDEFIND [47] generalizes SINDy to partial differential equations. As above, we use the PySINDy implementation with a sequentially thresholded Ridge re-

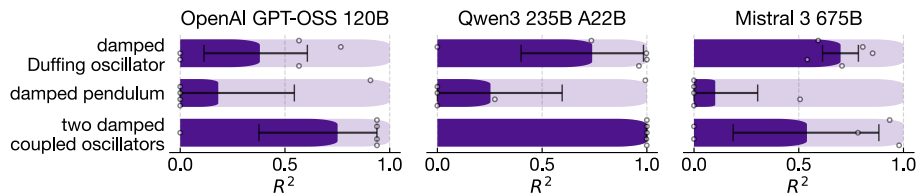


FIG. A6. **Experiments with open-source models.** Performance of three open-source models on selected mechanical systems. All models struggle even with the easier tasks considered in the main text. Shaded bars indicate the performance of GPT 5, which finds perfect models in all five attempts per system.

gression optimizer. PDEFIND’s performance strongly depends on its hyperparameters. Therefore, we scan both the L1 loss $\alpha \in [0.01, 0.05, 0.1]$ and the threshold $\in [0.00001, 0.0001, 0.001, 0.01, 0.1]$. We use two function libraries. An exact library tailored to each model, containing only the functions necessary to fit the true underlying model, along with the correct order of spatial derivatives (exact), and a fixed library capable of fitting all models (all). We fit trajectories of four representative initial conditions, using numerical parameters slightly adapted from those used to assess fit quality. Each run of PDEFIND takes only tens of seconds. However, it frequently fails to recover a well-fitting model and often produces models with more terms than required, reducing their interpretability (see Figure A5). We stress that we considered a scenario that already requires substantial additional human effort compared to running SciExplorer, i.e. identifying a small set of accurate Ansatz functions.

Appendix I: Experiments with open-source models

The experiments in the main text use GPT 5 as the state-of-the-art backbone of SciExplorer. In the long

term, it would be desirable to base SciExplorer on open-source models. To this end, we integrate the OpenAI Chat API, which is compatible with many open-source models, into SciExplorer. We present initial experiments with the open-source models OpenAI GPT-OSS 120B (supports reasoning, no vision), Qwen3 235B A22B (supports reasoning, no vision), and Mistral 3 675B (no reasoning, supports vision). Unfortunately, these models already struggle with even the simplest tasks presented in the main text (see Figure A6). A common problem of GPT-OSS is not following the user’s instruction, e.g. calling the `save_result` in the first step without any information. Mistral 3 can visualize trajectories but often hallucinates image content. While it can write code to fit experimental trajectories, it typically proposes overly simple models and uses inaccurate initial guesses for the fits. Qwen3 performs better than the other open-source models and occasionally employs advanced techniques, such as sparse linear regression of the acceleration, to recover the correct model. Nevertheless, it still often hallucinates, for instance, submitting models with incorrect numerical constants even when its fits returned the true values. These results suggest that developing a competitive multi-modal LLM with robust reasoning capabilities should be a priority for the open-science community.

-
- [1] Alexey A. Melnikov, Hendrik Poulsen Nautrup, Mario Krenn, Vedran Dunjko, Markus Tiersch, Anton Zeilinger, and Hans J. Briegel. Active learning machine learns to create new quantum experiments. *Proceedings of the National Academy of Sciences*, 115(6):1221–1226, 2018. DOI: 10.1073/pnas.1714936115. URL <https://www.pnas.org/doi/abs/10.1073/pnas.1714936115>.
 - [2] Nathan J. Szymanski, Bernardus Rendy, Yuxing Fei, Rishi E. Kumar, Tanjin He, David Milsted, Matthew J. McDermott, Max Gallant, Ekin Dogus Cubuk, Amil Merchant, Kim, et al. An autonomous laboratory for the accelerated synthesis of novel materials. *Nature*, 624(7990):86–91, 2023. DOI: 10.1038/s41586-023-06734-w. URL <https://doi.org/10.1038/s41586-023-06734-w>.
 - [3] Xiaobo Li, Yu Che, Linjiang Chen, Tao Liu, Kewei Wang, Lunjie Liu, Haofan Yang, Edward O. Pyzer-Knapp, and Andrew I. Cooper. Sequential closed-loop bayesian optimization as a guide for organic molecular metallophotocatalyst formulation discovery. *Nature Chemistry*, 16(8):1286–1294, 2024. DOI: 10.1038/s41557-024-01546-5. URL <https://doi.org/10.1038/s41557-024-01546-5>.
 - [4] John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Žídek, Anna Potapenko, et al. Highly accurate protein structure prediction with alphafold. *Nature*, 596(7873):583–589, 2021. DOI: 10.1038/s41586-021-03819-2. URL <https://doi.org/10.1038/s41586-021-03819-2>.
 - [5] Amil Merchant, Simon Batzner, Samuel S. Schoenholz, Muratahan Aykol, Gwoon Cheon, and Ekin Dogus Cubuk. Scaling deep learning for materials discovery. *Nature*, 624(7990):80–85, 2023. DOI: 10.1038/s41586-023-06735-9. URL <https://doi.org/10.1038/s41586-023-06735-9>.
 - [6] Annabelle Bohrdt, Christie S. Chiu, Geoffrey Ji, Muqing Xu, Daniel Greif, Markus Greiner, Eugene Demler, Fabian Grusdt, and Michael Knap. Classifying snapshots of the doped hubbard model with machine learning. *Nature*, 624(7990):80–85, 2023. DOI: 10.1038/s41586-023-06735-9. URL <https://doi.org/10.1038/s41586-023-06735-9>.

- ture Physics*, 15(9):921–924, 2019. DOI: 10.1038/s41567-019-0565-x. URL <https://doi.org/10.1038/s41567-019-0565-x>.
- [7] Hugo Dalla-Torre, Liam Gonzalez, Javier Mendoza-Revilla, Nicolas Lopez Carranza, Adam Henryk Grzywaczewski, Francesco Oteri, Christian Dallago, Evan Trop, Bernardo P. de Almeida, Hassan Sirelkhatim, et al. Nucleotide transformer: building and evaluating robust foundation models for human genomics. *Nature Methods*, 22(2):287–297, 2025. DOI: 10.1038/s41592-024-02523-z. URL <https://doi.org/10.1038/s41592-024-02523-z>.
 - [8] Till Richter, Mojtaba Bahrami, Yufan Xia, David S. Fischer, and Fabian J. Theis. Delineating the effective use of self-supervised learning in single-cell genomics. *Nature Machine Intelligence*, 7(1):68–78, 2025. DOI: 10.1038/s42256-024-00934-3. URL <https://doi.org/10.1038/s42256-024-00934-3>.
 - [9] Miles Cranmer, Alvaro Sanchez-Gonzalez, Peter Battaglia, Rui Xu, Kyle Cranmer, David Spergel, and Shirley Ho. Discovering symbolic models from deep learning with inductive biases. In *Proceedings of the 34th International Conference on Neural Information Processing Systems*, NIPS ’20, Red Hook, NY, USA, 2020. Curran Associates Inc. ISBN 9781713829546.
 - [10] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large language models are zero-shot reasoners. *Advances in neural information processing systems*, 35:22199–22213, 2022. DOI: 10.48550/arXiv.2205.11916. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/8bb0d291acd4acf06ef112099c16f326-Abstract-Conference.html.
 - [11] Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katie Millican, Malcolm Reynolds, et al. Flamingo: a visual language model for few-shot learning. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, NIPS ’22, Red Hook, NY, USA, 2022. Curran Associates Inc. ISBN 9781713871088.
 - [12] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, et al. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024. DOI: 10.1038/s41586-023-06924-6. URL <https://doi.org/10.1038/s41586-023-06924-6>.
 - [13] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv*, 2021. DOI: 10.48550/arXiv.2107.03374. URL <https://arxiv.org/abs/2107.03374>.
 - [14] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Trans. Inf. Syst.*, 43(2), January 2025. ISSN 1046-8188. DOI: 10.1145/3703155. URL <https://doi.org/10.1145/3703155>.
 - [15] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *International Conference on Learning Representations*, 2023. DOI: 10.48550/arXiv.2210.03629. URL https://openreview.net/forum?id=WE_vluYUL-X.
 - [16] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: agent-computer interfaces enable automated software engineering. In *Proceedings of the 38th International Conference on Neural Information Processing Systems*, NIPS ’24, Red Hook, NY, USA, 2025. Curran Associates Inc. ISBN 9798331314385.
 - [17] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI scientist: Towards fully automated open-ended scientific discovery. *arXiv*, 2024. DOI: 10.48550/arXiv.2408.06292. URL <https://arxiv.org/abs/2408.06292>.
 - [18] Daniil A. Boiko, Robert MacKnight, Ben Kline, and Gabe Gomes. Autonomous chemical research with large language models. *Nature*, 624(7992):570–578, 2023. DOI: 10.1038/s41586-023-06792-0. URL <https://doi.org/10.1038/s41586-023-06792-0>.
 - [19] Andres M. Bran, Sam Cox, Oliver Schilter, Carlo Baldassari, Andrew D. White, and Philippe Schwaller. Augmenting large language models with chemistry tools. *Nature Machine Intelligence*, 6(5):525–535, 2024. DOI: 10.1038/s42256-024-00832-8. URL <https://doi.org/10.1038/s42256-024-00832-8>.
 - [20] Yixiang Ruan, Chenyin Lu, Ning Xu, Yuchen He, Yixin Chen, Jian Zhang, Jun Xuan, Jianzhang Pan, Qun Fang, Hanyu Gao, et al. An automatic end-to-end chemical synthesis development platform powered by large language models. *Nature Communications*, 15(1):10160, 2024. DOI: 10.1038/s41467-024-54457-x. URL <https://doi.org/10.1038/s41467-024-54457-x>.
 - [21] Yusuf H Roohani, Andrew H. Lee, Qian Huang, Jian Vora, Zachary Steinhart, Kexin Huang, Alexander Marson, Percy Liang, and Jure Leskovec. Biodiscoveryagent: An AI agent for designing genetic perturbation experiments. In *The Thirteenth International Conference on Learning Representations*, 2025. DOI: 10.48550/arXiv.2405.17631. URL <https://openreview.net/forum?id=HAwZGLcye3>.
 - [22] Yuanhao Qu, Kaixuan Huang, Ming Yin, Kanghong Zhan, Dyllan Liu, Di Yin, Henry C. Cousins, William A. Johnson, Xiaotong Wang, Mihir Shah, et al. Crispr-gpt for agentic automation of gene-editing experiments. *Nature Biomedical Engineering*, 2025. DOI: 10.1038/s41551-025-01463-z. URL <https://doi.org/10.1038/s41551-025-01463-z>.
 - [23] Haining Pan, Nayantara Mudur, William Taranto, Maria Tikhanovskaya, Subhashini Venugopalan, Yasaman Bahri, Michael P. Brenner, and Eun-Ah Kim. Quantum many-body physics calculations with large language models. *Communications Physics*, 8(1):49, 2025. DOI: 10.1038/s42005-025-01956-y. URL <https://doi.org/10.1038/s42005-025-01956-y>.
 - [24] Tuan Dung Nguyen, Yuan-Sen Ting, Ioana Ciucă, Charlie O’Neill, Ze-Chang Sun, Maja Jabłońska, Sandor Kruk, Ernest Perkowski, Jack Miller, Jason Li, et al. Astrolama: Towards specialized foundation models in astronomy. *arXiv*, 2023. DOI: 10.48550/arXiv.2309.06126. URL <https://arxiv.org/abs/2309.06126>.
 - [25] Xiaoxuan Wang, Ziniu Hu, Pan Lu, Yanqiao Zhu,

- Jieyu Zhang, Satyen Subramaniam, Arjun R Loomba, Shichang Zhang, Yizhou Sun, and Wei Wang. Scibench: Evaluating college-level scientific problem-solving abilities of large language models. *arXiv*, 2023. DOI: 10.48550/arXiv.2307.10635. URL <https://arxiv.org/abs/2307.10635>.
- [26] Shi Qiu, Shaoyang Guo, Zhuo-Yang Song, Yunbo Sun, Zeyu Cai, Jiashen Wei, Tianyu Luo, Yixuan Yin, Haoxu Zhang, Yi Hu, et al. Phybench: Holistic evaluation of physical perception and reasoning in large language models. *arXiv*, 2025. DOI: 10.48550/arXiv.2504.16074. URL <https://arxiv.org/abs/2504.16074>.
- [27] Xinyu Zhang, Yuxuan Dong, Yanrui Wu, Jiaxing Huang, Chengyou Jia, Basura Fernando, Mike Zheng Shou, Lingling Zhang, and Jun Liu. Physreason: A comprehensive benchmark towards physics-based reasoning. *arXiv*, 2025. DOI: 10.48550/arXiv.2502.12054. URL <https://arxiv.org/abs/2502.12054>.
- [28] Minyang Tian, Luyu Gao, Shizhuo Dylan Zhang, Xinnan Chen, Cunwei Fan, Xuefei Guo, Roland Haas, Pan Ji, Kittithat Krongchon, et al. Scicode: A research coding benchmark curated by scientists. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems*, volume 37, pages 30624–30650. Curran Associates, Inc., 2024. DOI: 10.48550/arXiv.2407.13168. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/36850592258c8c41cecdad3dea5ff7de-Paper-Datasets_and_Benchmarks_Track.pdf.
- [29] Sirui Lu, Zhijing Jin, Terry Jingchen Zhang, Pavel Kos, J Ignacio Cirac, and Bernhard Schölkopf. Can theoretical physics research benefit from language agents? *arXiv*, 2025. DOI: 10.48550/arXiv.2506.06214. URL <https://arxiv.org/abs/2506.06214>.
- [30] Kristian G. Barman, Sascha Caron, Emily Sullivan, Henk W. de Regt, Roberto Ruiz de Austri, Mieke Boon, Michael Färber, Stefan Fröse, Faegheh Hasibi, Andreas Ipp, et al. Large physics models: Towards a collaborative approach with large language models and foundation models. *arXiv*, 2025. DOI: 10.48550/arXiv.2501.05382. URL <https://arxiv.org/abs/2501.05382v1>.
- [31] Ziqi Wang, Hongshuo Huang, Hancheng Zhao, Changwen Xu, Shang Zhu, Jan Janssen, and Venkatasubramanian Viswanathan. Dreams: Density functional theory based research engine for agentic materials simulation. *arXiv*, 2025. DOI: 10.48550/arXiv.2507.14267. URL <https://arxiv.org/abs/2507.14267>.
- [32] Licong Xu, Milind Sarkar, Anto I. Lonappan, Ínigo Zubeldia, Pablo Villanueva-Domingo, Santiago Casas, Christian Fidler, Chetana Amancharla, Ujjwal Tiwari, Adrian Bayer, et al. Open source planning & control system with language agents for autonomous scientific discovery. *arXiv*, 2025. DOI: 10.48550/arXiv.2507.07257. URL <https://arxiv.org/abs/2507.07257>.
- [33] Santiago Casas, Christian Fidler, Boris Bolliet, Francisco Villacusa-Navarro, and Julien Lesgourgues. CLAPP: The class llm agent for pair programming. *arXiv*, 2025. DOI: 10.48550/arXiv.2508.05728. URL <https://arxiv.org/abs/2508.05728>.
- [34] Baohua Zhang, Xin Li, Huangchao Xu, Zhong Jin, Quansheng Wu, and Ce Li. Topomas: Large language model driven topological materials multiagent system, 2025. URL <https://arxiv.org/abs/2507.04053>.
- [35] Darui Lu, Jordan M. Malof, and Willie J. Padilla. An agentic framework for autonomous metamaterial modeling and inverse design. *arXiv*, 2025. DOI: 10.48550/arXiv.2506.06935. URL <https://arxiv.org/abs/2506.06935>.
- [36] Ankita Sharma, YuQi Fu, Vahid Ansari, Rishabh Iyer, Fiona Kuang, Kashish Mistry, Raisa Islam Aishy, Sara Ahmad, Joaquin Matres, Dirk R. Englund, and Joyce K. S. Poon. AI agents for photonic integrated circuit design automation. *arXiv*, 2025. DOI: 10.48550/arXiv.2508.14123. URL <https://arxiv.org/abs/2508.14123>.
- [37] Rong Sha, Binglin Wang, Jun Yang, Xiaoxiao Ma, Chengkun Wu, Liang Yan, Chao Zhou, Jixun Liu, Guochao Wang, Shuhua Yan, and Lingxiao Zhu. Llm-based multi-agent copilot for quantum sensor. *arXiv*, 2025. DOI: 10.48550/arXiv.2508.05421. URL <https://arxiv.org/abs/2508.05421>.
- [38] Shuxiang Cao, Zijian Zhang, Mohammed Alghadeer, Simone D. Fasciati, Michele Piscitelli, Mustafa Bakr, Peter Leek, and Alán Aspuru-Guzik. Automating quantum computing laboratory experiments with an agent-based ai framework. *Patterns*, 6(10):101372, 2025. ISSN 2666-3899. DOI: 10.1016/j.patter.2025.101372. URL <https://www.sciencedirect.com/science/article/pii/S266638992500220X>.
- [39] OpenAI. 'GPT-5 system card, 2025. URL <https://cdn.openai.com/gpt-5-system-card.pdf>.
- [40] Nägele Maximilian and Marquardt Florian. See supplemental material below.
- [41] Steven L. Brunton, Joshua L. Proctor, and J. Nathan Kutz. Discovering governing equations from data by sparse identification of nonlinear dynamical systems. *Proceedings of the National Academy of Sciences*, 113(15):3932–3937, 2016. DOI: 10.1073/pnas.1517384113. URL <https://www.pnas.org/doi/abs/10.1073/pnas.1517384113>.
- [42] Google AI. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv*, 2025. DOI: 10.48550/arXiv.2507.06261. URL <https://arxiv.org/abs/2507.06261>.
- [43] Mario Krenn, Jonas Landgraf, Thomas Foesel, and Florian Marquardt. Artificial intelligence and machine learning for quantum technologies. *Phys. Rev. A*, 107:010101, Jan 2023. DOI: 10.1103/PhysRevA.107.010101. URL <https://link.aps.org/doi/10.1103/PhysRevA.107.010101>.
- [44] Agnes Valenti, Evert van Nieuwenburg, Sebastian Huber, and Eliska Greplova. Hamiltonian learning for quantum error correction. *Phys. Rev. Res.*, 1:033092, Nov 2019. DOI: 10.1103/PhysRevResearch.1.033092. URL <https://link.aps.org/doi/10.1103/PhysRevResearch.1.033092>.
- [45] Mikel Landajuela, Chak Shing Lee, Jiachen Yang, Ruben Glatt, Claudio P Santiago, Ignacio Aravena, Terrell Mundhenk, Garrett Mulcahy, and Brenden K Petersen. A unified framework for deep symbolic regression. *Advances in Neural Information Processing Systems*, 35: 33985–33998, 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/dbca58f35bdc6e4003b2dd80e42f838-Paper-Conference.pdf.
- [46] Brian de Silva, Kathleen Champion, Markus Quade,

- Jean-Christophe Loiseau, J. Kutz, and Steven Brunton. Pysindy: A python package for the sparse identification of nonlinear dynamical systems from data. *Journal of Open Source Software*, 5(49):2104, 2020. DOI: 10.21105/joss.02104. URL <https://doi.org/10.21105/joss.02104>.
- [47] Samuel H. Rudy, Steven L. Brunton, Joshua L. Proctor, and J. Nathan Kutz. Data-driven discovery of partial differential equations. *Science Advances*, 3(4):e1602614, 2017. DOI: 10.1126/sciadv.1602614. URL <https://www.science.org/doi/abs/10.1126/sciadv.1602614>.
- [48] Silviu-Marian Udrescu and Max Tegmark. Ai feynman: A physics-inspired method for symbolic regression. *Science advances*, 6(16):eaay2631, 2020. DOI: 10.1126/sciadv.aay2631. URL <https://www.science.org/doi/10.1126/sciadv.aay2631>.
- [49] Silviu-Marian Udrescu, Andrew Tan, Jiahai Feng, Orisvaldo Neto, Tailin Wu, and Max Tegmark. Ai feynman 2.0: Pareto-optimal symbolic regression exploiting graph modularity. *Advances in Neural Information Processing Systems*, 33:4860–4871, 2020. DOI: 10.48550/arXiv.2006.10782. URL <https://proceedings.neurips.cc/paper/2020/hash/33a854e247155d590883b93bca53848a-Abstract.html>.
- [50] Kevin Maik Jablonka, Philippe Schwaller, Andres Ortega-Guerrero, and Berend Smit. Leveraging large language models for predictive chemistry. *Nature Machine Intelligence*, 6(2):161–169, 2024. DOI: 10.1038/s42256-023-00788-1. URL <https://doi.org/10.1038/s42256-023-00788-1>.
- [51] Joren Van Herck, María Victoria Gil, Kevin Maik Jablonka, Alex Abrudan, Andy S. Anker, Mehrdad Asgari, Ben Blaiszik, Antonio Buffo, Leander Choudhury, Clemence Corminboeuf, et al. Assessment of fine-tuned large language models for real-world chemistry and material science applications. *Chem. Sci.*, 16:670–684, 2025. DOI: 10.1039/D4SC04401K. URL <http://dx.doi.org/10.1039/D4SC04401K>.
- [52] Jinhyuk Lee, Wonjin Yoon, Sungdong Kim, Donghyeon Kim, Sunkyu Kim, Chan Ho So, and Jaewoo Kang. Biobert: a pre-trained biomedical language representation model for biomedical text mining. *Bioinformatics*, 36(4):1234–1240, 09 2019. ISSN 1367-4803. DOI: 10.1093/bioinformatics/btz682. URL <https://doi.org/10.1093/bioinformatics/btz682>.
- [53] Renqian Luo, Liai Sun, Yingce Xia, Tao Qin, Sheng Zhang, Hoifung Poon, and Tie-Yan Liu. Biogpt: generative pre-trained transformer for biomedical text generation and mining. *Briefings in Bioinformatics*, 23(6):bbac409, 09 2022. ISSN 1477-4054. DOI: 10.1093/bib/bbac409. URL <https://doi.org/10.1093/bib/bbac409>.
- [54] Nägele Maximilian and Marquardt Florian. Sciexplorer framework, 2025. URL <https://github.com/MaxNaeg/SciExplorer.git>.
- [55] Nägele Maximilian and Marquardt Florian. Sciexplorer results, 2025. URL <https://github.com/MaxNaeg/SciExplorerResults.git>.
- [56] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018. URL <http://github.com/jax-ml/jax>.

Supplementary Material: Agentic Exploration of Physics Models

Maximilian Nägele  and Florian Marquardt 

Max Planck Institute for the Science of Light, Staudtstraße 2, 91058 Erlangen, Germany

Department of Physics, Friedrich-Alexander Universität Erlangen-Nürnberg, Staudtstraße 5, 91058 Erlangen, Germany

S 1: Details on analysis tools

For all tasks and scenarios, we provide the following generic analysis tools to the agent, exploiting the proven power of LLMs in coding. We here display the documentation strings, which are the only information available to the LLM agent:

Analysis: code-based plotting

```
def plot_from_code(code: str):
    Execute python code that produces a plot from one or more previously saved arrays.
    The following variables are available during evaluation:
        all previously saved fields with their previously stated result_key as variable names
        (as global variables).
    You may use the following libraries:
        matplotlib: for plotting.
        matplotlib.pyplot as plt: for plotting.
        jax: for numerical operations.
        jax.numpy: for numerical operations.
        numpy: for numerical operations.
    Args:
        code: python code that produces a plot (without a plt.show() call!).
    Returns:
        Image
```

Analysis: code based analysis

```
def execute_code(code: str):
    Evaluate python code.
    This code can be e.g. be used to transform the previously saved fields or to
    calculate or save new fields.
    You can not see any plots created with this tool.
    The following variables are available during evaluation:
        all previously saved fields with their previously stated result_key as variable
        names (as global variables).
        jax: jax for numerical operations.
        jnp: jax.numpy for numerical operations.
        np: numpy for numerical operations.
        scipy: scipy for numerical operations including optimization and solving
        differential equations.
        sklearn: scikit-learn.
    IMPORTANT: Your code must set the variable 'result' to a dictionary in the end which
    should contain the newly generated data, for example: result={'<result_key>': <data>,
    ...}.
    Args:
        code: python code that sets the result variable to a dictionary containing some newly
        generated data.
    Returns:
        The result dictionary.
```

S 2: Details on physical systems including experiment tools

Here, we state the governing equations of each physical system, the solutions discovered by the agent, and the experimental tools available to the agent when exploring the system.

A. Mechanical systems

The agent is asked to produce a Python function that reproduces the `observe_experiment` tool. Depending on the system type, we provide different descriptions of the system, experimental tools, and tools to save the predicted differential equation, as listed in detail below.

In short, for the generic dynamical systems, the agent is told their dimensionality, but coordinates are called 'generalized coordinates' and do not have descriptive names. For the systems with particles moving in 2d, the agent is told the number of particles and that they move in 2d. For the systems with hidden degrees of freedom, the agent is told that it can observe a particle moving in 1d/2d interacting with an unknown number of unobserved particles. For each system, we provide a tool where the agent can pass a list of up to 5 initial conditions to observe multiple trajectories with one tool call. The exact equations of motion of the systems are listed in Table S1 and the solutions discovered by the agent in Table S2.

The task given to the agent is the same across all mechanics experiments:

Task supplied to agent for mechanics tasks

Can you find a model that reproduces the `observe_experiment` function? After your exploration, save it using the `save_result` function.

For tasks involving noisy measurements, the agent is supplied with the description:

Additional description in case of measurement noise

The observations you are getting may be affected by measurement noise.

1. Generic dynamical systems

This section contains the description of the system and tools provided to the agent for the generic dynamical systems (damped Duffing oscillator, damped pendulum, damped asymmetric double well, non-linear damping, arbitrary 1d potential, damped driven oscillator, damped parametric oscillator, damped double pendulum, two damped coupled oscillators, three damped coupled oscillators).

Description supplied to agent for generic dynamical systems

You are investigating a dynamical physical system.

Below we show the `observe_experiment` and `save_result` tools for the case of two generalized coordinates. In other cases, the docstrings are slightly adapted.

Experiment: observe evolution given initial conditions for generic dynamical systems

```
def observe_experiment(q_inits:str, q_dot_inits:str, t0:float=0.0, T:float=20.0,
    nt:int=20001):
    Observe multiple trajectories of the experiment with given initial conditions.
    The maximum number of trajectories is 5.
    The system has 2 generalized coordinates.
    Args:
        q_inits: string in the form of a list of initial generalized coordinates at time t0,
        in the form [(q00, q10,), (q01, q11,), ..., (q04, q14,)],
        where qij is the i-th generalized coordinate for the j-th initial condition.
        q_dot_inits: string in the form of a list of initial generalized velocities at
        time t0, in the same form as q_inits.
        t0: float, initial time of the experiment (default 0.0)
        T: float, end time of the experiment (maximum 100.0, default 20.0)
        nt: int, number of time steps where to evaluate the experiment
        (maximum 20001, default 20001).
    Returns a dictionary with:
        'ts': np.ndarray of shape [nt] with the time steps [t0, t0+dt, ..., T]
        (equal for all trajectories).
```

```
'arrays':np.ndarray of shape [n_trajectories , nt, 4] with the solution.
Here, the first half of the entries along the last axis correspond to the generalized
coordinates in the same order as the input, and the second half of the entries
correspond to the generalized velocities in the same order.
```

The agent is asked to save its final prediction using the following tool:

Save result for generic dynamical systems

```
def save_result(code:str):
    Saves your final result.
    You can only call this once. Do not call it when you could potentially further improve
    your result!
    You should provide code that defines a predict_experiment function which should reproduce
    trajectories of the experiment.
    The code has access to numpy (as np) and scipy (as scipy).
    Args:
        code: String defining a function with the following signature:
            def predict_experiment(q_init: np.ndarray, q_dot_init: np.ndarray, t0: float,
                T: float, nt: int) -> Tuple[np.ndarray, np.ndarray]:
                Args (of predict_experiment):
                    q_init: np.ndarray of shape [2] with the initial generalized coordinates
                        at time t0, in the form np.ndarray([q0, q1]).
                        Here, qi is the initial condition for the i-th generalized coordinate.
                    q_dot_init: np.ndarray of shape [2] with the initial generalized
                        velocities at time t0 in the same order.
                    t0: float, initial time of the experiment.
                    T: float, end time of the experiment.
                    nt: int, number of time steps where to evaluate the experiment.
                Returns (of predict_experiment):
                    Xs: np.ndarray of shape [nt, 4] with the solution.
                        Here, the first half of the entries along the last axis correspond to
                        the generalized coordinates in the same order as the input,
                        and the second half of the entries correspond to the generalized
                        velocities in the same order.
                    ts: np.ndarray of length nt with the time steps [t0, t0+dt, ... , T].
```

2. Particles in 2d

This section contains the description of the system and tools provided to the agent for the 2d systems with particles (damped Mexican hat potential, particle in off-center gravity, arbitrary 2d potential, two particles with gravity, three particles with gravity, ten particles with exponential potential).

Description supplied to agent for particles in 2d

You are investigating a physical system consisting of particles moving in two dimensions.

We use slightly adapted `observe_experiment` and `save_result` tools. Below are examples for one particle.

Experiment: observe evolution given initial conditions for particles in 2d

```
def observe_experiment(pos_inits:str, vel_inits:str, t0:float=0.0, T:float=20.0,
    nt:int=20001):
    Observe multiple trajectories of the experiment with given initial conditions.
    The maximum number of trajectories is 5.
    The system contains 1 particle in 2D.
    Args:
        pos_inits: string in the form of a list of initial positions at time t0,
            in the form [(x0_0, y0_0), (x0_1, y0_1), ..., (x0_4, y0_4)], where xi_j and yi_j
            are the initial x and y positions of the i-th particle in the j-th trajectory.
```



```

    vel_inits: string in the form of a list of initial velocities at time t0,
    in the same form as pos_inits.
    t0: float, initial time of the experiment (default 0.0)
    T: float, end time of the experiment (maximum 100.0, default 20.0)
    nt: int, number of time steps where to evaluate the experiment
    (maximum 20001, default 20001).
Returns a dictionary with:
    'ts': np.ndarray of shape [nt] with the time steps [t0, t0+dt, ..., T]
    (equal for all trajectories).
    'arrays': np.ndarray of shape [n_trajectories, nt, 4] with the solution.
Here, the first half of the entries along the last axis correspond to the x and y
positions in the same order as the input,
and the second half of the entries correspond to the x and y velocities in the same
order.

```

Save result for particles in 2d

```

def save_result(code:str):
    Saves your final result.
    You can only call this once. Do not call it when you could potentially further improve
    your result!
    You should provide code that defines a predict_experiment function which should reproduce
    trajectories of the experiment.
    The code has access to numpy (as np) and scipy (as scipy).
    Args:
        code: String defining a function with the following signature:
        def predict_experiment(pos_init: np.ndarray, vel_init: np.ndarray, t0: float,
                               T: float, nt: int) -> Tuple[np.ndarray, np.ndarray]:
            Args (of predict_experiment):
                pos_init: np.ndarray of shape [2] with the initial positions at time t0,
                in the form np.ndarray([x0, y0]). Here, xi and yi are the initial x and y
                positions of the i-th particle.
                vel_init: np.ndarray of shape [2] with the initial velocities at time t0
                in the same order.
                t0: float, initial time of the experiment.
                T: float, end time of the experiment.
                nt: int, number of time steps where to evaluate the experiment.
            Returns (of predict_experiment):
                Xs: np.ndarray of shape [nt, 4] with the solution.
                Here, the first half of the entries along the last axis correspond to
                the x and y positions in the same order as the input,
                and the second half of the entries correspond to the x and y
                velocities in the same order.
                ts: np.ndarray of length nt with the time steps [t0, t0+dt, ..., T].

```

3. Systems with hidden degrees of freedom

In case of hidden coordinates we add to the initial description

Additional description supplied to agent for hidden generic dynamical systems

You can only observe the first generalized coordinate of this system. However, there might be additional hidden generalized coordinates influencing the dynamics.

or

Additional description supplied to agent for hidden particles in 2d

You can only observe the coordinates of one of the particles. However, there might be additional hidden particles influencing the dynamics.

Since starting the experiment at a later time is not possible (as the hidden particles' positions are only fixed at $t = 0$), we set the start- and end-time and time-resolution of the experiment. Everything else is kept as in the descriptions and docstrings above.

Damped Duffing oscillator		
$\ddot{x} = -ax^3 - bx - \gamma\dot{x}$		$a = 4.528, b = 1.625, \gamma = 0.043$
Damped pendulum		
$\ddot{x} = -\alpha \sin(x) - \gamma\dot{x}$		$\alpha = 1.712, \gamma = 0.043$
Damped asymmetric double well		
$\ddot{x} = -ax^3 + bx + c - \gamma\dot{x}$		$a = 4.528, b = 1.625, c = 0.1, \gamma = 0.043$
Velocity position coupling		
$\ddot{x} = -ax^2\dot{x} - kx$		$a = 1.7, k = 0.4,$
Arbitrary 1d potential		
$\ddot{x} = -x - a \cos(kx)$		$a = 4.8, k = 6$
Damped driven oscillator		
$\ddot{x} = -kx - \gamma\dot{x} + A \cos(\omega t)$		$k = 2.319, \gamma = 0.6, A = 1.712, \omega = 1.551$
Damped parametric oscillator		
$\ddot{x} = -(k + A \cos(\omega t))x - \gamma\dot{x}$		$k = 2.319, A = 1.712, \omega = 1.551, \gamma = 0.3$
Damped double pendulum		
$\Delta = \theta_1 - \theta_2$		$m_1 = m_2 = 1$
$\ddot{\theta}_1 = -\frac{(2m_1+m_2)\sin(\theta_1)-m_2\sin(\theta_1-2\theta_2)-2\sin(\Delta)m_2(\omega_2^2 l_2+\omega_1^2 l_1 \cos(\Delta))}{l_1(2m_1+m_2-m_2 \cos(2\Delta))} - \gamma\dot{\theta}_1$		$l_1 = 1.712, l_2 = 0.851$
$\ddot{\theta}_2 = \frac{2\sin(\Delta)(\omega_1^2 l_1(m_1+m_2)+(m_1+m_2)\cos(\theta_1)+\omega_2^2 l_2 m_2 \cos(\Delta))}{l_2(2m_1+m_2-m_2 \cos(2\Delta))} - \gamma\dot{\theta}_2$		$\gamma = 0.143$
Two damped coupled oscillators		
$\ddot{x}_1 = -k_1 x_1 + k_{12}(x_2 - x_1) - \gamma\dot{x}_1$ $\ddot{x}_2 = -k_2 x_2 - k_{12}(x_2 - x_1) - \gamma\dot{x}_2$		$k_1 = 1.712, k_{12} = 0.15$ $\gamma = 0.043$
Three damped coupled oscillators		
Analogous to two coupled oscillators, $k_1 = 1, k_2 = 1.5, k_3 = 0.5, k_{1,2} = 0.2, k_{13} = 0.3, k_{23} = 0.4$		
Damped Mexican hat potential		
$\ddot{x} = (a - b(x^2 + y^2))x - \gamma\dot{x}$ $\ddot{y} = (a - b(x^2 + y^2))y - \gamma\dot{y}$		$a = 4, b = 2.8$ $\gamma = 0.2$
Particle in off-center gravity		
$r = \sqrt{(x - c_x)^2 + (y - c_y)^2 + \epsilon^2}$ $\ddot{x} = -\frac{G(x - c_x)}{r^3}, \ddot{y} = -\frac{G(y - c_y)}{r^3}$		$c_x = -0.7, c_y = 0.2$ $G = 2.3, \epsilon = 0.01$
Arbitrary 2d potential		
$r = \sqrt{(x^2 + y^2)}$ $\ddot{x} = -kx - \frac{ax \cos(\omega r)}{r}, \ddot{y} = -ky - \frac{ay \cos(\omega r)}{r}$		$k = 0.6, a = 0.8$ $\omega = 6$
Two particles with gravity		
$\Delta_x = x_1 - x_2, \Delta_y = y_1 - y_2, r = \sqrt{\Delta_x^2 + \Delta_y^2 + \epsilon^2}$ $\ddot{x}_1 = -\frac{m_2 \Delta_x}{r^3}, \ddot{y}_1 = -\frac{m_2 \Delta_y}{r^3}, \ddot{x}_2 = \frac{m_1 \Delta_x}{r^3}, \ddot{y}_2 = \frac{m_1 \Delta_y}{r^3}$		$m_1 = 8.123, m_2 = 0.781$ $\epsilon = 0.01$
Three particles with gravity		
Analogous to two particles in gravity, $m_1 = 1.3, m_2 = 9.0, m_3 = 0.2, \epsilon = 0.01$		
Ten particles with exponential potential		
$r_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$ $\ddot{x}_i = \sum_j -ab \frac{x_i - x_j}{r_{ij}} \exp(-br_{ij}), \ddot{y}_i = \sum_j -ab \frac{y_i - y_j}{r_{ij}} \exp(-br_{ij})$		$a = 0.8$ $b = 1.3$
Two partially observed oscillators		
Analogous to two coupled oscillators, $k_1 = 1.1, k_2 = 1.54, k_{12} = 1.2, \gamma = 0$ Hidden initial conditions, $x_2 = 0.4, \dot{x}_2 = -0.2$		
Three partially observed oscillators		
Analogous to three coupled oscillators, $k_1 = 0.611, k_2 = 1.907, k_3 = 1.473, k_{12} = 1.317, k_{13} = 0.537, k_{23} = 1.811, \gamma = 0$ Hidden initial conditions, $x_2 = -0.123, \dot{x}_2 = 0.895, x_3 = 0.068, \dot{x}_3 = 0.957$		
Two partially observed particles with gravity		
Analogous to two particles in gravity, $m_1 = 0.7, m_2 = 1.8$ Hidden initial conditions, $x_2 = 0.5, y_2 = 0.5, \dot{x}_2 = -0.5, \dot{y}_2 = 0$		

TABLE S1. Equations of motion for the considered mechanical systems.

Damped Duffing oscillator [Tiny extra non-linear damping term.]		
$\ddot{x} = -\gamma\dot{x} - \gamma_2 x^2 \dot{x} - kx - ax^3$		$\gamma = 0.046, \gamma_2 = 0.007, k = 1.661, a = 4.512$
Damped pendulum		
$\ddot{x} = -\alpha \sin(x) - \gamma\dot{x}$		$\alpha = 1.719, \gamma = 0.042$
Damped asymmetric double well [Tiny extra non-linear damping term.]		
$\ddot{x} = -\delta\dot{x} + \alpha x - \beta x^3 + \kappa x^5 + \gamma x^2 \dot{x} + F_0$		$\delta = 0.040, \alpha = 1.606, \beta = 4.510,$ $\kappa = 0.002, \gamma = -0.003, F_0 = 0.104$
Velocity position coupling		
$\ddot{x} = -ax^2\dot{x} - kx$		$a = 1.702, k = 0.400,$
Arbitrary 1d potential		
$\ddot{x} = -kx + B \cos(\omega x) + c$		$k = 1, B = -4.819, c = 0.001, \omega = 6$
Damped driven oscillator		
$\ddot{x} = -\gamma\dot{x} - kx + F_c \cos(\omega t)$		$k = 2.320, \gamma = 0.601, F_c = 1.713, \omega = 1.551$
Damped parametric oscillator [Missed external driving.]		
$\ddot{x} = -c\dot{x} - kx + bx^3$		$k = 2.242, b = 0.004, c = 0.154$
Damped double pendulum		
$\Delta = \theta_1 - \theta_2$		$m_1 = m_2 = 1$
$\ddot{\theta}_1 = \frac{-g(2m_1+m_2)\sin(\theta_1) - m_2 g \sin(\theta_1 - 2\theta_2) - 2m_2 \sin(\Delta)(\dot{\theta}_2^2 l_2 + \dot{\theta}_1^2 l_1 \cos(\Delta))}{l_1(2m_1+m_2-m_2 \cos(2\Delta))} - d_1 \dot{\theta}_1$		$l_1 = 3.233, l_2 = 1.597$
$\ddot{\theta}_2 = \frac{2 \sin(\Delta)(\dot{\theta}_1^2 l_1(m_1+m_2) + g(m_1+m_2) \cos(\theta_1) + \dot{\theta}_2^2 l_2 m_2 \cos(\Delta))}{l_2(2m_1+m_2-m_2 \cos(2\Delta))} - d_2 \dot{\theta}_2$		$g = 1.911, d_1 = 0.145, d_2 = 0.134$
Two damped coupled oscillators		
$\vec{x} = A\vec{x} + B\vec{x}$		$A = \begin{bmatrix} -1.863 & 0.150 \\ 0.150 & -1.001 \end{bmatrix}, B = \begin{bmatrix} -0.043 & 0 \\ 0 & -0.043 \end{bmatrix}$
Three damped coupled oscillators		
Analogous to above with :		$A = \begin{bmatrix} -1.500 & 0.200 & 0.300 \\ 0.200 & -2.101 & 0.400 \\ 0.300 & 0.400 & -1.200 \end{bmatrix}, B = \begin{bmatrix} -0.162 & 0.000 & 0.000 \\ 0.000 & -0.162 & 0.000 \\ 0.000 & 0.000 & -0.162 \end{bmatrix}$
Damped Mexican hat potential		
$\ddot{x} = \alpha x + \beta(x^2 + y^2)x + \gamma\dot{x}$ $\ddot{y} = \alpha y + \beta(x^2 + y^2)y + \gamma\dot{y}$		$\alpha = 2, \beta = -0.7$ $\gamma = -0.2$
Particle in off-center gravity		
$r = \sqrt{(x - c_x)^2 + (y - c_y)^2 + \epsilon^2}$ $\ddot{x} = -\frac{G(x - c_x)}{r^3}, \ddot{y} = -\frac{G(y - c_y)}{r^3}$		$c_x = -0.700, c_y = 0.200$ $G = 2.297, \epsilon = 0.000$
Arbitrary 2d potential [Missed analytic description of radial force. However, the numerical fit closely follows the true law.]		
$r = \sqrt{(x^2 + y^2)}$ $\ddot{x} = -\frac{g(r)}{r}x, \ddot{y} = -\frac{g(r)}{r}y$		$g(r)$: radial acceleration from lookup table
Two particles with gravity		
$\Delta_x = x_1 - x_2, \Delta_y = y_1 - y_2, r = \sqrt{\Delta_x^2 + \Delta_y^2 + \epsilon^2}$ $\ddot{x}_1 = -\frac{m_2 \Delta_x}{r^3}, \ddot{y}_1 = -\frac{m_2 \Delta_y}{r^3}, \ddot{x}_2 = \frac{m_1 \Delta_x}{r^3}, \ddot{y}_2 = \frac{m_1 \Delta_y}{r^3}$		$m_1 = 8.338, m_2 = 0.794$ $\epsilon = 0.031$
Three particles with gravity		
Analogous to two particles in gravity,		$m_1 = 1.298, m_2 = 9.003, m_3 = 0.200, \epsilon = 0.010$
Ten particles with exponential potential [Understood that it is a pairwise force but missed the exact form.]		
$r_{ij}^2 = (x_j - x_i)^2 + (y_j - y_i)^2$ $\ddot{x}_i = G \sum_{j \neq i} \frac{x_j - x_i}{(r_{ij}^2 + s^2)^{(p+1)/2}}, \ddot{y}_i = G \sum_{j \neq i} \frac{y_j - y_i}{(r_{ij}^2 + s^2)^{(p+1)/2}}$		$G = 0.247$ $s = 0.290, p = 2.160$
Two partially observed oscillators		
Analogous to two coupled oscillators,		$k_1 = 1.1, k_2 = 1.54, k_{12} = 1.2, \gamma = 0$

Hidden initial conditions, $x_2 = 0.4, \dot{x}_2 = -0.2$

Three partially observed oscillators

[Exact analytical solution with coefficients fitted from data instead of simulating hidden particle.]

$$x(t) = \sum_{k=0}^2 \left[\left(B_k^{(c)} + A_k^{(c)} x_0 + V_k^{(c)} \dot{q}_0 \right) \cos(\omega_k t) + \left(B_k^{(s)} + A_k^{(s)} x_0 + V_k^{(s)} \dot{q}_0 \right) \sin(\omega_k t) \right] + C_0$$

Two partially observed particles with gravity

[Completely wrong model. Not an ode.]

Specular reflection model on five infinite straight lines.

Dynamics: free motion (constant velocity) except when crossing a line $y = m x + b$, where velocity reflects across the line's normal.

TABLE S2: Best fitting equations of motion for the mechanical systems extracted from the `predict_experiment` function saved by the agent. We include comments on the system for which the agent failed to recover the true model in red.

B. Field systems

The agent is tasked to reproduce the time evolution of a complex-valued field on a 1d lattice. Additionally, it is given the number of lattice points, the maximum experiment time, and the space resolution of the system. We show the dynamics of the considered fields in Figure S1, the exact equations of motion in Table S3, and the solutions discovered by the agent in Table S4.

The task of the agent is exactly the same as for the mechanical systems above:

Task supplied to agent for field tasks

Can you find a model that reproduces the `observe_experiment` function? After your exploration, save it using the `save_result` function.

The description of the system is focused purely on the topology of the system:

Default description supplied to agent in field tasks

This physical system consists of a complex-valued field evolving on 100 discrete coupled nodes. The nodes are arranged in 1D with periodic boundary conditions, i.e. the first and last node are neighbors.

Additional description in case of measurement noise

The observations you are getting may be affected by measurement noise.

The agent can run an experiment via the following tool:

Experiment: observe the evolution of a field with given initial conditions

```
def observe_experiment(initial_condition_code:str, t0:float=0., T:float=100., nt:float=1001):
    Run one experiment of the evolution of the system, where you can choose the initial
    condition.
    Args:
        initial_condition_code: python code with jax.numpy syntax that sets the variable
        phi0, which represents the complex field at time t0. Use np.sin(...) etc.
        phi0 needs to be of shape (n_nodes,) and can be complex-valued.
        t0: float, initial time of the experiment (default 0.0)
        T: float, end time of the experiment (default 20.0, maximum 100.0)
        nt: int, number of time steps where to evaluate the experiment (default 2001,
        maximum 10001)
    Returns a dict of the form:
        ts: jax.Array of shape (nt,), the time points
       phis: jax.Array of shape (nt, n_nodes), the complex field solution
```

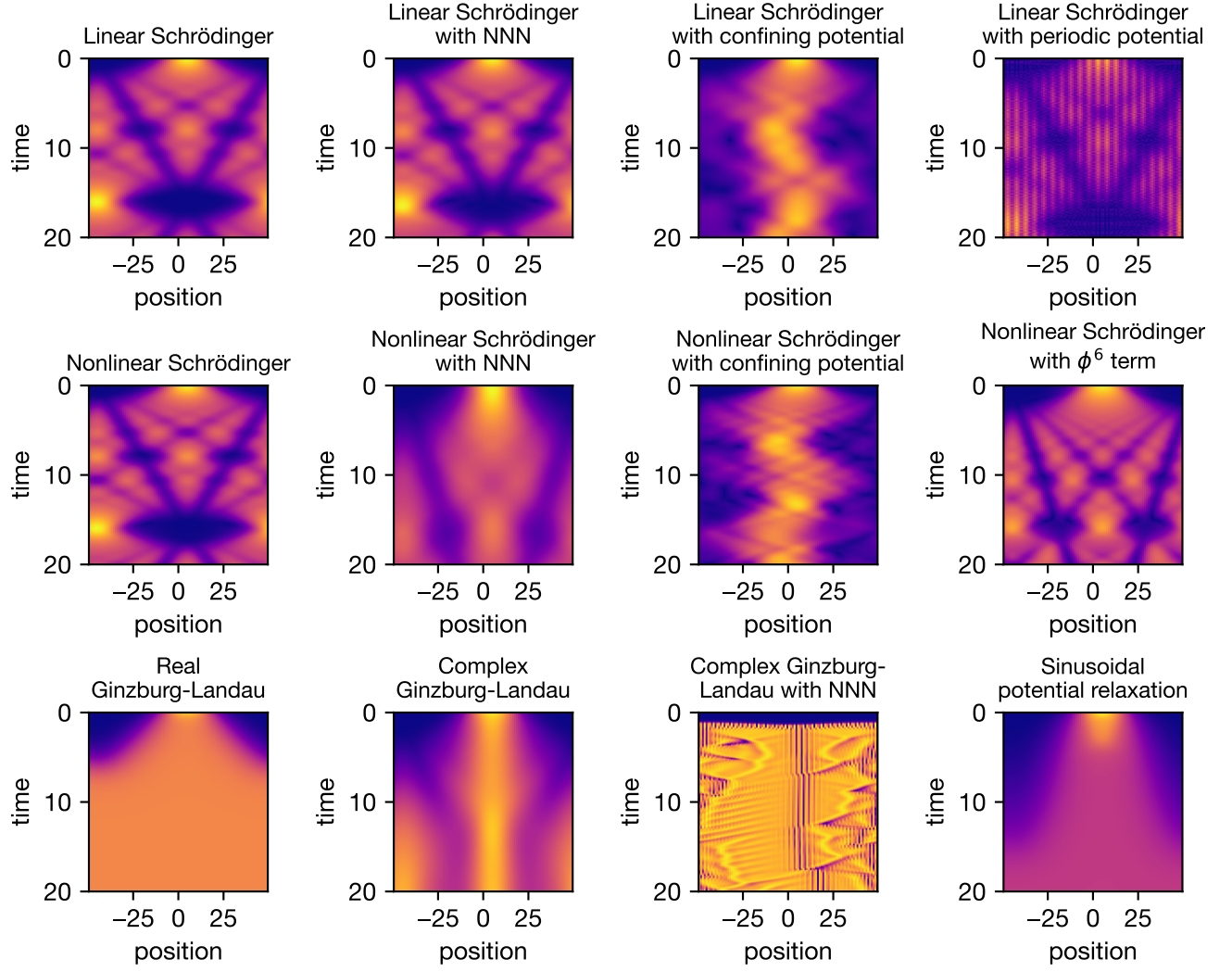


FIG. S1. **Field dynamics.** Dynamics of the different field systems with a Gaussian wave packet as the initial condition. The squared absolute value of the field $|\phi|^2$ is depicted with the color scale ranging from blue (zero) to yellow (high).

Linear Schrödinger $U_{\text{kin}}(\phi_k, k, t, \Delta t) = \exp(-i[1 - \cos(k)]\Delta t) \phi_k$ $U_{\text{pot}}(\phi, x, t, \Delta t) = \phi$	$i \frac{d\phi_n}{dt} = -\frac{1}{2}(\Delta_d \phi)_n$
Linear Schrödinger with NNN $U_{\text{kin}}(\phi_k, k, t, \Delta t) = \exp(-i[A - \cos(k) + B \cos(2k)]\Delta t) \phi_k$ $U_{\text{pot}}(\phi, x, t, \Delta t) = \phi$	$i \frac{d\phi_n}{dt} = (A - 1 + B)\phi_n - \frac{1}{2}(\Delta_d \phi)_n + \frac{B}{2}(\Delta_{d,2} \phi)_n$ $A = 0.5, B = 0.5$
Linear Schrödinger with confining potential $U_{\text{kin}}(\phi_k, k, t, \Delta t) = \exp(-i[1 - \cos(k)]\Delta t) \phi_k$ $U_{\text{pot}}(\phi, x, t, \Delta t) = \exp(-iB \cos(\pi x/x_{\text{max}})\Delta t) \phi$	$i \frac{d\phi_n}{dt} = -\frac{1}{2}(\Delta_d \phi)_n + B \cos\left(\frac{\pi(n-x_{\text{max}})}{x_{\text{max}}}\right) \phi_n$ $B = -0.5, x_{\text{max}} = 49.5$
Linear Schrödinger with periodic potential $U_{\text{kin}}(\phi_k, k, t, \Delta t) = \exp(-i[1 - \cos(k)]\Delta t) \phi_k$ $U_{\text{pot}}(\phi, x, t, \Delta t) = \exp(-iB \cos(N 2\pi x/x_{\text{max}})\Delta t) \phi$	$i \frac{d\phi_n}{dt} = -\frac{1}{2}(\Delta_d \phi)_n + B \cos\left(\frac{2\pi N(n-x_{\text{max}})}{x_{\text{max}}}\right) \phi_n$ $B = 0.2, N = 10, x_{\text{max}} = 49.5$
Nonlinear Schrödinger $U_{\text{kin}}(\phi_k, k, t, \Delta t) = \exp(-iA[1 - \cos(k)]\Delta t) \phi_k$ $U_{\text{pot}}(\phi, x, t, \Delta t) = \exp(-iB \phi ^2 \Delta t) \phi$	$i \frac{d\phi_n}{dt} = -\frac{A}{2}(\Delta_d \phi)_n + B \phi_n ^2 \phi_n$ $A = 0.5$ $B = 0.25$
Nonlinear Schrödinger with NNN $U_{\text{kin}}(\phi_k, k, t, \Delta t) = \exp(-i[A(B - \cos(k) + C \cos(2k))]\Delta t) \phi_k$ $U_{\text{pot}}(\phi, x, t, \Delta t) = \exp(-iD \phi ^2 \Delta t) \phi$	$i \frac{d\phi_n}{dt} = (AB - 1 + AC)\phi_n - \frac{A}{2}(\Delta_d \phi)_n$ $+ \frac{AC}{2}(\Delta_{d,2} \phi)_n + D \phi_n ^2 \phi_n$ $A = 0.5, B = 1.5, C = 0.5$ $D = 0.25$
Nonlinear Schrödinger with confining potential $U_{\text{kin}}(\phi_k, k, t, \Delta t) = \exp(-i[1 - \cos(k)]\Delta t) \phi_k$ $U_{\text{pot}}(\phi, x, t, \Delta t) = \exp(-i[-\cos(\pi x/x_{\text{max}}) + \phi ^2]\Delta t) \phi$	$i \frac{d\phi_n}{dt} = -\frac{1}{2}(\Delta_d \phi)_n + \left(-\cos\left(\frac{\pi(n-x_{\text{max}})}{x_{\text{max}}}\right) + \phi_n ^2\right) \phi_n$ $N = 10, x_{\text{max}} = 49.5$
Nonlinear Schrödinger with ϕ^6 term $U_{\text{kin}}(\phi_k, k, t, \Delta t) = \exp(-iA[1 - \cos(k)]\Delta t) \phi_k$ $U_{\text{pot}}(\phi, x, t, \Delta t) = \exp(-iB(\phi ^2 + 2 \phi ^4)\Delta t) \phi$	$i \frac{d\phi_n}{dt} = -\frac{A}{2}(\Delta_d \phi)_n + B(\phi_n ^2 + 2 \phi_n ^4) \phi_n$ $A = 0.5$ $B = 0.25$
Real Ginzburg–Landau $U_{\text{kin}}(\phi_k, k, t, \Delta t) = \exp(-A[1 - \cos(k)]\Delta t) \phi_k$ $U_{\text{pot}}(\phi, x, t, \Delta t) = \exp(-B(2 \phi ^2 - 1)\Delta t) \phi$	$\frac{d\phi_n}{dt} = -A(\Delta_d \phi)_n - B(2 \phi_n ^2 - 1)\phi_n$ $A = 0.5$ $B = 0.5$
Complex Ginzburg–Landau $U_{\text{kin}}(\phi_k, k, t, \Delta t) = \exp(-A[1 - \cos(k)]\Delta t) \phi_k$ $U_{\text{pot}}(\phi, x, t, \Delta t) = \exp(-B(2C \phi ^2 - 1)\Delta t) \phi$	$\frac{d\phi_n}{dt} = -A(\Delta_d \phi)_n - B(2C \phi_n ^2 - 1)\phi_n$ $A = 0.2(0.5 + 2i)$ $B = 0.2, C = (1 - 1.5i)$
Complex Ginzburg–Landau with NNN $U_{\text{kin}}(\phi_k, k, t, \Delta t) = \exp(-A[B - \cos(k) + C \cos(2k)]\Delta t) \phi_k$ $U_{\text{pot}}(\phi, x, t, \Delta t) = \exp(-D(2E \phi ^2 - 1)\Delta t) \phi$	$\frac{d\phi_n}{dt} = -A\left[(B - 1 + C)\phi_n - \frac{1}{2}(\Delta_d \phi)_n + \frac{C}{2}(\Delta_{d,2} \phi)_n\right]$ $- D(2E \phi_n ^2 - 1)\phi_n$ $A = 0.8(0.5 + 0.5i), B = 0.65, C = 0.8$ $D = 0.2, E = (1 - 1.5i)$
Sinusoidal potential relaxation $U_{\text{kin}}(\phi_k, k, t, \Delta t) = \exp(-A[1 - \cos(k)]\Delta t) \phi_k$ $U_{\text{pot}}(\phi, x, t, \Delta t) = \exp(B \sin(C \phi ^2)\Delta t) \phi$	$\frac{d\phi_n}{dt} = -A(\Delta_d \phi)_n + B \sin(C \phi_n ^2) \phi_n$ $A = 0.5$ $B = 0.1, C = 15$

TABLE S3. Field equation and corresponding kinetic and potential propagators for field systems. Here $(\Delta_d \phi)_n = \phi_{n+1} - 2\phi_n + \phi_{n-1}$ and $(\Delta_{d,2} \phi)_n = \phi_{n+2} - 2\phi_n + \phi_{n-2}$.

Linear Schrödinger		
$i \frac{d\phi_n}{dt} = -\kappa (\Delta_d \phi)_n$		$\kappa = 50.0$
Linear Schrödinger with NNN		
$i \frac{d\phi_n}{dt} = \alpha (\Delta_d \phi)_n + \beta (\Delta_{d,2} \phi)_n$		$\alpha = 50.0$ $\beta = 25.0$
Linear Schrödinger with confining potential		
$i \frac{d\phi_n}{dt} = -\kappa (\Delta_d \phi)_n - V_n \phi_n, \quad V_n = V_0 \cos\left(\frac{2\pi n}{N} + \Phi\right)$		$\kappa = 50.0$ $V_0 = 0.5$ $N = \text{number of lattice sites}$ $\Phi = -\pi + \frac{\pi}{N}$
Linear Schrödinger with periodic potential		
[Included unnecessary couplings beyond nearest-neighbors.]		
Linear Schrödinger equation with up to fourth nearest-neighbor coupling fitted from data and onsite potential V_n		
$V_n = A + B \cos(2\pi \frac{n}{P}) + C \sin(2\pi \frac{n}{P})$		$A = 70.4,$ $B = -14.2,$ $C = -10.1,$ $P = 5$
Nonlinear Schrödinger		
$i \frac{d\phi_n}{dt} = -\kappa (\Delta_d \phi)_n + g \phi_n ^2 \phi_n$		$\kappa = 50.0$ $g = 0.25$
Nonlinear Schrödinger with NNN		
$i \frac{d\phi_n}{dt} = \omega_0 \phi_n + \alpha (\Delta_d \phi)_n + \beta (\Delta_{d,2} \phi)_n + g \phi_n ^2 \phi_n$		$\omega_0 = 50.0$ $\alpha = -25.0$ $\beta = 12.5$ $g = 0.25$
Nonlinear Schrödinger with confining potential		
$i \frac{d\phi_n}{dt} = -\kappa (\Delta_d \phi)_n - V_n \phi_n + \gamma \phi_n ^2 \phi_n,$ $V_n = a \cos\left(\frac{2\pi n}{N}\right) + b \sin\left(\frac{2\pi n}{N}\right)$		$\kappa = 50.01$ $\gamma = 1.00$ $a = -1.00$ $b = 0.03$ $N = \text{lattice size (ring)}$
Nonlinear Schrödinger with ϕ^6 term		
$i \frac{d\phi_n}{dt} = -\kappa (\Delta_d \phi)_n + g \phi_n ^2 \phi_n + h \phi_n ^4 \phi_n$		$\kappa = 25.0$ $g = 0.25$ $h = 0.5$
Real Ginzburg-Landau		
$\frac{d\phi_n}{dt} = \mu \phi_n + D (\Delta_d \phi)_n - g \phi_n ^2 \phi_n$		$\mu = 0.5$ $D = 25.0$ $g = 1.0$
Complex Ginzburg-Landau		
$\frac{d\phi_n}{dt} = a \phi_n + b (\Delta_d \phi)_n - c \phi_n ^2 \phi_n$		$a = 0.2$ $b = 5 + 20i$ $c = 0.4 - 0.6i$
Complex Ginzburg-Landau with NNN		
[Included unnecessary Laplacian-nonlinear term (prefactor E)]		
$\frac{d\phi_n}{dt} = A \phi_n + B (\Delta_d \phi)_n + C (\Delta_{d,2} \phi)_n + D \phi_n ^2 \phi_n + E \phi_n ^2 (\Delta_d \phi)_n$		$A = -15.22 - 17.39i$ $B = -42.85 - 40.58i$ $C = -15.89 - 14.63i$ $D = -0.49 + 0.66i$ $E = -0.037 + 0.012i$
Sinusoidal potential relaxation		
[Found completely different model.]		

$$\frac{d\phi_n}{dt} = a\phi_n + b(\Delta_d\phi)_n - c|\phi_n|^2\phi_n + \alpha|\phi_n|^2(\Delta_d\phi)_n + \beta(\Delta_d(|\phi|^2\phi))_n + \gamma(\Delta_{d,2}\phi)_n + \mu\langle|\phi|^2\rangle\phi_n$$

Here $\langle|\phi|^2\rangle$ is the mean value at a given time.

$a = 0.05$
 $b = 25.08$
 $c = 0.16$
 $\alpha = 2.88$
 $\beta = -3.00$
 $\gamma = -1.07$
 $\mu = 0.11$

TABLE S4: Best fitting equations of motion for the field systems extracted from the `predict_experiment` function saved by the agent. We include comments on the system for which the agent failed to recover the true model in red.

The agent saves its result via:

Save prediction for field systems

```
def save_result(code:str):
    Saves your final result.
    You can only call this once. Do not call it when you could potentially further improve
    your result!
    You should provide code that defines a predict_experiment function which should reproduce
    trajectories of the experiment.
    Your code does not have access to previously computed arrays, so make sure your model
    does not need access to a large set of numerical constants.
    The code has access to numpy (as np), jax (as jax), jax.numpy (as jnp), and scipy (as
    scipy).
    Args:
        code: String defining a function with the following signature:
        def predict_experiment(initial_cond:jax.Array, t0:float, T:float, nt:int)
        -> Tuple[jax.Array, jax.Array]:
        Args (of predict_experiment):
            initial_cond : complex jax.Array of shape (n_nodes,), the initial
            condition of the complex field at time t=t0
            t0 : float, initial time of the experiment
            T : float, end time of the experiment
            nt : int, number of time steps where to evaluate the experiment
        Returns a tuple of the form (ts, phis):
            ts : jax.Array of shape (nt,) with time points [t0, t0+dt, ..., T]
            phis : complex jax.Array of shape (nt, n_nodes), the complex field
            solution
```

C. Quantum systems

The agent is asked to discover the underlying Hamiltonian for an unknown quantum many-body system composed of spins (spin 1/2, i.e. qubits). In some cases, the Hamiltonian may contain one or several parameters (fields, couplings). In some cases, the agent can change the size N of the experimental system as well. The agent does not have any information except that it is dealing with a spin 1/2 system and that the number of spins is a certain value (or variable) and the connectivity of the system (1d vs 2d grid). If there is a parameter, the agent is just being told the name of that parameter (e.g. 'A') without reference to its meaning. The exact Hamiltonians are specified in Table S5 and the Hamiltonians predicted by the agent in Table S6.

The system is described in the following way:

Default description supplied to agent in quantum tasks

You do have access to an experimental system of ... spins.

Additional description in case of 2d systems

The system is a 2D system of spins on a 3x3 grid, with spin index $j=j_x+j_y*M$, for $M=3$.

Additional description in case of variable system size

You can change the number of spins N in each experiment call. You should not choose values larger than about $N=12$.

Additional description for partially observed dynamics

You can only observe and initialize the spins at the indices ..., whose spin expectation values will be returned in this order.

Additional description in case of shot noise

Expectation values are estimated from ... measurement shots, leading to some statistical noise.

The task for the agent is:

Task supplied to agent for field tasks

Can you find the Hamiltonian of this system? After your exploration, save it using the `save_result` function.

To save its prediction, the agent uses:

Save result for quantum systems

```
def save_result(code:str)
    Saves your final result.
    You can only call this once. Do not call it when you could potentially further improve
    your result!
    You pass a python code that must produce an operator H, constructing it out of provided
    spin operators. Here Sx is a list of spin operator x-components, Sy likewise for the y-
    components, and Sz for the z-components. These are Pauli matrices. They can be accessed
    like Sx[2] etc. Remember to use the "@" matrix multiplication operator when taking the
    product of several spin operators. Otherwise you can use jax.numpy syntax in the form "
    jnp.sin(...)".
    You may also use the current variable N (number of spins in the experiment), but you
    cannot set a new value for it here.
    Args:
        code: the python code defining the Hamiltonian.
    Returns:
        Message that the Hamiltonian has been stored.
```

1. Ground state tasks

For ground state tasks, the agent has access to the following experiment tools:

Experiment: define operators to be observed in ground state of system

```
def set_operator(operator_label:str, operator_code: str):
    Define a Hermitian operator, whose ground state expectation value can later be evaluated.

    You pass a python code that must produce an operator H, constructing it out of provided
    spin operators. Here Sx is a list of spin operator x-components, Sy likewise for the y-
    components, and Sz for the z-components. These are Pauli matrices. They can be accessed
    like Sx[2] etc. Remember to use the "@" matrix multiplication operator when taking the
    product of several spin operators. Otherwise you can use jax.numpy syntax in the form "
    jnp.sin(...)".
    You may also use the current variable N (number of spins in the experiment), but you
    cannot set a new value for it here.
```

GS Transverse-field Ising chain		
$\hat{H} = J \sum_{j=0}^{N-2} \hat{\sigma}_j^z \hat{\sigma}_{j+1}^z - h \sum_{j=0}^{N-1} \hat{\sigma}_j^x$	$N = 10, J = 1.5, h = 0.6$	
GS Transverse-field Ising chain with tunable coupling		
$\hat{H} = A \sum_{j=0}^{N-2} \hat{\sigma}_j^z \hat{\sigma}_{j+1}^z - h \sum_{j=0}^{N-1} \hat{\sigma}_j^x$	$N = 10, h = 0.6, A \text{ tunable}$	
GS Transverse-field Ising chain with tunable coupling and number of spins		
$\hat{H} = A \sum_{j=0}^{N-2} \hat{\sigma}_j^z \hat{\sigma}_{j+1}^z - h \sum_{j=0}^{N-1} \hat{\sigma}_j^x$	$N \text{ tunable}, h = 0.6, A \text{ tunable},$	
GS 2d Heisenberg model		
$\hat{H} = J \sum_{\langle r, r' \rangle} (\hat{\sigma}_r^x \hat{\sigma}_{r'}^x + \hat{\sigma}_r^y \hat{\sigma}_{r'}^y + \hat{\sigma}_r^z \hat{\sigma}_{r'}^z) - h \sum_r \hat{\sigma}_r^x$	$N = 9, J = 1, h = 2$	
GS 2d Heisenberg model with tunable coupling		
$\hat{H} = 2A \sum_{\langle r, r' \rangle} (\hat{\sigma}_r^x \hat{\sigma}_{r'}^x + \hat{\sigma}_r^y \hat{\sigma}_{r'}^y + \hat{\sigma}_r^z \hat{\sigma}_{r'}^z) - h \sum_r \hat{\sigma}_r^x$	$N = 9, h = 2, A \text{ tunable}$	
GS 2d Heisenberg model with tunable coupling and field strength		
$\hat{H} = A \sum_{\langle r, r' \rangle} (\hat{\sigma}_r^x \hat{\sigma}_{r'}^x + \hat{\sigma}_r^y \hat{\sigma}_{r'}^y + \hat{\sigma}_r^z \hat{\sigma}_{r'}^z) - 2B \sum_r \hat{\sigma}_r^x$	$N = 9, A \text{ tunable}, B \text{ tunable}$	
GS Cluster Ising chain		
$\hat{H} = K \sum_{j=0}^{N-3} \hat{\sigma}_j^z \hat{\sigma}_{j+1}^x \hat{\sigma}_{j+2}^z - J \sum_{j=0}^{N-2} \hat{\sigma}_j^z \hat{\sigma}_{j+1}^z - h \sum_{j=0}^{N-1} \hat{\sigma}_j^x$	$N = 10, K = 0.5, J = 1, h = 0.3$	
GS Cluster Ising chain with tunable coupling		
$\hat{H} = A \sum_{j=0}^{N-3} \hat{\sigma}_j^z \hat{\sigma}_{j+1}^x \hat{\sigma}_{j+2}^z - J \sum_{j=0}^{N-2} \hat{\sigma}_j^z \hat{\sigma}_{j+1}^z - h \sum_{j=0}^{N-1} \hat{\sigma}_j^x$	$N = 10, A \text{ tunable}, J = 1, h = 1$	
GS Topological Ising chain with tunable coupling and number of spins		
$\hat{H} = A \sum_{j=0}^{N-3} \hat{\sigma}_j^z \hat{\sigma}_{j+1}^x \hat{\sigma}_{j+2}^z - J \sum_{j=0}^{N-2} \hat{\sigma}_j^z \hat{\sigma}_{j+1}^z - h \sum_{j=0}^{N-1} \hat{\sigma}_j^x$	$N \text{ tunable}, A \text{ tunable}, J = 1, h = 1$	
GS Arbitrary Hamiltonian		
$\hat{H} = J_{xz} \sum_{j=0}^{N-2} \hat{\sigma}_j^x \hat{\sigma}_{j+1}^z - J_{yx} \sum_{j=0}^{N-2} \hat{\sigma}_j^y \hat{\sigma}_{j+1}^x - h_x \sum_{j=0}^{N-1} \hat{\sigma}_j^x + h_y \sum_{j=0}^{N-1} \hat{\sigma}_j^y$	$N = 10, J_{xz} = 1.5, J_{yx} = 0.7, h_x = 0.6, h_y = 0.4$	
DYN Arbitrary Hamiltonian		
$\hat{H} = J_1 \hat{\sigma}_0^x \hat{\sigma}_1^z + J_2 \hat{\sigma}_0^y \hat{\sigma}_2^x - h_1 \hat{\sigma}_1^y + h_2 \hat{\sigma}_2^y - K \hat{\sigma}_1^x \hat{\sigma}_2^y$	$N = 3, J_1 = 1, J_2 = 0.5, h_1 = 0.7, h_2 = 0.3, K = 0.8$	
DYN Arbitrary Hamiltonian with access to two spins		
$\hat{H} = J_1 \hat{\sigma}_0^x \hat{\sigma}_1^z + J_2 \hat{\sigma}_0^y \hat{\sigma}_2^x - h_1 \hat{\sigma}_1^y + h_2 \hat{\sigma}_2^y - K \hat{\sigma}_1^x \hat{\sigma}_2^y$	$N = 3, J_1 = 1, J_2 = 0.5, h_1 = 0.7, h_2 = 0.3, K = 0.8$	
DYN Arbitrary Hamiltonian with access to two spins with tunable field		
$\hat{H} = J_1 \hat{\sigma}_0^x \hat{\sigma}_1^z + J_2 \hat{\sigma}_0^y \hat{\sigma}_2^x - A \hat{\sigma}_1^y + h_2 \hat{\sigma}_2^y - K \hat{\sigma}_1^x \hat{\sigma}_2^y$	$N = 3, J_1 = 1, J_2 = 0.5, h_2 = 0.3, K = 0.8, A \text{ tunable}$	
DYN Heisenberg chain		
$\hat{H} = J \sum_{j=0}^{N-2} (\hat{\sigma}_j^x \hat{\sigma}_{j+1}^x + \hat{\sigma}_j^y \hat{\sigma}_{j+1}^y + \hat{\sigma}_j^z \hat{\sigma}_{j+1}^z) - h \sum_{j=0}^{N-1} \hat{\sigma}_j^x$	$N = 10, J = 0.5, h = 1.5$	
DYN Heisenberg chain with tunable field		
$\hat{H} = J \sum_{j=0}^{N-2} (\hat{\sigma}_j^x \hat{\sigma}_{j+1}^x + \hat{\sigma}_j^y \hat{\sigma}_{j+1}^y + \hat{\sigma}_j^z \hat{\sigma}_{j+1}^z) - A \sum_{j=0}^{N-1} \hat{\sigma}_j^x$	$N = 10, J = 0.5, A \text{ tunable}$	
DYN Heisenberg chain with access to two spins with tunable field		
$\hat{H} = J \sum_{j=0}^{N-2} (\hat{\sigma}_j^x \hat{\sigma}_{j+1}^x + \hat{\sigma}_j^y \hat{\sigma}_{j+1}^y + \hat{\sigma}_j^z \hat{\sigma}_{j+1}^z) - A \sum_{j=0}^{N-1} \hat{\sigma}_j^x$	$N = 10, J = 0.5, A \text{ tunable}$	
DYN Transverse-field Ising chain with tunable coupling		
$\hat{H} = A \sum_{j=0}^{N-2} \hat{\sigma}_j^z \hat{\sigma}_{j+1}^z - h \sum_{j=0}^{N-1} \hat{\sigma}_j^x$	$N = 10, A \text{ tunable}, h = 1$	
DYN Transverse-field Ising chain with access to two spins with tunable coupling		
$\hat{H} = A \sum_{j=0}^{N-2} \hat{\sigma}_j^z \hat{\sigma}_{j+1}^z - h \sum_{j=0}^{N-1} \hat{\sigma}_j^x$	$N = 10, A \text{ tunable}, h = 1$	

TABLE S5. Hamiltonians for the considered quantum ground state (GS) and dynamics tasks (DYN).

Args:

operator_label: the label the operator will be stored under.
operator_code: the python code defining the operator.

Returns:

Message indicating whether the operator was set successfully.

In the following, we show the `observe_experiment` tool for the setting with tunable parameters and system size. In other settings the corresponding parts are removed.

Experiment: observe expectation values in ground state with variable physical parameter

```
def observe_experiment(operator_labels: str, set_params_code: str, N: int=10):
    Observe the ground state expectation values of specified operators.
    Args:
        operator_labels: a string with a comma-delimited list of operator labels
        (previously defined via set_operator)
        N: the desired number N of spins. Do not choose values larger than about N=12.
        set_params_code: python code that sets the numerical values of the system parameters
        (but not the system size).
    Returns a dict containing the expectation values for each of the operators (dict key
    named according to the operator label).
```

2. Dynamics tasks

For dynamics tasks, the agent has access to the following experiment tools (again, slightly adapted if the system has no tunable parameters):

Experiment: set initial Bloch vectors

```
def set_blochvectors(bloch_vector_label:str, bloch_vector_code:str):
    Define the initial Bloch vectors for the dynamics experiment.
    You pass a python code that must produce a variable 'bloch_vectors', which is a list or
    array of shape (N,3), where N is the number of spins you can observe in the experiment.
    Each row of bloch_vectors corresponds to one spin, and contains the x,y,z components of
    the Bloch vector (which must be normalized).
    You may use N (the number of spins you can observe in the experiment) in your code, but
    you cannot set a new value for it here.
    The code must include bloch_vectors = ... to define the variable.
    You may use jax and jnp in your code.
    Args:
        bloch_vector_label: the label the bloch vectors will be stored under.
        bloch_vector_code: the python code defining the bloch vectors.
    Returns:
        Message indicating whether the bloch vectors were set successfully.
```

Experiment: observe single spin expectation dynamics

```
def observe_experiment(bloch_vector_label: str, set_params_code: str, T: float=10.0, dt:
float=0.1):
    Observe the dynamics of the quantum spin system.
    Args:
        bloch_vector_label: the label of the initial bloch vectors
        (previously defined via set_blochvectors).
        set_params_code: python code that sets the numerical values of the system parameters.
        T: the total time to simulate.
        dt: the time step for the simulation. Do not choose values resulting in more than
        1001 time steps.
    Returns:
        A tuple (ts, Sx_t, Sy_t, Sz_t), where ts is the time array, and Sx_t, Sy_t, Sz_t are
        arrays of shape (N, nsteps) containing the expectation values of the spin components
        for each spin you observe over time.
```


GS Transverse-field Ising chain		
$\hat{H} = \sum_{j=0}^{N-2} \hat{\sigma}_j^z \hat{\sigma}_{j+1}^z - h \sum_{j=0}^{N-1} \hat{\sigma}_j^x$	$N = 10, h = 0.4$	
GS Transverse-field Ising chain with tunable coupling		
$\hat{H} = A \sum_{j=0}^{N-2} \hat{\sigma}_j^z \hat{\sigma}_{j+1}^z - 0.6 \sum_{j=0}^{N-1} \hat{\sigma}_j^x$	N tunable, A tunable, $h = 0.6$	
GS Transverse-field Ising chain with tunable coupling and number of spins		
$\hat{H} = A \sum_{j=0}^{N-2} \hat{\sigma}_j^z \hat{\sigma}_{j+1}^z - g \sum_{j=0}^{N-1} \hat{\sigma}_j^x$	$N = 10, g = 0.6, A$ tunable,	
GS 2d Heisenberg model		
$\hat{H} = J \sum_{\langle r, r' \rangle} (\hat{\sigma}_r^x \hat{\sigma}_{r'}^x + \hat{\sigma}_r^y \hat{\sigma}_{r'}^y + \hat{\sigma}_r^z \hat{\sigma}_{r'}^z) - \sum_r \hat{\sigma}_r^x$	$N = 9, J = 0.5$	
GS 2d Heisenberg model with tunable coupling		
[Missed factor of two for σ_x terms. Wrong boundary conditions.]		
$\hat{H} = A \sum_{\langle r, r' \rangle}^{\text{PBC}} (\hat{\sigma}_r^x \hat{\sigma}_{r'}^x + \hat{\sigma}_r^y \hat{\sigma}_{r'}^y + \hat{\sigma}_r^z \hat{\sigma}_{r'}^z) - \sum_r \hat{\sigma}_r^x$	$N = 9, A$ tunable	
GS 2d Heisenberg model with tunable coupling and field strength		
[Missed factor of two for σ_x terms. Wrong boundary conditions.]		
$\hat{H} = A \sum_{\langle r, r' \rangle}^{\text{PBC}} (\hat{\sigma}_r^x \hat{\sigma}_{r'}^x + \hat{\sigma}_r^y \hat{\sigma}_{r'}^y + \hat{\sigma}_r^z \hat{\sigma}_{r'}^z) - B \sum_r \hat{\sigma}_r^x$	$N = 9, A$ tunable, B tunable	
GS Cluster Ising chain		
$\hat{H} = K \sum_{j=0}^{N-3} \hat{\sigma}_j^z \hat{\sigma}_{j+1}^x \hat{\sigma}_{j+2}^z - J \sum_{j=0}^{N-2} \hat{\sigma}_j^z \hat{\sigma}_{j+1}^z - h \sum_{j=0}^{N-1} \hat{\sigma}_j^x$	$N = 10, K = 0.5, J = 1, h = 0.3$	
GS Cluster Ising chain with tunable coupling		
$\hat{H} = -\sum_{j=0}^{N-1} \hat{\sigma}_j^x - \sum_{j=0}^{N-2} \hat{\sigma}_j^z \hat{\sigma}_{j+1}^z + A \sum_{j=0}^{N-3} \hat{\sigma}_j^z \hat{\sigma}_{j+1}^x \hat{\sigma}_{j+2}^z$	$N = 10, A$ tunable,	
GS Topological Ising chain with tunable coupling and number of spins		
$\hat{H} = A \sum_{j=0}^{N-3} \hat{\sigma}_j^z \hat{\sigma}_{j+1}^x \hat{\sigma}_{j+2}^z - \sum_{j=0}^{N-2} \hat{\sigma}_j^z \hat{\sigma}_{j+1}^z - \sum_{j=0}^{N-1} \hat{\sigma}_j^x$	N tunable, A tunable,	
GS Arbitrary Hamiltonian		
$\hat{H} = \sum_{j=0}^{N-1} (h_x \hat{\sigma}_j^x + h_y \hat{\sigma}_j^y) + \sum_{j=0}^{N-2} (J_{yx} \hat{\sigma}_j^y \hat{\sigma}_{j+1}^x + J_{xz} \hat{\sigma}_j^x \hat{\sigma}_{j+1}^z)$	$N = 10, J_{yx} = -0.39, J_{xz} = 0.83,$ $h_x = -0.33, h_y = 0.22$	
DYN Arbitrary Hamiltonian		
$\hat{H} = J_1 \hat{\sigma}_0^x \hat{\sigma}_1^z + J_2 \hat{\sigma}_0^y \hat{\sigma}_2^x - h_1 \hat{\sigma}_1^y + h_2 \hat{\sigma}_2^y - K \hat{\sigma}_1^x \hat{\sigma}_2^y$	$N = 3, J_1 = 1.0, J_2 = 0.5,$ $h_1 = 0.7, h_2 = 0.3, K = 0.8$	
DYN Arbitrary Hamiltonian with access to two spins		
[Overly complicated model. Wrong constants in fit.]		
$\hat{H} = \sum_{a,b \in \{x,y,z\}} \left(J_{01}^{ab} \sigma_0^a \sigma_1^b + J_{12}^{ab} \sigma_1^a \sigma_2^b + J_{02}^{ab} \sigma_0^a \sigma_2^b \right) + \sum_{i=0}^2 (h_i^x \sigma_i^x + h_i^y \sigma_i^y + h_i^z \sigma_i^z)$	$N = 3, J_{01} \in \mathbb{R}^{3 \times 3}$ (fitted), $J_{12} \in \mathbb{R}^{3 \times 3}$ (fitted), $J_{02} \in \mathbb{R}^{3 \times 3}$ (fitted), $h \in \mathbb{R}^{3 \times 3}$ (fitted)	
DYN Arbitrary Hamiltonian with access to two spins with tunable field		
[Wrong model.]		
$\hat{H} = J_{xz}^{01} \hat{\sigma}_0^x \hat{\sigma}_1^z + J_{xy}^{12} (\hat{\sigma}_1^x \hat{\sigma}_2^x + \hat{\sigma}_1^y \hat{\sigma}_2^y) + B_{2y} \hat{\sigma}_2^y - A \hat{\sigma}_1^y$	$N = 3, J_{xz}^{01} = 0.85, J_{xy}^{12} = 0.38,$ $B_{2y} = 0.52, A$ tunable	
DYN Heisenberg chain		
$\hat{H} = J \sum_{j=0}^{N-2} (\hat{\sigma}_j^x \hat{\sigma}_{j+1}^x + \hat{\sigma}_j^y \hat{\sigma}_{j+1}^y + \hat{\sigma}_j^z \hat{\sigma}_{j+1}^z) + h_x \sum_{j=0}^{N-1} \hat{\sigma}_j^x$	$N = 10, J = 0.5, h_x = -1.5$	
DYN Heisenberg chain with tunable field		
$\hat{H} = J \sum_{j=0}^{N-2} (\hat{\sigma}_j^x \hat{\sigma}_{j+1}^x + \hat{\sigma}_j^y \hat{\sigma}_{j+1}^y + \hat{\sigma}_j^z \hat{\sigma}_{j+1}^z) - A \sum_{j=0}^{N-1} \hat{\sigma}_j^x$	$N = 10, J = 0.5, A$ tunable	
DYN Heisenberg chain with access to two spins with tunable field		
$\hat{H} = J \sum_{j=0}^{N-2} (\hat{\sigma}_j^x \hat{\sigma}_{j+1}^x + \hat{\sigma}_j^y \hat{\sigma}_{j+1}^y + \hat{\sigma}_j^z \hat{\sigma}_{j+1}^z) - A \sum_{j=0}^{N-1} \hat{\sigma}_j^x$	$N = 10, J = 0.5, A$ tunable	
DYN Transverse-field Ising chain with tunable coupling		
$\hat{H} = -\sum_{j=0}^{N-1} \hat{\sigma}_j^x + \alpha A \sum_{j=0}^{N-2} \hat{\sigma}_j^z \hat{\sigma}_{j+1}^z$	$N = 10, A$ tunable, $\alpha = 0.996$	
DYN Transverse-field Ising chain with access to two spins with tunable coupling		
$\hat{H} = A \sum_{j=0}^{N-2} \hat{\sigma}_j^z \hat{\sigma}_{j+1}^z - \sum_{j=0}^{N-1} \hat{\sigma}_j^x$	$N = 10, A$ tunable	

TABLE S6. Best fitting Hamiltonians predicted by the agent for the considered quantum ground state (GS) and dynamics tasks (DYN). We include comments on the system for which the agent failed to recover the true model in red.

S 3: Example conversations

To give an impression of typical strategies used by the agent, we ask GPT 5 to summarize example conversations. The LLM is asked to produce LaTeX code and to include the original visualizations that SciExplorer generated (in their original resolution). We show the discovery of the equations of motion of a particle in the arbitrary potential in Supplement S3 A, the discovery of the equations of motion of a particle in a Mexican hat potential with damping in Supplement S3 B, a failed exploration of the arbitrary 2d potential system in Supplement S3 C, the discovery of a complex Ginzburg-Landau equation in Supplement S3 D, the discovery of a partially observed Heisenberg model with tunable transverse field by observing dynamics in Supplement S3 E, and the discovery of a Cluster Ising model with tunable three-body term from ground state expectation values in Supplement S3 F.

A. Summary of exploration of the arbitrary potential

[Text below is LLM generated]

Goal.

The objective was to infer, validate, and save a predictive model that reproduces the hidden dynamical system behind the black-box function `observe_experiment`. The task required acting as a computational physicist: plan the investigation, generate hypotheses, run at least five experiments over diverse initial conditions, visualize and quantify the data, iteratively fit candidate models, and finally deliver a function `predict_experiment` that matches the observed trajectories.

Experimental design and reconnaissance. Five multi-trajectory experiments were run, each over a horizon of 20 s with a dense grid ($nt = 20001$), covering: (i) small to moderate amplitudes and velocities (Exp1), (ii) large positions at zero velocity (Exp2), (iii) zero position with high velocities, both signs (Exp3), (iv) mixed large positions and velocities (Exp4), and (v) near-equilibrium small motions (Exp5). For every experiment, I plotted $q(t)$, $v(t)$, and the phase portrait q versus v .

Key qualitative observations from the visualizations (Figs. S2–S6): - Oscillations persist without visible decay across 20 s, indicating negligible damping. - Phase portraits are closed loops, i.e., the system is conservative and autonomous. - For small motions, the loops are nearly elliptical but are centered at a slightly negative position (mean q near about -0.25). - At large amplitudes, the loops display pronounced, regularly spaced “wiggles,” suggesting a spatially periodic nonlinearity in the force.

Hypotheses and rationale. Guided by the observations above, I considered: - H1: simple harmonic oscillator (SHO), $q_{tt} + \omega^2 q = 0$; - H2: damped SHO; H3: Duffing; H4: pendulum-like $q_{tt} + \omega_0^2 \sin q = 0$; - nonlinear damping/friction terms; external time forcing; and - spatially periodic potentials superimposed on a linear spring.

Undamped closed orbits and time-reversal symmetry ruled out damping and time-forcing. The multi-lobed phase curves indicated a periodic-in-position force, not a purely polynomial stiffness.

Quantification and model discovery. Accelerations were estimated from the measured velocity via centered differences. I then examined the conditional mean $E[a \mid q]$ across all experiments, which revealed a strong periodic dependence in q with an apparent spatial frequency near 6 rad per unit q and a weak long-wavelength trend (Fig. S7).

I regressed a against libraries including $\{1, q, q^3, \sin q, \sin(2q), q_t, |q_t|q_t\}$. Velocity terms were consistently negligible, confirming the system is conservative with $a = a(q)$. Fits restricted to $\sin(q)$ or low harmonics produced low R^2 (approx. 0.07). Nonlinear least squares with

$$a(q) = A \sin(w(q - q_0)) + C$$

raised R^2 to ≈ 0.83 , but direct simulations of this model failed for extreme initial conditions (Fig. S8).

To capture the long-wavelength trend evident in Fig. S7, I next fitted

$$a(q) = -k(q - q_c) + \sum_{n=1}^3 A_n \sin(n w (q - q_0)),$$

which yielded $R^2 \approx 0.99962$ with parameters $k \approx 1.00$, $w \approx 6.00$, $q_0 \approx -0.262$, and only the first harmonic significant ($A_1 \approx -4.82$; higher harmonics ≈ 0). Using trigonometric identities this reduces to a very compact and equivalent form

$$a(q) = -k q + B \cos(6q) + c,$$

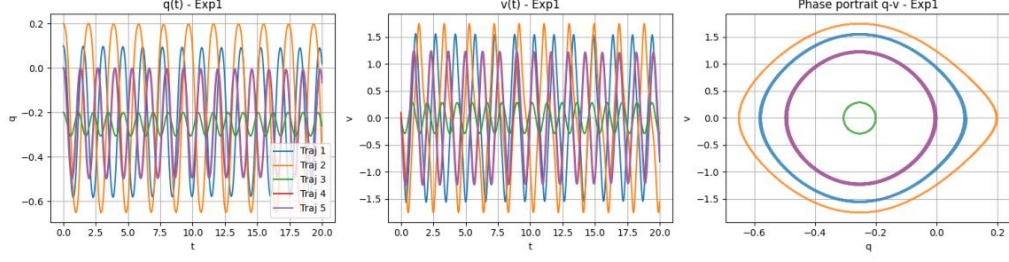


FIG. S2. Experiment 1: $q(t)$, $v(t)$, and phase portraits show undamped, closed orbits with a slight negative bias in q and weak amplitude dependence.

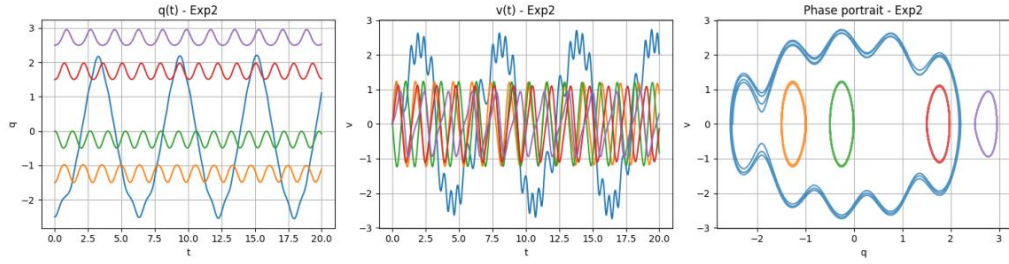


FIG. S3. Experiment 2: large-amplitude motions exhibit clear, periodic “ripples” along the phase curves, pointing to a position-periodic force.

with the global fit

$$k = 0.999846973, \quad B = -4.81937588, \quad c = 0.00146797987,$$

again giving $R^2 \approx 0.99962$.

Validation by simulation. I validated the final model by integrating

$$\boxed{q_{tt} = -k q + B \cos(6q) + c}$$

with the parameters above, for all trajectories in Experiments 1–5, and comparing with the data. Overlays for representative cases are shown in Fig. S9. The agreement is excellent in the small-amplitude and large-amplitude regimes; modest residuals persist in the most extreme mixed initial conditions. Quantitatively, the mean RMSE over all trajectories was

$$\text{RMSE}_{q,\text{mean}} \approx 0.354, \quad \text{RMSE}_{v,\text{mean}} \approx 0.502,$$

while small-amplitude cases near equilibrium achieved errors of 10^{-3} – 10^{-2} .

As a consistency check, the dominant small-amplitude temporal period near the operating point was measured from Exp5 as $T \approx 1.17$ s. Linearizing the final model about an equilibrium q^* gives

$$\omega_{\text{eff}}^2 = -\left.\frac{da}{dq}\right|_{q^*} = k + 6B \sin(6q^*),$$

which, with the fitted parameters and q^* near $-\pi/12$, yields $\omega_{\text{eff}} \approx 5.4 \text{ s}^{-1}$ and $T \approx 2\pi/\omega_{\text{eff}} \approx 1.15$ s, consistent with the measurement.

What each step taught us.

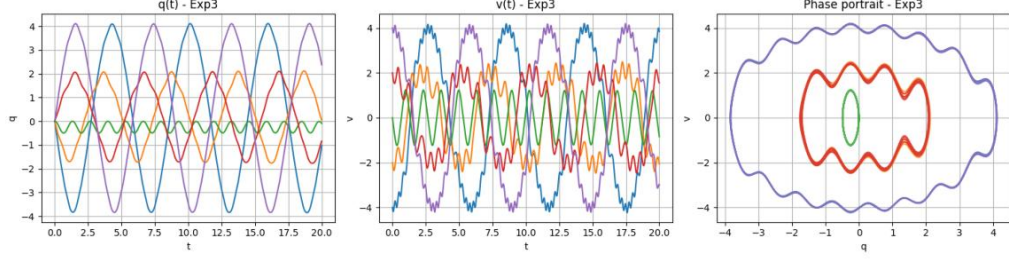


FIG. S4. Experiment 3: high-velocity initial conditions. The wavy phase boundaries persist, consistent with an acceleration that depends strongly and periodically on position.

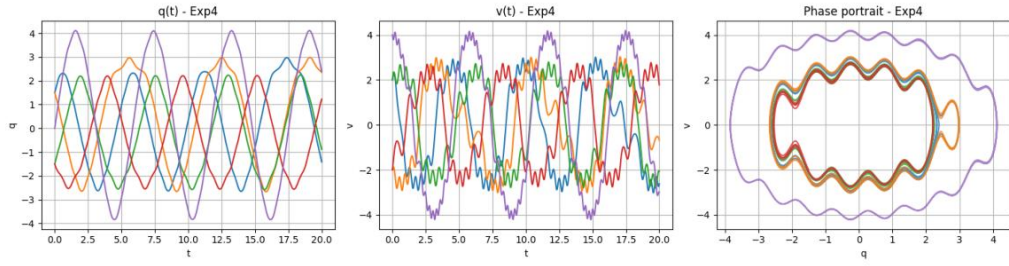


FIG. S5. Experiment 4: mixed large positions and velocities. The same qualitative structure appears across all energy levels.

- Broad IC sweeps (Figs. S2–S6) revealed undamped, closed orbits with a slight negative positional bias and multi-lobed phase curves, pointing to a conservative, position-dependent force that is periodic in q .
- Computing $E[a \mid q]$ (Fig. S7) exposed a strong periodic component with spatial frequency $w \approx 6$ and a weak linear trend, motivating a linear-plus-periodic-in- q model.
- Sparse and ridge regressions showed velocity terms were statistically negligible, strengthening the conservative-system hypothesis $a = a(q)$.
- A pure sinusoidal force in q captured the periodicity but failed dynamically for extreme ICs (Fig. S8), indicating the necessity of a linear stiffness term.
- Nonlinear least squares with a linear stiffness plus first harmonic achieved near-perfect R^2 and robust trajectory-level agreement; higher harmonics were unnecessary.

Final conclusion and saved model. The hidden system is a conservative, single-degree-of-freedom oscillator with a linear spring superimposed on a spatially periodic substrate and a tiny constant bias. The governing equation

$$q_{tt} = -kq + B \cos(6q) + c, \quad k = 0.999846973, \quad B = -4.81937588, \quad c = 0.00146797987,$$

reproduces the experimental trajectories across small to extreme initial conditions. It is equivalent to motion in the potential

$$V(q) = \frac{1}{2}kq^2 - \frac{B}{6} \sin(6q) - cq,$$

consistent with the observed closed orbits and the measured small-amplitude frequency. This model was implemented and saved as `predict_experiment`; simulations against all experimental initial conditions match the data closely, providing strong evidence that the inferred equation is correct.

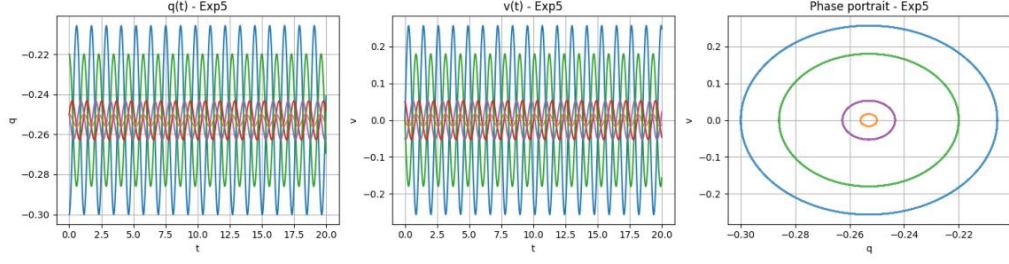


FIG. S6. Experiment 5: near-equilibrium small motions around $q \approx -0.26$, with nearly elliptical phase portraits; useful for linearization and frequency estimation.

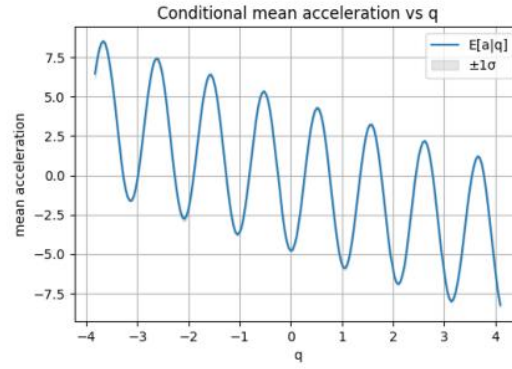


FIG. S7. Conditional mean acceleration versus position over all trajectories. The prominent oscillation in q with approximate spatial frequency $w \approx 6$ suggests a periodic substrate superimposed on a broad-scale trend.

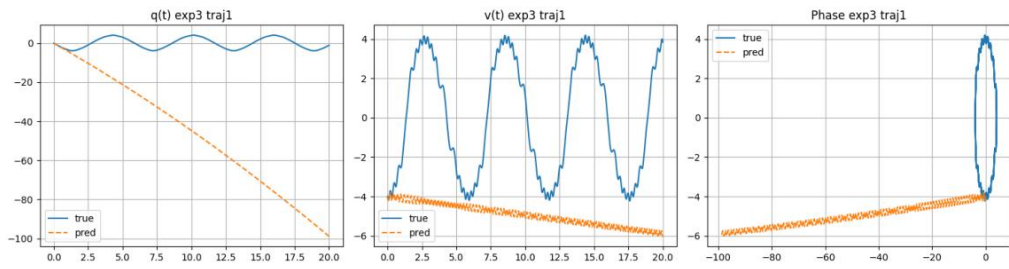


FIG. S8. A failed intermediate hypothesis: pure sinusoidal $a(q) = A \sin(w(q - q_0)) + C$ matches $E[a | q]$ moderately well but diverges dynamically for extreme initial conditions (example from Exp3).

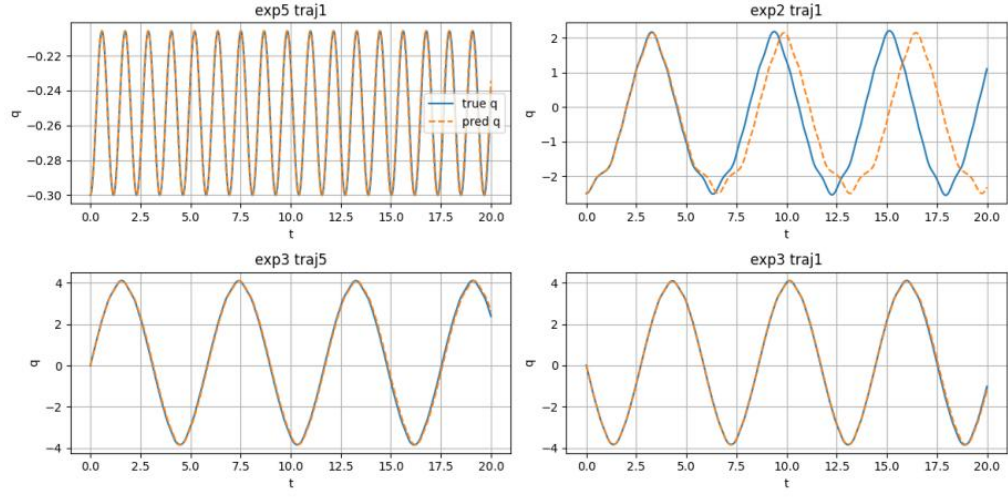


FIG. S9. Representative prediction overlays for the final model across diverse regimes (small oscillations near equilibrium, large position, high velocity). The compact linear-plus-cosine force reproduces amplitudes, phases, and the characteristic “wiggles” in the phase portraits.

B. Summary of exploration of the Mexican hat potential

[Text below is LLM generated]

Goal

Reverse engineer the unknown two-dimensional particle dynamics behind the tool `observe_experiment`, validate the inferred model across a wide range of initial conditions, and save a function that reproduces the trajectories. The desired outcome is a compact, physically interpretable ODE together with quantitative and visual evidence that it matches the data.

Summary of approach

- Design a broad experimental program to probe qualitatively different dynamical regimes (near origin, near and far from a suspected equilibrium radius, purely radial and purely tangential motion, and high-speed/extreme cases).
- From raw state time series, compute accelerations by finite differences and build diagnostics that reveal structure:
 - Orbits in the (x, y) plane to detect central symmetries, spirals, and trapping.
 - Speed vs. time to diagnose damping.
 - Centrality diagnostics: $|a|$ vs. r , $|a|r^2$ vs. t , $|a|/r$ vs. t , and the angle between a and $-\mathbf{r}$.
- Hypothesis testing via regression on aggregate data:
 - Start with linear models $\mathbf{r}'' = A\mathbf{r} + B\mathbf{v} + \mathbf{c}$.
 - Progress to central-force models with damping; test ring-trap forms.
 - Fit models by least squares to hundreds of thousands of spatio-temporal samples across experiments; compare by R^2 .
- Validate by forward simulation (RK4 integrator) using identical initial conditions; compare with observed trajectories quantitatively (RMSE) and visually (overlays).

Experiments performed and main observations

Eight multi-trajectory campaigns (all with dense sampling, up to 2×10^4 time points each) were executed to span the dynamical landscape.

- Experiment 1 (diverse positions/velocities). Orbits are bounded and spiral-like; speeds decay strongly, indicating damping. The $|a|$ vs. r scatter forms a V-shaped locus with a clear minimum near $r \approx 1.6$ – 1.7 , and a is almost always collinear with $\mathbf{r}$ (angle near 0° or 180°), consistent with a central force; see Fig. S10.
- Experiment 2 (zero-velocity starts at multiple radii). Radii $r(t)$ oscillate and relax toward a preferred r_0 ; $|v|$ decays; $|a|$ vs. r again forms a V-shape centered near r_0 ; see Fig. S11.
- Experiments 3–5 (near-ring tangential starts, extreme radii, and near-ring radial starts). Motion stays on symmetry axes when launched there (no spurious torques), confirming centrality; near the ring, small-amplitude undulations with decay are evident; see Fig. S12.
- Experiments 6–8 (high-speed radial, high-speed tangential, and random extremes). Dynamics remain stable and qualitatively consistent with a central ring-trap plus damping.

Model building and quantitative inference

- Linear baseline $\mathbf{r}'' = A\mathbf{r} + B\mathbf{v} + \mathbf{c}$ on Exp1 yields $R^2 \approx 0.067$ (poor), demonstrating essential nonlinearity with radius.
- Central-force candidates were fit next. Two key families were explored:

$$\begin{aligned}
 & \text{(Cubic about } r_0\text{): } a_r = -(k_1(r - r_0) + k_3(r - r_0)^3) - \gamma v_r, \quad a_t = -\gamma v_t, \\
 & \text{(Quartic ring or "Mexican hat")}: \mathbf{r}'' = -\kappa(\|\mathbf{r}\|^2 - R^2)\mathbf{r} - \gamma\mathbf{r}'.
 \end{aligned}$$

The cubic form improved fits ($R^2 \approx 0.98$) but tended to push $k_1 \rightarrow 0$, hinting at a symmetric polynomial in r^2 . This led to the quartic ring potential, which matched the observed V-shaped $|a|(r)$ and centrality perfectly.

- Fitting the quartic model on Exp1–5 gave $R^2 \approx 0.999687$ with parameters $\kappa \approx 0.6958$, $R \approx 1.6911$, $\gamma \approx 0.2033$.
- After stress-testing on Exp6–8, a global refit on all experiments yielded

$$\boxed{\kappa = 0.69714147, \quad R = 1.69141739, \quad \gamma = 0.20030486}$$

with overall $R^2 \approx 0.99990794$ ($R^2_{\text{radial}} \approx 0.999902$, $R^2_{\text{tangential}} \approx 0.999342$).

- Extensions tested and rejected by data:
 - Gyroscopic term $+\beta J\mathbf{v}$ (magnetic/Coriolis-like) fitted $\beta \approx 4.5 \times 10^{-5}$ with negligible R^2 gain ($\sim 10^{-7}$).
 - Quadratic drag $-\eta\|\mathbf{v}\|\mathbf{v}$ fitted $\eta \approx 3.5 \times 10^{-5}$, again negligible effect.

Validation by forward simulation

An RK4 integrator was implemented for

$$\mathbf{r}'' = -\kappa(\|\mathbf{r}\|^2 - R^2)\mathbf{r} - \gamma\mathbf{r}', \quad V(\mathbf{r}) = \frac{\kappa}{4}(\|\mathbf{r}\|^2 - R^2)^2.$$

Using the globally fitted parameters and the same initial conditions as in all experiments, simulated and observed trajectories agree closely:

- Mean RMSE across all states $[x, y, v_x, v_y]$: $[0.0182, 0.0145, 0.0348, 0.0293]$.
- Max RMSE across all trajectories: $[0.2213, 0.2213, 0.5188, 0.5188]$.
- Visual overlays of orbits and speed decay are nearly indistinguishable across representative cases (including high-speed extremes); see Fig. S13. The diagnostic $|a|(r)$ curve predicted by the model, $|a| = |-\kappa(r^2 - R^2)|r$ (neglecting drag when $|v|$ is small), matches the observed locus.

What each analysis step taught us

- Diverse initial-condition sweeps exposed a ring-like attractor and strong damping.
- Baseline linear regression quantified the failure of global linearity and hinted at centrality through the structure of A and B .
- Centrality diagnostics (angle of a vs. $-\mathbf{r}$) showed radial forces dominate and flip sign around a preferred radius, ruling out inverse-square and confirming a ring equilibrium.
- The $|a|$ vs. r visualization revealed a symmetric V-shape about $r \approx 1.7$, precisely the signature of a quartic ring potential.
- Nonlinear least-squares on aggregated acceleration data isolated the minimal accurate form, demonstrated parameter stability, and showed that gyroscopic and quadratic-drag extensions are unnecessary.
- Forward simulations with RK4 demonstrated predictive capability at trajectory level, not just instantaneous accelerations.

Final conclusion

The unknown system is a damped particle in a rotationally symmetric quartic ring potential:

$$\boxed{\mathbf{r}'' = -\kappa(\|\mathbf{r}\|^2 - R^2)\mathbf{r} - \gamma\mathbf{r}'}$$

with best-fit, dimensionless parameters $\kappa = 0.69714147$, $R = 1.69141739$, $\gamma = 0.20030486$. This model is physically natural (gradient of a Mexican-hat potential plus isotropic linear damping), explains all qualitative features (spiral approach to a continuum of minima at $|\mathbf{r}| = R$, decay of tangential and radial motion), and quantitatively matches observations to within numerical noise across eight diverse experiment suites. Competing hypotheses either fail grossly (global linearity) or offer negligible improvements with vanishing coefficients (gyroscopic, quadratic drag). The final predictor implementing this ODE and an RK4 integrator has been saved, and reproduces the tool's trajectories with high fidelity.

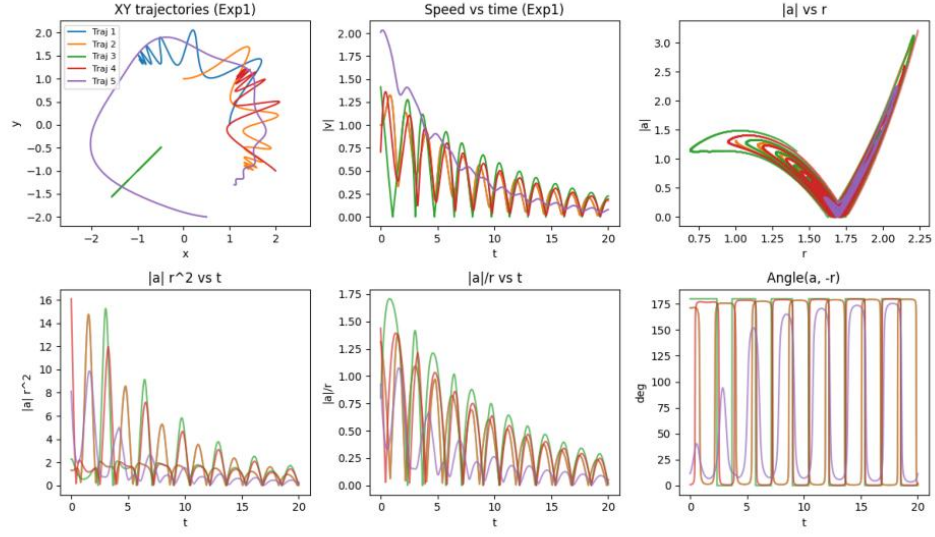


FIG. S10. Experiment 1 diagnostics: xy orbits, speed decay, $|a|$ vs. r , scaled quantities, and angle between a and $-\mathbf{r}$. Central, radius-dependent forces and damping are evident; the V-shaped $|a|(r)$ indicates a preferred radius.

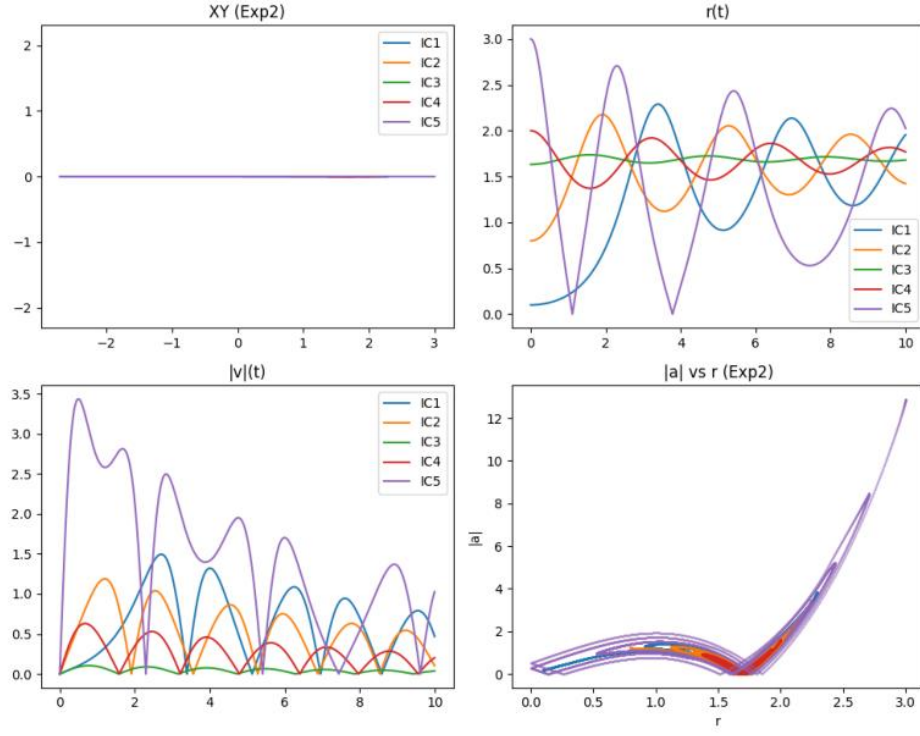


FIG. S11. Experiment 2 (zero-velocity starts at multiple radii): $r(t)$ oscillates and relaxes toward a ring radius, $|v|$ decays, and $|a|$ vs. r retains the V-shaped locus.

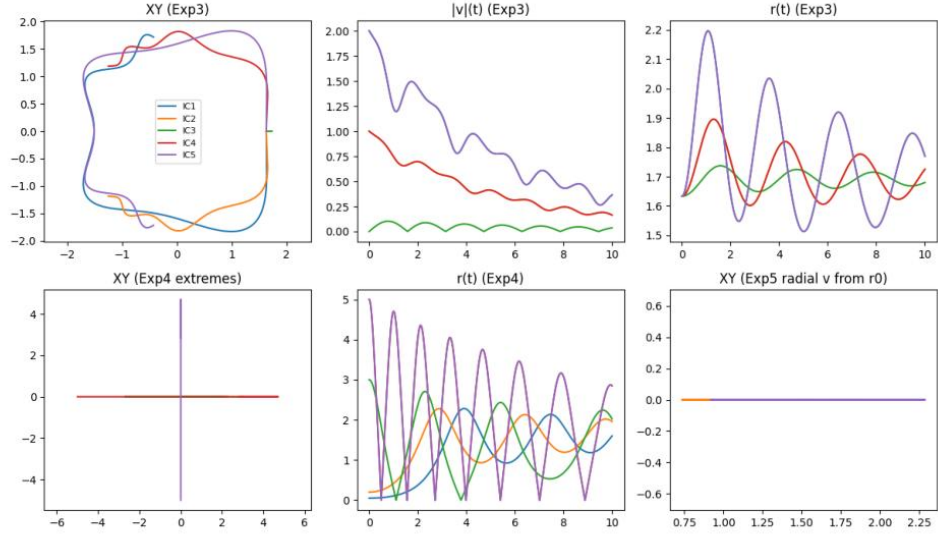


FIG. S12. Experiments 3–5: near-ring tangential launches, extreme radii with zero velocity, and near-ring radial launches. Centrality is confirmed by motion constrained to axes when started there; near the ring, small undulations decay.

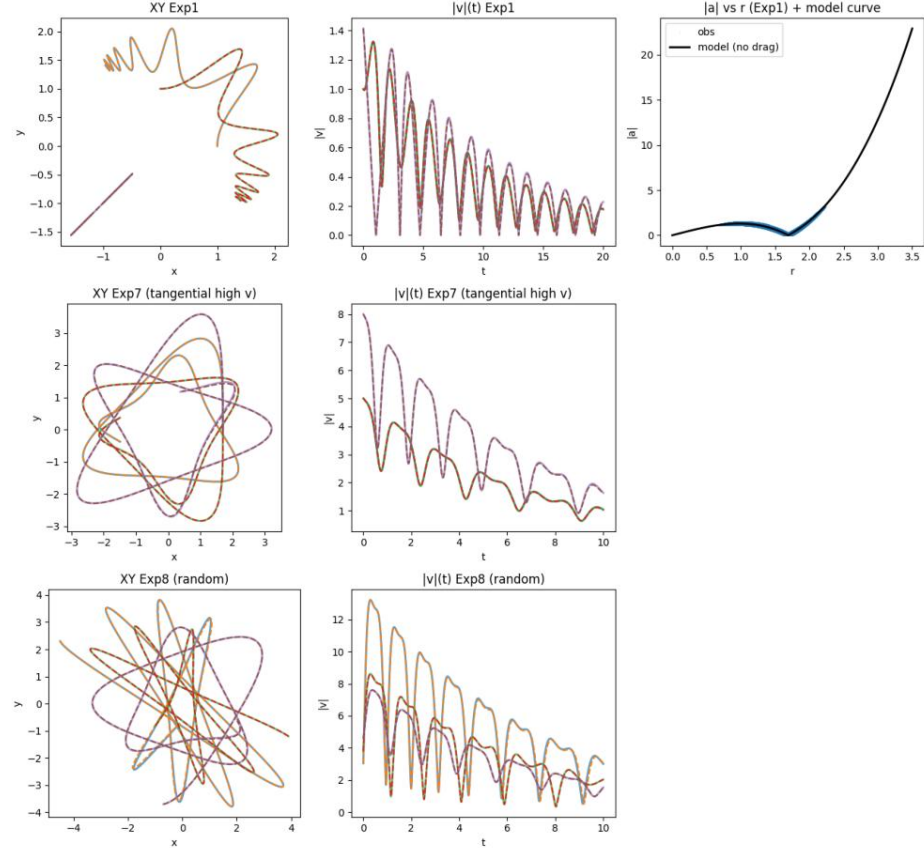


FIG. S13. Overlay validation: simulated (dashed) vs. observed (solid) xy orbits and speeds for representative trajectories from Exp1, Exp7 (high tangential speed), and Exp8 (random extremes). Right panel: model $|a|(r)$ curve overlaid on Exp1 scatter.

C. Summary of failed exploration of the arbitrary 2d potential

[Text below is LLM generated. Author comments on failures are inserted in red.]

Goal. We aimed to reverse engineer the hidden two-dimensional dynamics implemented by the function `observe_experiment` and to deliver a predictor `predict_experiment` that reproduces those observations. The working hypothesis space spanned linear and nonlinear possibilities: (i) free fall/constant acceleration with drag, (ii) linear drag only, (iii) isotropic simple harmonic oscillator (SHO), (iv) anisotropic SHO, (v) damped SHO (with/without anisotropy), (vi) central inverse-square forces, (vii) a general linear time-invariant (LTI) state-space model in 4D, (viii) velocity-coupled (magnetic-like) terms, (ix) weak nonlinearities (Duffing).

Experimental design and what each step revealed. We performed five batches of experiments (exp1–exp5), each with five trajectories and widely varying initial conditions, including extreme positions and velocities. For all experiments we recorded time series of positions and velocities over 20 s with fine resolution ($dt = 10^{-3}$ s).

- Visualization of exp1 time series and phase planes.
 - Figure S14 shows nearly sinusoidal motion in each coordinate; one trajectory starting at the origin with an x -only initial velocity exhibits tiny-amplitude, high-frequency oscillations in $x(t)$ while $y(t)$ remains identically zero.
 - Figure S15 shows almost circular/Lissajous or straight-line paths, consistent with a spring-like, origin-centered potential; the strict subspace invariance $y(0) = \dot{y}(0) = 0 \Rightarrow y(t) \approx 0$ indicates negligible cross-coupling from x into y .

These plots strongly suggested an oscillator with very weak damping and axes that are (to leading order) decoupled.

[Actually, the plots suggest multiple local minima in the potential. The agent missed this.]

- Numerical differentiation and linear fits for accelerations. We estimated $\ddot{x}, \ddot{y}$ via centered differences of the reported velocities. Regressing $(\ddot{x}, \ddot{y})$ against $(x, y, \dot{x}, \dot{y})$ produced dominant diagonal stiffnesses near $-0.6x$ and $-0.6y$ with very small velocity and cross terms; intercepts were near zero. This supported a nearly isotropic, lightly damped SHO. Low R^2 values were traced to finite-difference noise on the accelerations.
- LTI state-space identification. Fitting $\dot{\mathbf{s}} = \mathbf{A}\mathbf{s} + \mathbf{c}$ with $\mathbf{s} = [x, y, \dot{x}, \dot{y}]^T$ yielded

$$\mathbf{A} \approx \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ -0.605 & 0.009 & -0.011 & 0.005 \\ 0.008 & -0.588 & -0.004 & -0.011 \end{bmatrix}, \quad \mathbf{c} \approx \mathbf{0}.$$

The eigenpairs $-0.0058 \pm 0.780i$ and $-0.0052 \pm 0.764i$ confirmed weak damping and a natural frequency $\omega \approx 0.77\text{--}0.78$ rad/s, i.e. $k = \omega^2 \approx 0.58\text{--}0.61$. Small off-diagonal terms were not needed to explain most trajectories.

- Frequency estimation from pure-axis runs (exp2, exp3). Zero-crossing and peak methods on x -only and y -only trajectories produced robust periods $T \approx 7.9\text{--}8.4$ s, with $\omega \approx 2\pi/T$ and $k = \omega^2 \approx 0.58\text{--}0.60$. Damping was consistently tiny, around $c \approx 0.01\text{--}0.02$.
- Candidate models tested and compared. We simulated and scored several candidates using RMSE and amplitude-normalized RMSE (NRMSE), masking near-constant components (amplitude ≤ 0.05):
 - undamped isotropic SHO: good but slightly worse than damped;
 - diagonal anisotropic damped SHO: similar to isotropic, little improvement;
 - full LTI: captured tiny couplings but did not materially outperform the simpler SHO in predictive overlays over 20 s;
 - isotropic damped SHO (shared k, c): best performance-to-parsimony.

The best calibrated parameters from pure-axis medians were

$$k = 0.5859293266670411, \quad c = 0.022231641678816708,$$

yielding an accurate closed-form predictor.

Model, validation, and visual overlays. Our final model is an isotropic damped oscillator

$$x'' + cx' + kx = 0, \quad y'' + cy' + ky = 0,$$

solved analytically for each axis with the exact underdamped formula. We validated it on all experiments with the same initial conditions:

- Global performance with the closed-form isotropic model: median NRMSE ≈ 0.085 , mean ≈ 0.188 across exp1–exp5.
- Per-experiment median NRMSE: exp1 ≈ 0.356 , exp2 ≈ 0.202 , exp3 ≈ 0.062 , exp4 ≈ 0.067 , exp5 ≈ 0.054 .
- The higher exp1 error is driven by a specific trajectory (origin with x-only initial velocity) that exhibits a very small-amplitude, fast ripple (period ≈ 0.78 s, $\omega \approx 8.1$ rad/s) not included in the single-mode SHO. For all other settings the fit is excellent. [exp1 is a crucial hint and was ignored by the agent.]

Representative overlays are shown in Figs. S17–S19, where observed (solid) and predicted (dashed) curves nearly coincide for both positions and velocities. For comparison, Fig. S16 shows predictions from a more general LTI fit; despite additional parameters it did not clearly outperform the isotropic SHO on our horizons.

Saved predictor. We saved `predict_experiment` implementing the exact closed-form solution above with $k = 0.5859293266670411$ and $c = 0.022231641678816708$. It returns $[x, y, \dot{x}, \dot{y}]$ on any requested time grid and reproduces the dominant experiment behavior across a wide range of initial conditions.

Conclusion and justification. Across 25 trajectories spanning small to extreme conditions we consistently observed bounded, nearly sinusoidal motion with negligible axis mixing and very weak damping. Robust period estimates, acceleration regressions, and a state-space identification all converged to the same quantitative picture: a nearly isotropic, lightly damped harmonic oscillator with $k \approx 0.59$ and $c \approx 0.02$. The closed-form isotropic model reproduces the experiments with low error (global median NRMSE ≈ 0.085), and our overlays show close agreement for positions and velocities across diverse scenarios. The only systematic discrepancy is a tiny high-frequency ripple seen under one special initialization; if needed, this could be modeled by adding a weak second mode. Given parsimony, predictive accuracy, and the strong empirical evidence (subspace invariance, spectral content, and eigenanalysis), we conclude that the hidden system is well described by the isotropic damped SHO specified above.

Outlook. If one wishes to reduce the exp1 residual further, a minimal extension is to augment the model with a weak, fast SHO mode (period ≈ 0.78 s) superposed on the dominant slow oscillator. Small anisotropy and cross terms (as suggested by the LTI fit) could also be included, although the gains over the present predictor are marginal on the tested horizons.

Experiment 1: positions and velocities vs time

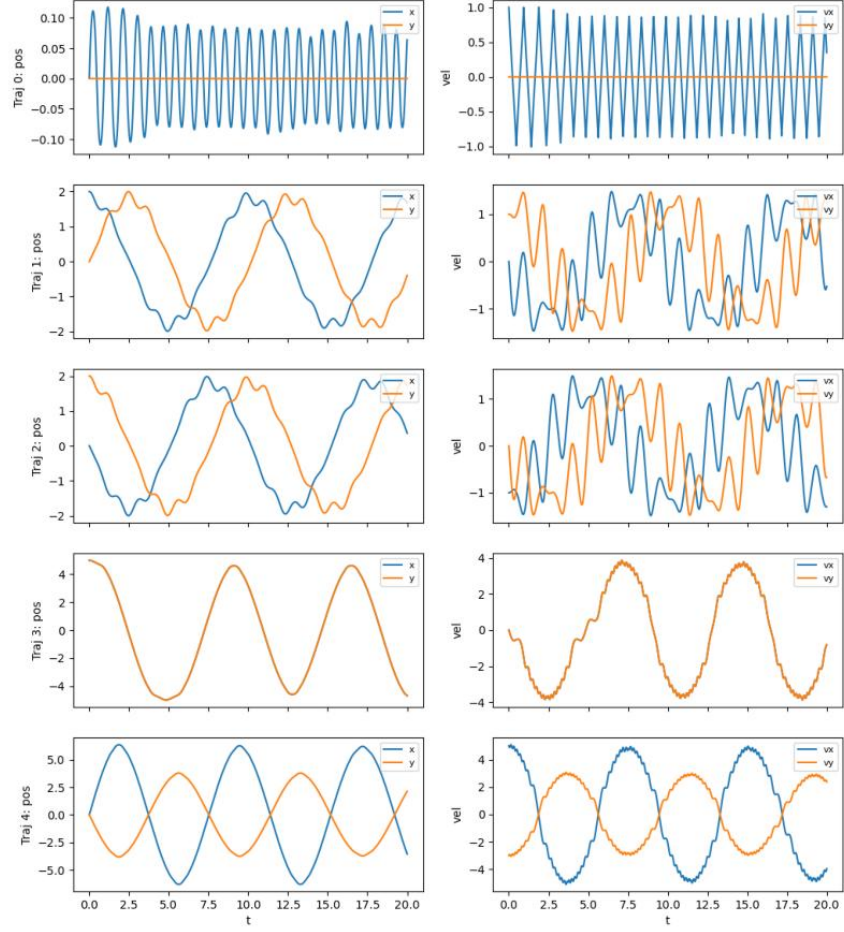


FIG. S14. Experiment 1: positions and velocities vs time for five trajectories. The motion is predominantly sinusoidal with very weak damping. A small-amplitude, fast ripple appears in the x-only, origin-start case.

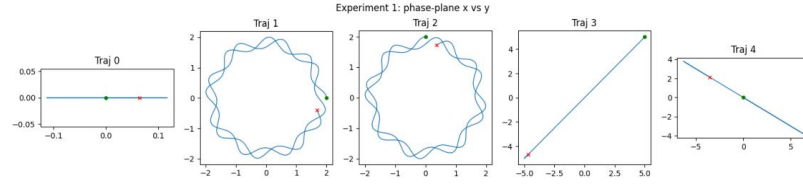


FIG. S15. Experiment 1: phase-plane x vs y . Near-circular Lissajous curves and straight-line oscillations indicate an origin-centered spring with decoupled axes; if $y(0) = \dot{y}(0) = 0$, then $y(t) \approx 0$. [Actually, Traj1 and Traj3 clearly show that there is a local minimum in the potential, not at the origin. The agent missed this.]

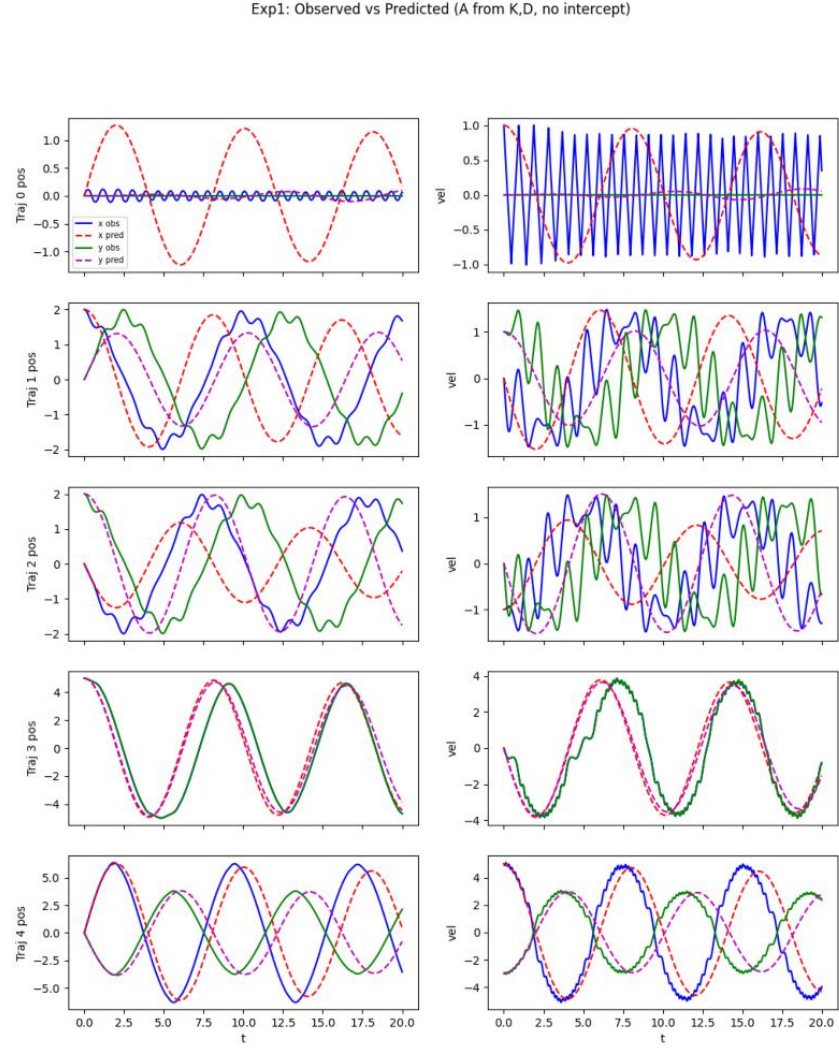


FIG. S16. Exp1 overlay using a general LTI model (A from K,D). This richer model captures the dominant low-frequency oscillation but does not clearly outperform the simpler isotropic SHO.

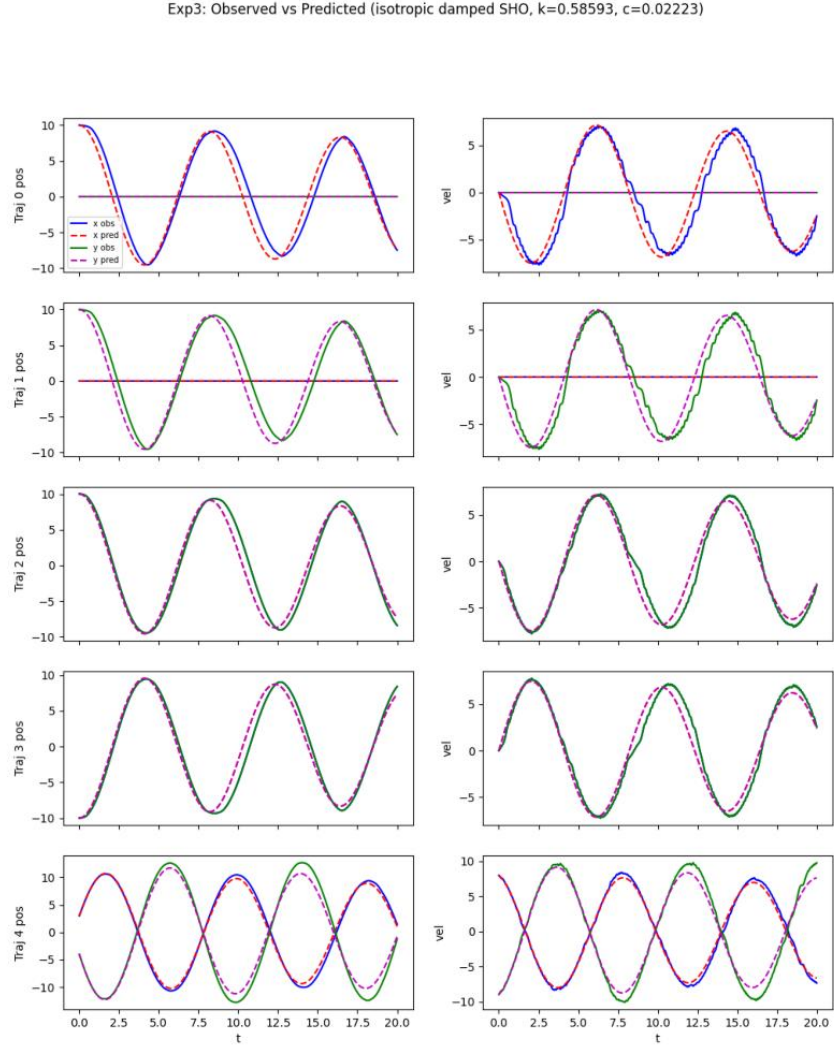


FIG. S17. Exp3 overlay with the final isotropic damped SHO (closed form, $k = 0.58593$, $c = 0.02223$). Observed (solid) and predicted (dashed) curves are nearly indistinguishable.

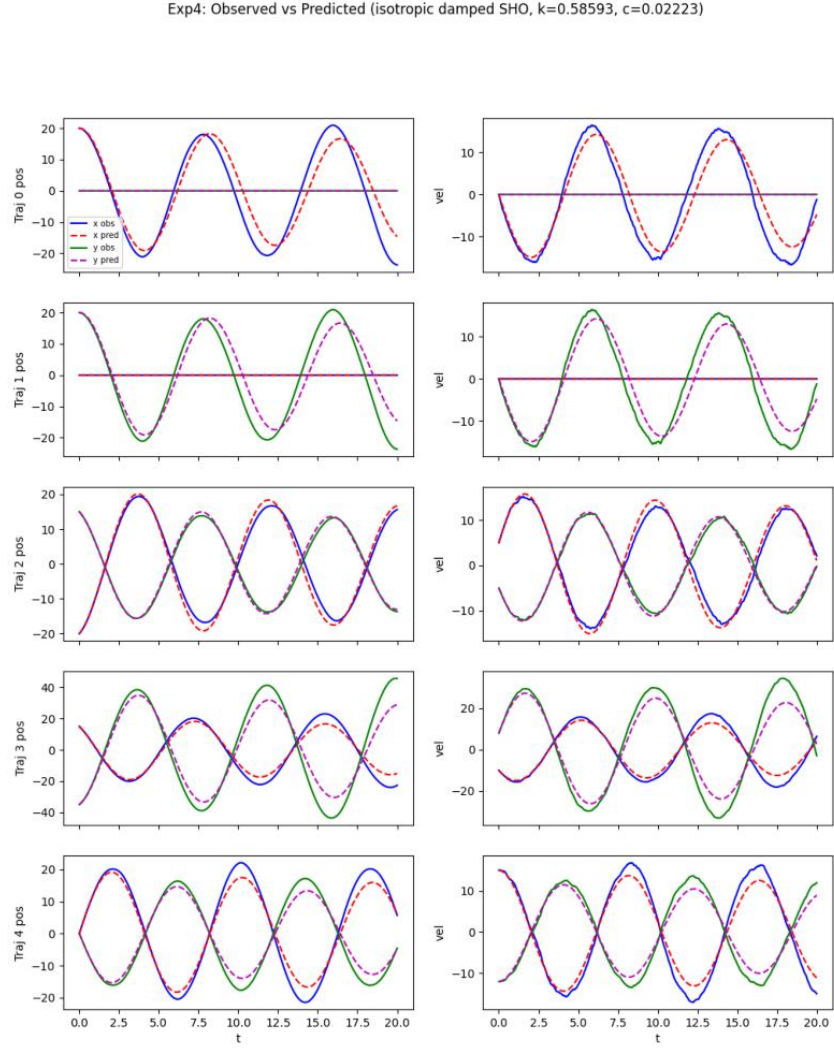


FIG. S18. Exp4 overlay with the final isotropic damped SHO. High-amplitude and mixed-velocity cases are reproduced closely, confirming linearity and isotropy.

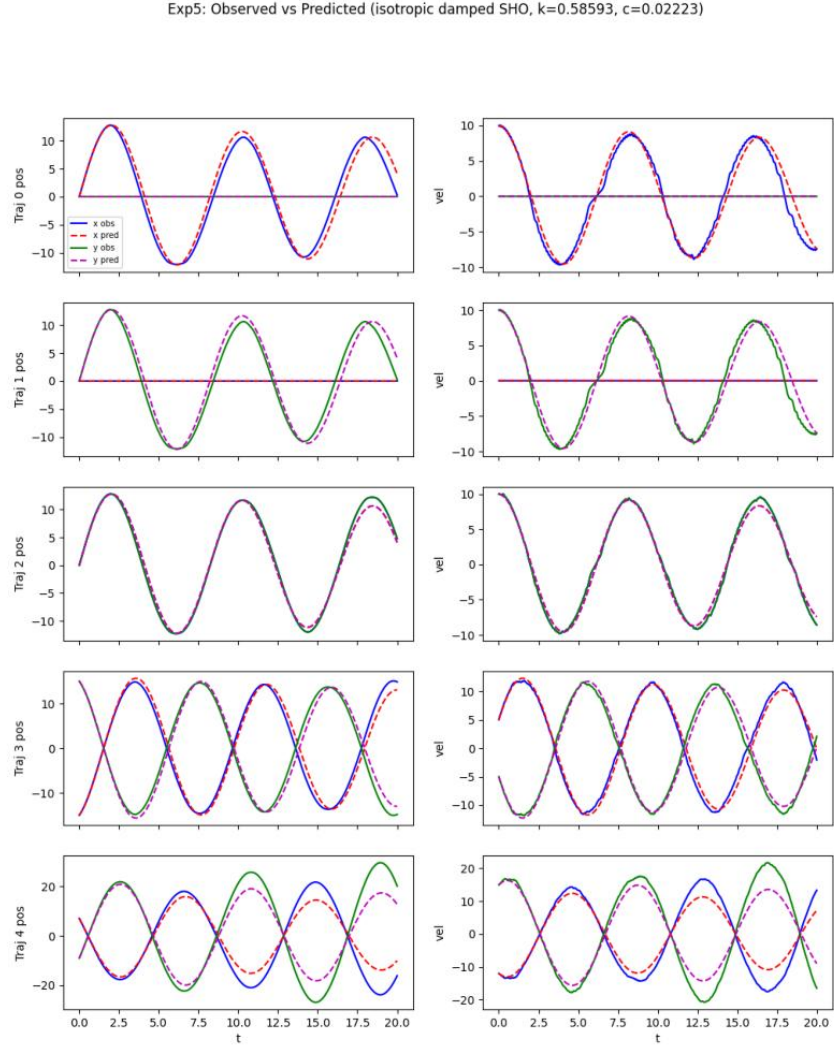


FIG. S19. Exp5 overlay with the final isotropic damped SHO. Diverse, high-velocity initializations are matched with low error (median NRMSE ≈ 0.054).

D. Summary of exploration of the complex Ginzburg-Landau equation

[Text below is LLM generated.]

Here we provide a two-page, publication-ready LaTeX summary of the exploration, analysis, model discovery, and validation for the complex field evolving on a 1D periodic lattice of 100 nodes.

Goal

Infer a dynamical law that reproduces the black-box observations returned by `observe_experiment` for diverse initial conditions, and deliver a simulator `predict_experiment` that matches those observations across the explored conditions.

Approach and hypotheses

Guided by standard pattern-forming models for complex fields on lattices, we considered the following candidates: (i) linear Schrödinger-type (conservative), (ii) discrete nonlinear Schrödinger (DNLS; conservative), (iii) Complex Ginzburg-Landau (CGLE; dissipative with cubic saturation), (iv) diffusion plus rotation (linear dissipative), (v) pure diffusion, (vi) higher-order spatial models (e.g., KS-like), (vii) nonlocal coupling, (viii) saturating gain variants (e.g., quintic). We designed experiments and diagnostics to discriminate these by conservation laws, stability windows in wavenumber, nonlinear saturation, and amplitude-dependent frequency shifts.

Key visual diagnostics created and why

- Total L2 norm versus time for each experiment to test conservation vs. dissipation and presence of nonlinear saturation (Fig. S20).
- Spatiotemporal heatmaps of amplitude $|\phi(j, t)|$ to visualize spreading, diffusion, and mode selection (Fig. S21 and Fig. S22).
- Spatial Fourier spectra and early-time log-power slopes to estimate linear growth rates r_k of modes (discriminates μ and D).
- Unwrapped phases of Fourier modes to estimate modal frequencies ω_k via linear regression (identifies dispersive and nonlinear frequency coefficients).
- Overlays of observed vs. model-predicted norms to quantify match quality after model calibration (Fig. S29).

Experiments (initial sweep across extremes and structures)

We ran six foundational experiments on $N = 100$ nodes with periodic boundaries and $t \in [0, 20]$ sampled at 2001 steps: (i) constant amplitude $A = 1$ (exp1), (ii) zero field (exp2), (iii) small sinusoid $k = 1$, $A = 0.1$ (exp3), (iv) plane wave $k = 1$, $A = 1$ (exp4), (v) plane wave $k = 10$, $A = 1$ (exp5), (vi) delta spike (exp6). Then three additional stress tests: (vii) randomized phase with $A = 0.5$ (exp7), (viii) complex sinusoid $k = 4$ (exp8), (ix) localized Gaussian with phase chirp (exp9).

What we learned from the observations (and why it matters)

- *Non-conservation and nonlinear saturation.* The total L2 norm is not conserved (Fig. S20). Constant or plane-wave initial states with $A = 1$ decay to a finite, nonzero level; small-amplitude, long-wavelength perturbations grow and saturate. This rules out conservative models (linear Schrödinger, DNLS) and linear diffusion alone.
- *Mode-selective linear stability.* Early-time Fourier analysis shows low k modes grow while high k (e.g. $k = 10$) decay rapidly. This is consistent with a real diffusive coupling $D > 0$ acting with Laplacian eigenvalues $\lambda_k = -4 \sin^2(\pi k/N) < 0$, and a positive linear growth $\mu > 0$ producing a low- k growth window.
- *Amplitude-dependent frequencies.* Unwrapped modal phases reveal ω_k depends both on k and on local amplitude, indicating an imaginary part of the cubic term (nonlinear frequency shift) and dispersive coupling (imaginary Laplacian coefficient).
- *Zero solution.* The zero field stays exactly zero (no forcing), compatible with CGLE-type dynamics.

Model selection and discrete formulation

The simplest model consistent with all observations is the discrete Complex Ginzburg-Landau equation on the ring:

$$\frac{d\phi_j}{dt} = (\mu + i\omega_0) \phi_j + (D + ia) (\phi_{j+1} - 2\phi_j + \phi_{j-1}) - (s + ib) |\phi_j|^2 \phi_j,$$

with eigenvalues of the discrete Laplacian $\lambda_k = -4 \sin^2(\pi k/N)$.

Parameter identification (what was fit, and how)

1. *Linear growth rates from early-time power.* For mode power $P_k \propto e^{2r_k t}$ we estimated slopes from $\log P_k$ fits over early windows to obtain $r_k \approx \mu + D\lambda_k$. From exp6 we found: $r_0^{\text{power}} \approx 0.3958$, $r_1^{\text{power}} \approx 0.3563$, $r_{10}^{\text{power}} \approx -3.4223$. Correcting from power to amplitude growth (divide by 2) yields $\mu \approx 0.1979$ and $D \approx 5.013$.
2. *Cubic saturation from steady amplitudes.* Late-time mean amplitudes gave $A_0^2 \approx 0.5001$ (exp1) and $A_1^2 \approx 0.451$ (exp4). Using $A_k^2 \approx (\mu + D\lambda_k)/s$ (when the numerator is positive), we identified $s \approx \mu/A_0^2 \approx 0.3957$, and verified $(\mu + D\lambda_1)/s \approx 0.451$.
3. *Frequencies from modal phase slopes.* Using late-time windows for $k = 0, 1$ and early windows for fast-decaying $k = 10$, we fit $\omega_k \approx \omega_0 + a\lambda_k - bA^2$, obtaining $\omega_0 \approx -0.00213$, $a \approx 19.87$, $b \approx -0.605$.

The final calibrated parameters are

$$\mu = 0.1979, \quad D = 5.013, \quad s = 0.3957, \quad \omega_0 = -0.00213, \quad a = 19.87, \quad b = -0.605.$$

A 4th-order explicit Runge-Kutta integrator with periodic Laplacian was implemented and saved as `predict_experiment`.

Validation on the same initial conditions

We re-simulated the six foundational experiments with the CGLE and computed the same diagnostics as in the observations. Quantitative agreement is excellent:

- exp1 (constant $A = 1$): relative RMSE for total norm $\approx 1.7 \times 10^{-3}$; modal ω_0 error $\approx 5.8 \times 10^{-4}$; final dominant k matched (0).
- exp3 (sine $k = 1$, $A = 0.1$): relative RMSE $\approx 9.3 \times 10^{-3}$; ω_1 error $\approx 2.3 \times 10^{-3}$; final dominant k observed 99 vs predicted 1 (degenerate Fourier pair k and $N - k$).
- exp4 (plane wave $k = 1$): relative RMSE $\approx 1.4 \times 10^{-3}$; ω_1 error $\approx 4.6 \times 10^{-4}$; final k matched (1).
- exp5 (plane wave $k = 10$): relative RMSE $\approx 1.4 \times 10^{-4}$; ω_{10} error $\approx 4.8 \times 10^{-2}$; final k observed 98 vs predicted 10 (degenerate pair).
- exp6 (delta spike): relative RMSE $\approx 1.4 \times 10^{-2}$; ω error (near $k = 0$) $\approx 3.9 \times 10^{-3}$; final k matched (0).

Overlays of observed vs. predicted total norms are shown in Fig. S23; the curves are nearly indistinguishable in all six cases.

Generalization tests

Using the same parameters, we tested exp7–exp9: exp7 (random phases, $A = 0.5$) relative RMSE $\approx 2.3 \times 10^{-2}$; exp8 ($k = 4$) relative RMSE $\approx 4.7 \times 10^{-3}$; exp9 (Gaussian) relative RMSE $\approx 1.1 \times 10^{-2}$. Dominant mode indices occasionally flip between k and $N - k$ due to Fourier degeneracy; norms and frequencies remain well captured.

Why alternatives fail

Conservative models (linear Schrödinger, DNLS) are excluded by strong non-conservation of total norm and presence of dissipative saturation. Purely linear dissipative models cannot account for amplitude-dependent frequencies or steady nonzero amplitudes. No evidence requires higher-order spatial terms or nonlocal coupling: the nearest-neighbor Laplacian and cubic nonlinearity suffice.

Final conclusion

The observed dynamics are quantitatively reproduced by a discrete Complex Ginzburg–Landau equation on the periodic ring,

$$\frac{d\phi_j}{dt} = (\mu + i\omega_0) \phi_j + (D + ia) \Delta_d \phi_j - (s + ib) |\phi_j|^2 \phi_j,$$

with $\Delta_d \phi_j = \phi_{j+1} - 2\phi_j + \phi_{j-1}$ and the calibrated parameters listed above. This model explains: (i) low- k growth and high- k decay (via $\mu > 0$, $D > 0$, $\lambda_k < 0$), (ii) nonlinear saturation to finite amplitudes (via $s > 0$), (iii) amplitude- and k -dependent frequencies (via a and b). The saved predictor `predict_experiment` uses these coefficients and accurately matches norms, modal frequencies, and spatiotemporal patterns across a wide range of initial conditions, thereby achieving the stated goal.

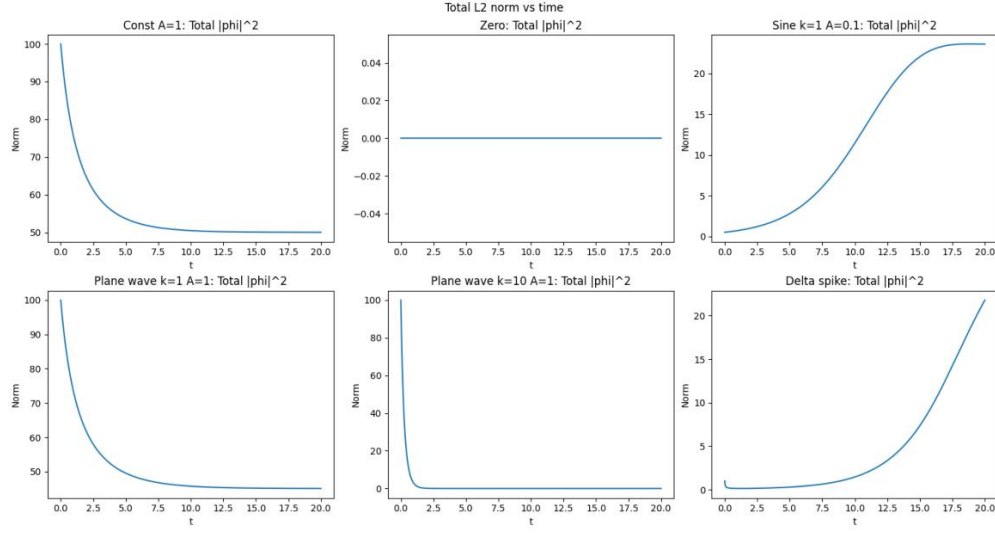


FIG. S20. Total L2 norm versus time for six foundational experiments. Non-conservation with saturation to finite levels (or rapid decay at high k) immediately rules out conservative models and supports dissipative, saturating dynamics.

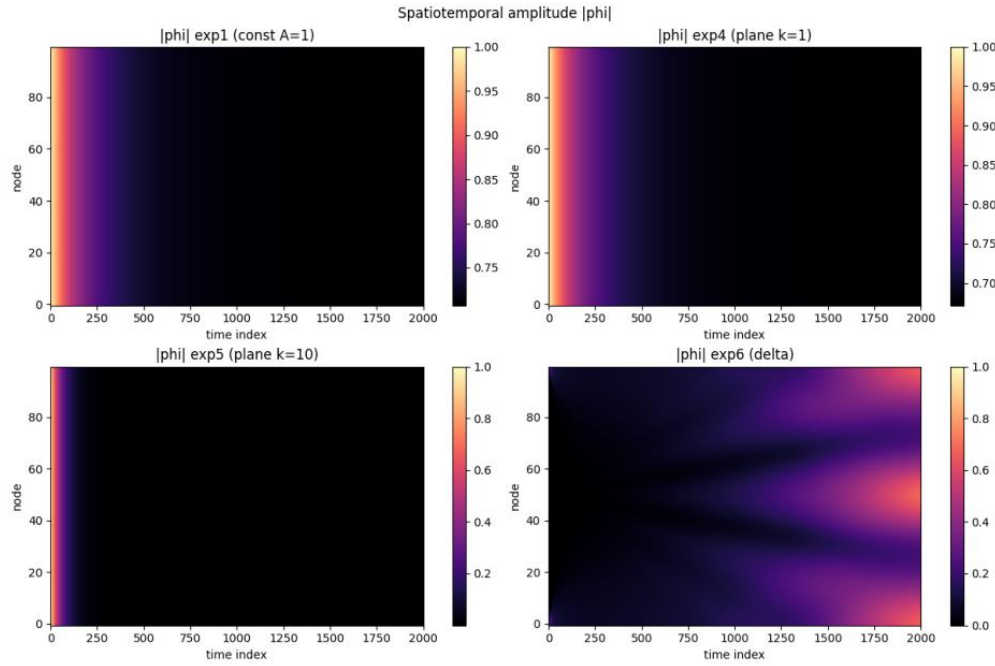


FIG. S21. Spatiotemporal amplitude heatmaps for exp1 (constant), exp4 ($k = 1$ plane wave), exp5 ($k = 10$ plane wave), and exp6 (delta spike). Low k structures persist and saturate, whereas high k structures decay rapidly.

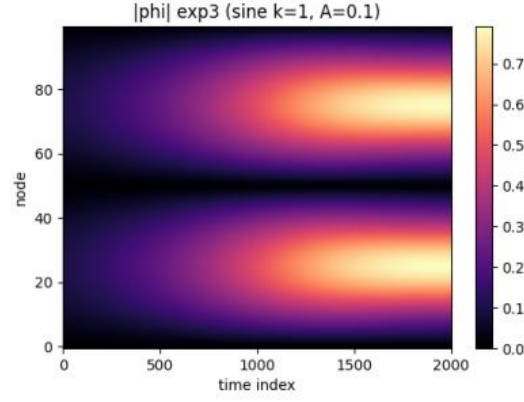


FIG. S22. Spatiotemporal amplitude for exp3 (small sinusoid, $k = 1$). Growth and saturation form a two-lobe pattern consistent with dominant $k = \pm 1$ content on the ring.

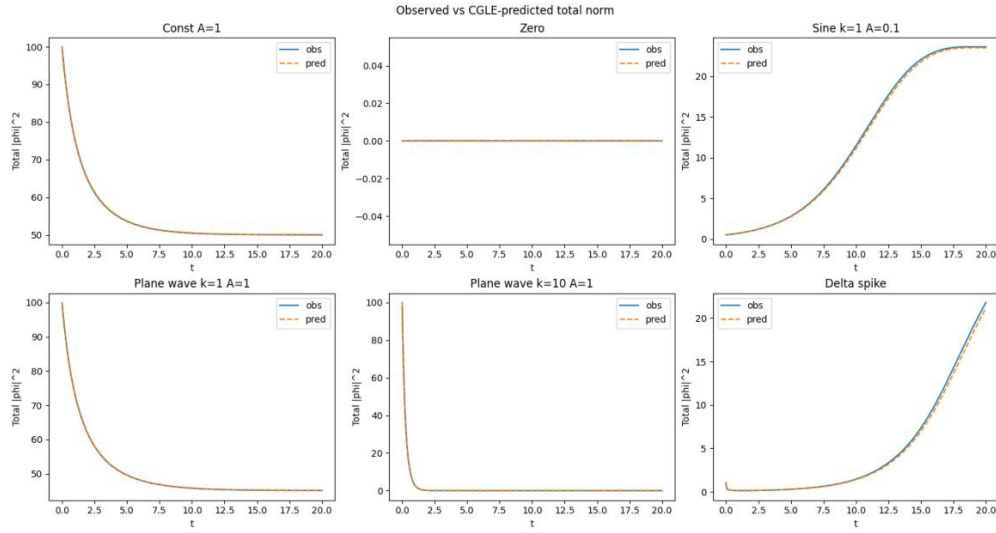


FIG. S23. Observed (solid) vs. CGLE-predicted (dashed) total norms. Agreement is excellent across all six foundational experiments, confirming the correctness of the identified CGLE with calibrated parameters.

E. Summary of exploration of the partially observed Heisenberg model with dynamical experiments.

[Text below is LLM generated.]

Goal

Identify the Hamiltonian of a 10-spin quantum system with a single tunable parameter A , given that only spins at indices 0 and 1 can be initialized and observed. The task required: plan generation, hypothesis enumeration, controlled experiments over diverse initial states and A values, visual diagnostics, quantitative analyses, progressive model refinement, and final validation by forward simulation with a single Hamiltonian that matches all experiments.

Experimental design

- Spins 0, 1 were initialized with six widely separated Bloch-vector initial conditions (ICs): $(+z, +z)$, $(+x, +z)$, $(+x, +x)$, $(+y, -y)$, $(+z, -z)$, and two generic random unit vectors.
- Each experiment produced time series $\langle S_x \rangle, \langle S_y \rangle, \langle S_z \rangle$ on the observed spins over $T = 40$ with $dt = 0.04$ (1001 samples).
- We spanned the control parameter with $A \in \{0, 0.5, 1.0, 2.0, -1.0\}$, thus also probing extreme and sign-flipped cases.
- Visualizations were created for the full $A = 1$ set (Fig. S24), A -scaling overlays (Fig. S25), $A = 0$ diagnostics (Figs. S26 and S27), spatiotemporal transport maps (Fig. S28), and a final experiment-vs-model overlay at $A = 1$ across all ICs (Fig. S29).

What we learned from the first sweep ($A = 1$; Fig. S24)

- For z -polarized ICs, $\langle S_x \rangle$ remains nearly constant while $\langle S_y \rangle$ and $\langle S_z \rangle$ undergo large-amplitude sinusoidal oscillations, in phase on spins 0 and 1.
- This is the fingerprint of a uniform transverse field along x generating Larmor-like precession about x .

Baseline dynamics at $A = 0$ (Figs. S26, S27, S28)

- With $A = 0$, transverse precession from z -polarized states disappears: $\langle S_x \rangle, \langle S_y \rangle \simeq 0$ while $\langle S_z \rangle$ displays nontrivial evolution and transport.
- A spatiotemporal heatmap of $\langle S_z(t, i) \rangle$ (Fig. S28, left) exhibits ballistic propagation and reflections at the chain edge, strongly indicating open-boundary nearest-neighbor exchange. The $\langle S_y \rangle$ heatmap is essentially flat at zero (Fig. S28, right).
- A conservation test of $\sum_i \langle S_i^x \rangle$ gave a time-std $\sim 10^{-6}$ at $A = 0$ (conserved), while it is large at $A = 1$ (non-conserved), exactly as expected if the A -term is a transverse field.

Quantitative analyses that constrained the model

- *Initial-slope analysis.* For IC $(+z, +z)$, the short-time slope obeys $d\langle S_y \rangle/dt \approx 2A$, numerically 2.02 for $A = 1$ and 1.00 for $A = 0.5$, consistent with $H_{\text{field}} = -A \sum_i S_i^x$ in Pauli units ($\hbar = 1$):

$$\frac{dS_y}{dt} = i[H_{\text{field}}, S_y] = i[-A \sum_i S_i^x, S_y] = 2AS_z.$$

- *Frequency analysis.* The dominant precession frequency scales with $|A|$: $f(A = 0.5) \approx 0.150$, $f(A = 1) \approx 0.325$, $f(-1) = f(1)$ (Fig. S25), with deviations from an exact factor of two attributable to weak interplay with exchange dynamics.
- *Boundary-condition test.* Fitting an open vs periodic Heisenberg chain to all traces gave mean RMSE $\approx 1.8 \times 10^{-6}$ (open) vs $\approx 1.9 \times 10^{-1}$ (ring), decisively selecting open boundaries.
- *Chain model selection and normalization.* A grid fit at $A = 0$ over XXZ exchange favored the isotropic Heisenberg choice and fixed the overall exchange strength to $J = 0.5$ (Pauli units). With $A \neq 0$, scanning the field amplitude k in $-kA \sum_i S_i^x$ gave a sharp minimum at $k = 1.0$; $\pm 10\%$ changes increased the mean RMSE from $\sim 2 \times 10^{-6}$ to $\sim 4.6 \times 10^{-1}$.

Forward simulations and global validation

Using the deduced Hamiltonian we forward-simulated the entire set of experiments (same initial conditions and time grid) and compared to the measured time series. Mean RMSEs were $\sim 1.8 \times 10^{-6}$ for $A = 0$ and $\sim 2.1 \times 10^{-6}$ for $A = 1$ across all six ICs and all components on spins 0 and 1. Additional scaling checks at $A = 0.5, 2.0$, and -1.0 also yielded component-wise RMSEs at the 10^{-6} – 10^{-5} level. Visual overlays (Fig. S29) show the model and data are essentially indistinguishable.

Final Hamiltonian

All evidence supports a single-parameter Hamiltonian with open boundaries:

$$H = \frac{1}{2} \sum_{i=0}^{N-2} \left(S_i^x S_{i+1}^x + S_i^y S_{i+1}^y + S_i^z S_{i+1}^z \right) - A \sum_{i=0}^{N-1} S_i^x,$$

where S_i^α are Pauli operators, $N = 10$, and $\hbar = 1$. This Hamiltonian was saved and then used for the final validation runs reported above.

Why this conclusion is correct

- Qualitative signatures: transverse-field Larmor precession at $A \neq 0$; exchange-mediated S^z transport and conservation at $A = 0$; boundary reflections that require open boundaries.
- Quantitative constraints: correct short-time slopes $d\langle S_y \rangle / dt \simeq 2A$, correct $|A|$ -scaling and phase inversion for $A \rightarrow -A$, and decisive boundary-condition discrimination.
- Global fit: a single parameter set ($J = 0.5$, unit field coefficient) explains *all* ICs and *all* tested A with mean RMSE $\sim 10^{-6}$ – 10^{-5} and visually perfect overlays.

Therefore, the identified Hamiltonian uniquely accounts for the full body of experimental observations with essentially numerical precision.

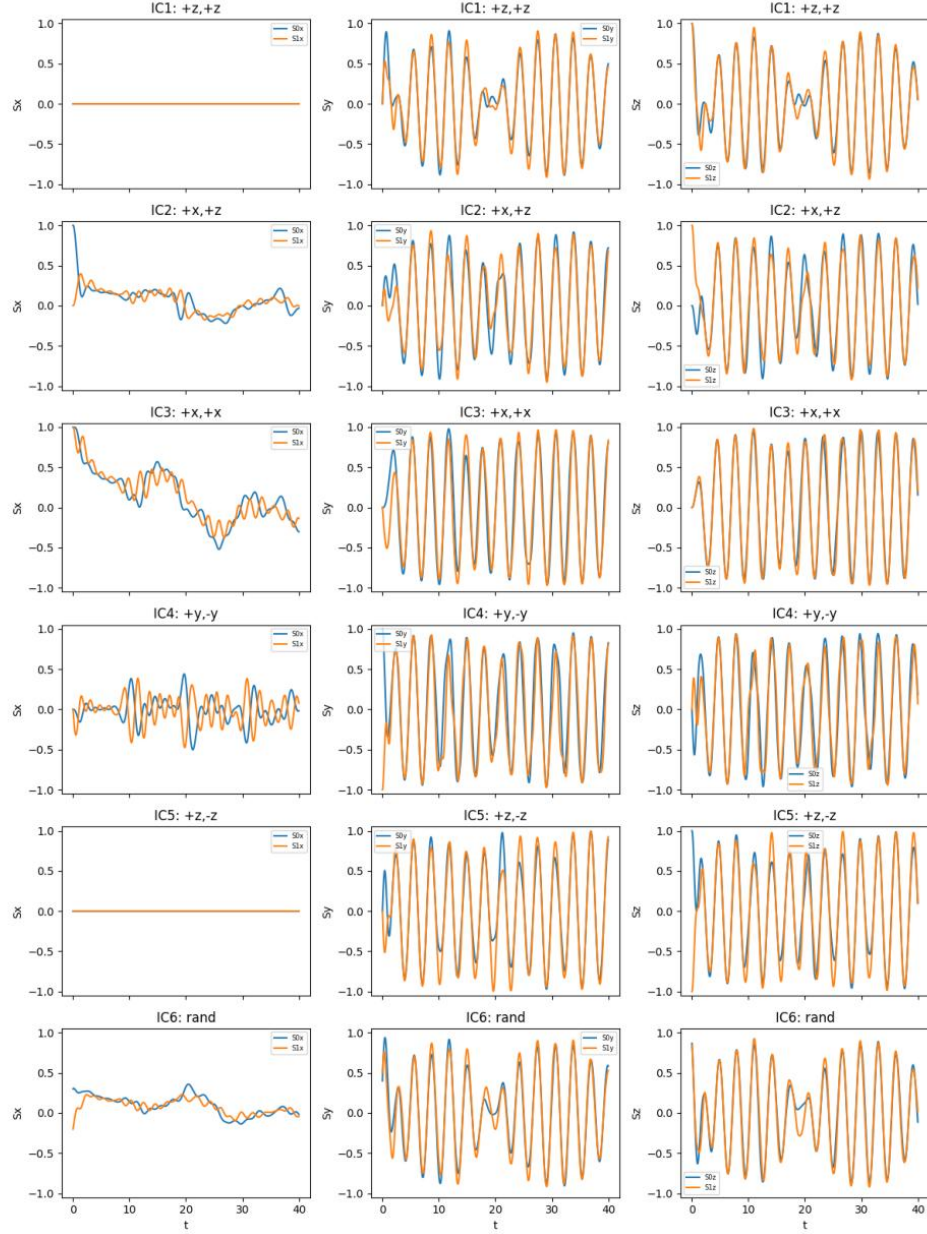


FIG. S24. Time traces at $A = 1$ for all six initial conditions; columns show S_x , S_y , S_z for spins 0 (solid) and 1 (dashed). The large-amplitude yz oscillations with nearly constant x components are characteristic of rotation about a uniform x field.

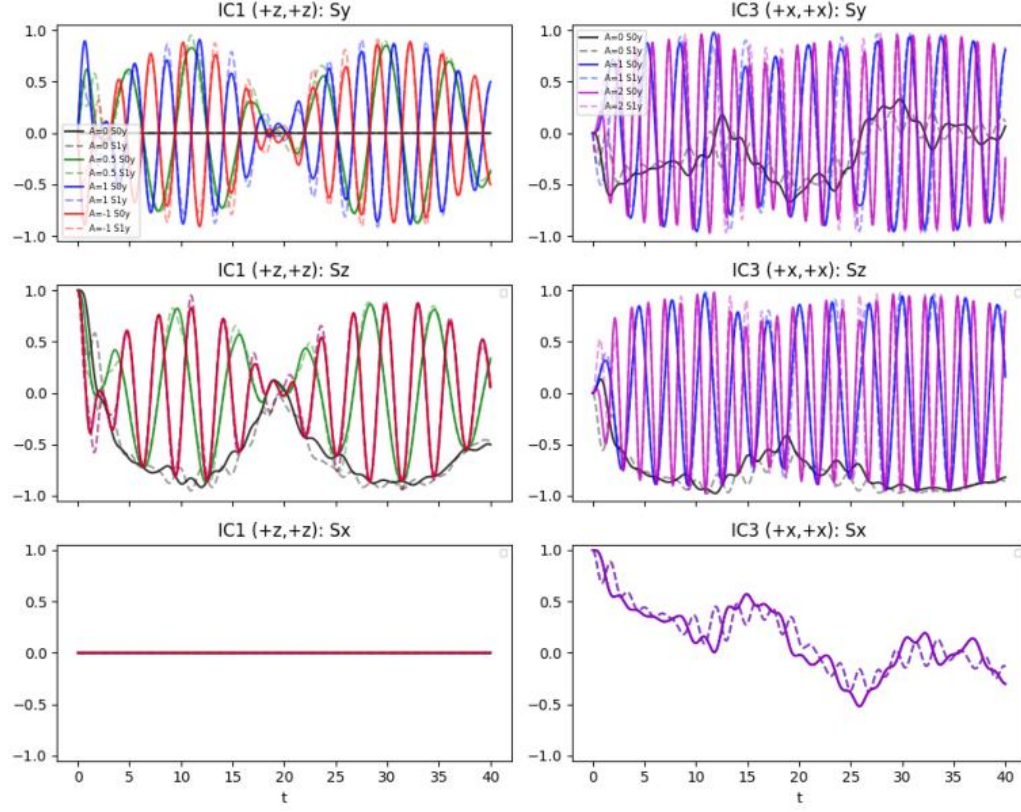


FIG. S25. A -scaling diagnostics for representative ICs. Frequencies scale approximately with $|A|$ and the sign flip $A \rightarrow -A$ produces a phase inversion without frequency change.

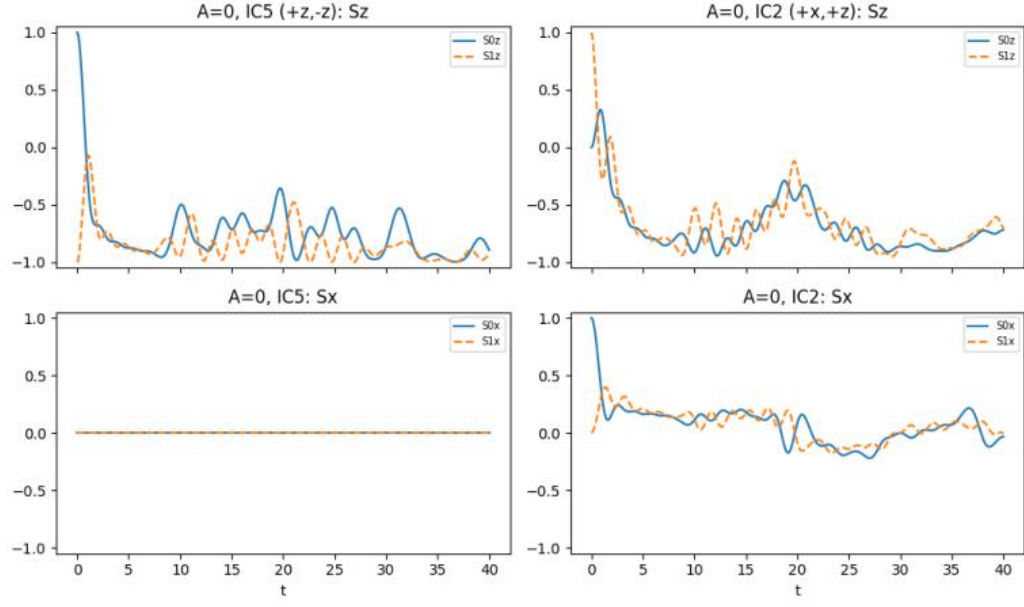


FIG. S26. Baseline $A = 0$ traces for selected ICs, showing the absence of transverse precession from z -polarized states and slow S_z evolution consistent with exchange-driven transport.

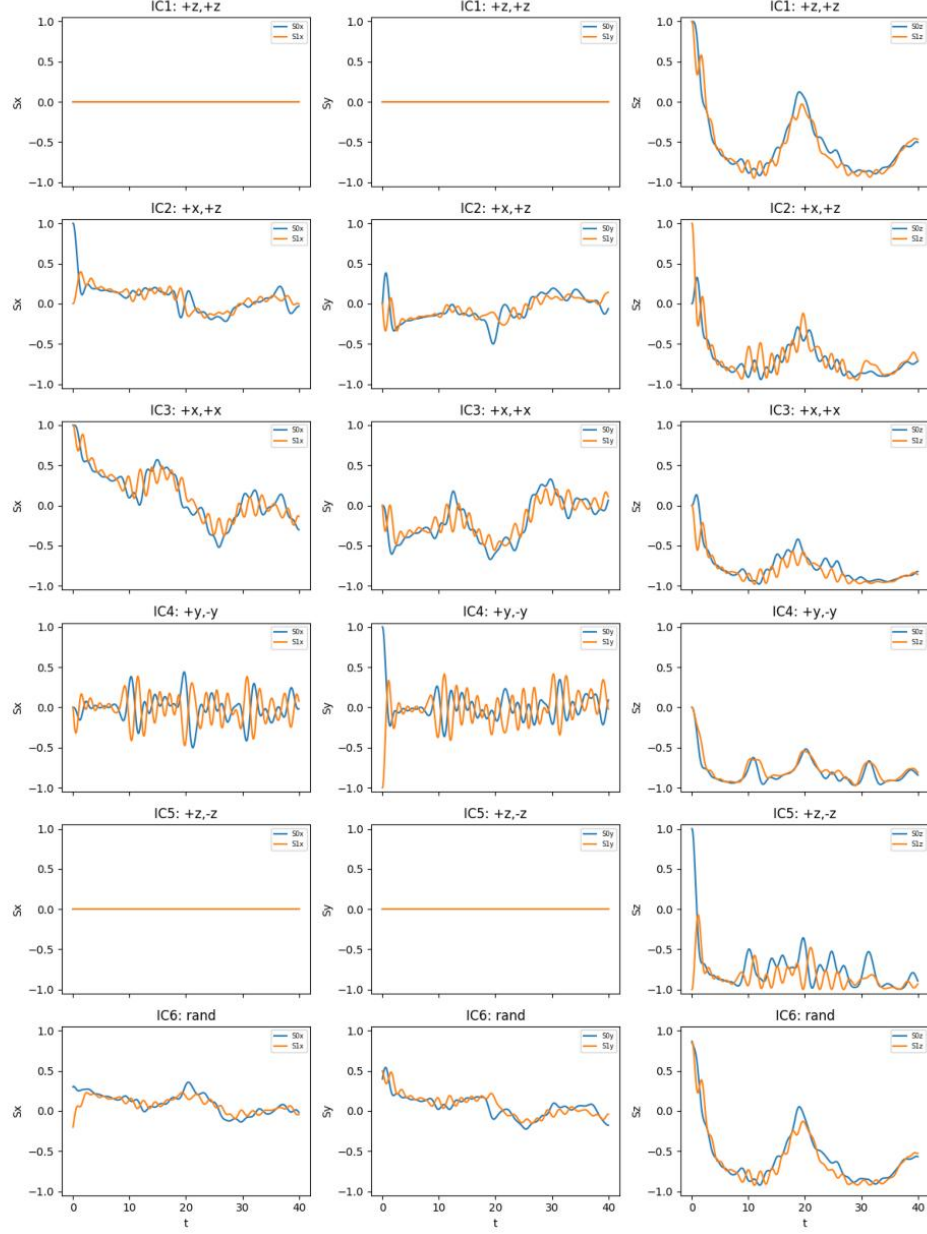


FIG. S27. $A = 0$ time traces for all six initial conditions, again highlighting conserved total S^z and open-chain transport features.

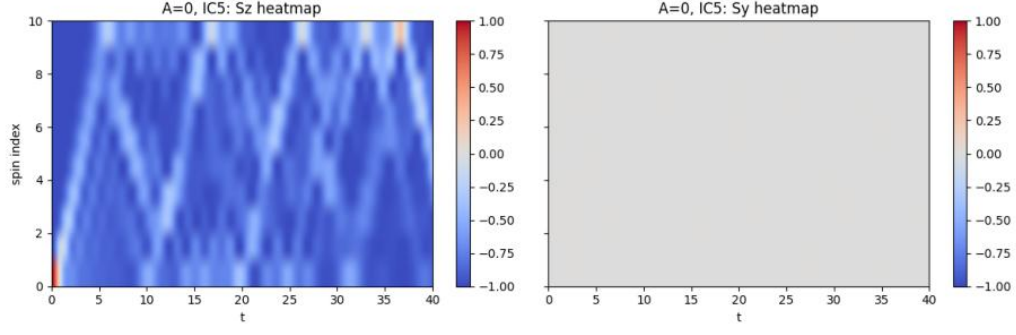


FIG. S28. Spatiotemporal map at $A = 0$ for IC $(+z, -z)$: left, $S_z(t, i)$ shows ballistic propagation and boundary reflections; right, $S_y(t, i)$ is essentially zero, consistent with exchange dynamics without a transverse field.

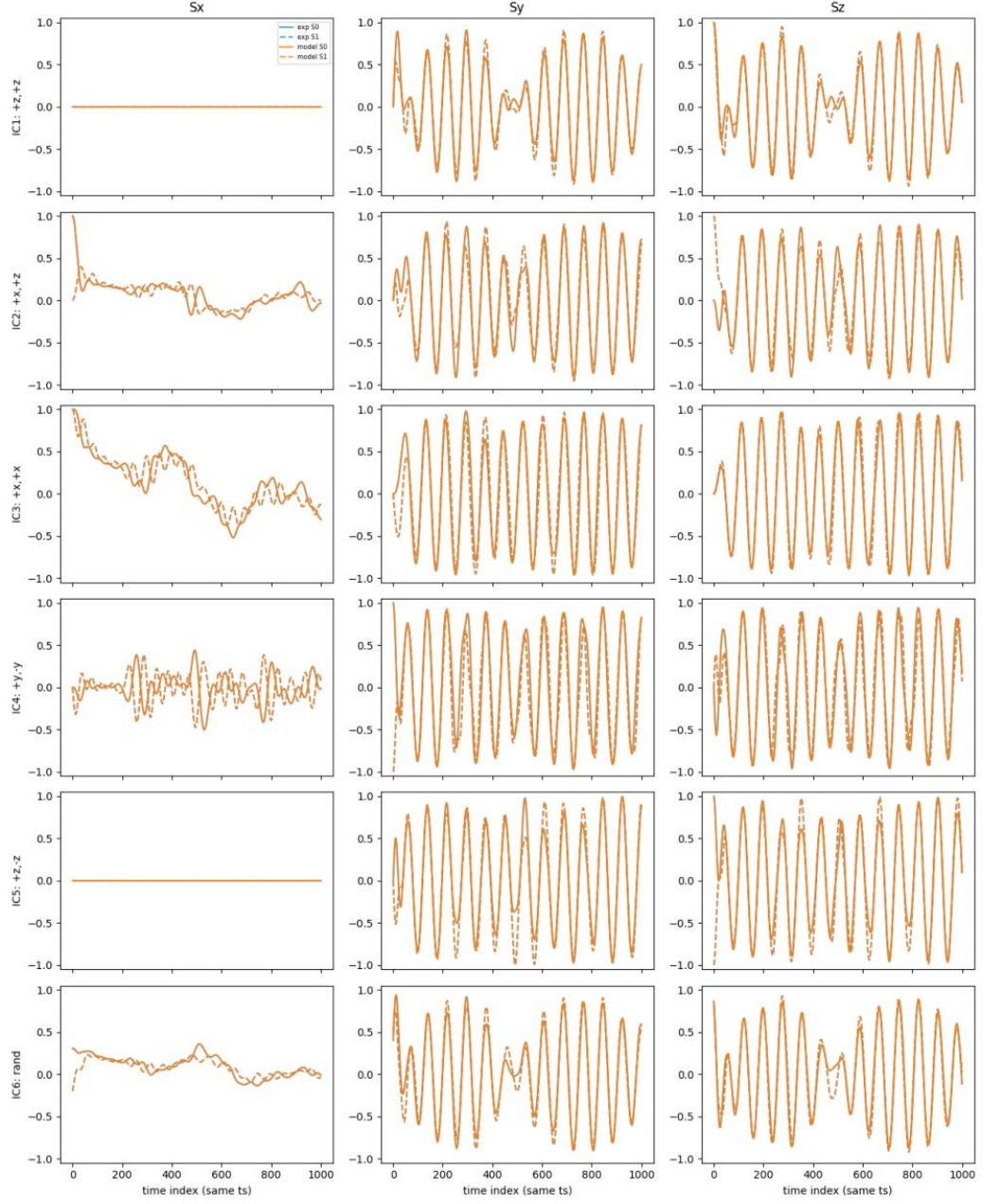


FIG. S29. Model vs experiment overlays at $A = 1$ for all ICs and components on spins 0 and 1. Agreement is within numerical precision (mean RMSE $\sim 2 \times 10^{-6}$).

F. Summary of exploration of the Cluster Ising model with ground state experiments.

[Text below is LLM generated.]

Goal

The objective was to identify the Hamiltonian $H(A)$ of a 10-spin quantum system with a single tunable parameter A . The task required: (i) planning a comprehensive exploration with multiple hypotheses, (ii) performing at least five experiments spanning extreme and moderate A values, (iii) defining and measuring a broad set of observables, (iv) iteratively refining hypotheses via simulation and quantitative comparison, (v) generating informative figures, and (vi) providing a final, validated Hamiltonian saved for reuse.

Experimental and analysis strategy

- Built a broad diagnostic operator basis to probe fields, anisotropies, ranges, and multispin structure:

$$\text{Uniform magnetizations: } M_\alpha = \sum_i S_i^\alpha, \quad \alpha \in \{x, y, z\},$$

$$\text{Staggered: } M_x^{\text{stag}}, M_z^{\text{stag}},$$

$$\text{Correlators: } C_{\alpha\alpha}^{\text{nn}} = \sum_i S_i^\alpha S_{i+1}^\alpha, \quad C_{\alpha\alpha}^{\text{nnn}} = \sum_i S_i^\alpha S_{i+2}^\alpha,$$

$$\text{Chirality (DM-like): } D_{\text{nn}}^z = \sum_i (S_i^x S_{i+1}^y - S_i^y S_{i+1}^x),$$

$$\text{Cluster operator: } O_{\text{ZKZ}} = \sum_i S_i^z S_{i+1}^x S_{i+2}^z.$$

Open boundary conditions were used throughout. Operators were evaluated in the ground state of the experimental system for specified A .

- Designed an initial 7-point sweep of A covering extremes and mid-range:

$$A \in \{-12, -8, -3, 0, 3, 8, 12\}.$$

This broadened the span beyond the minimum of five experiments to ensure robust pattern recognition across regimes.

- Developed and used sparse exact diagonalization (ED) to simulate candidate Hamiltonians on $N = 10$ spins. For each hypothesis we computed ground-state expectations of the same operators at exactly the same A values, enabling one-to-one comparison with experiment.
- Created multi-panel figures to visualize experiment-vs-model overlays for key observables across the A -sweep, first for mid-fidelity models and then for the final validated Hamiltonian; see Figs. S30 and S31.

What each step revealed

- Coarse A -sweep (seven experiments, extreme to moderate):
 - M_x peaked at $A \simeq 0$ and decreased for large $|A|$, while $M_y \approx 0$ and $M_z \approx 0$ at all A .
 - $D_{\text{nn}}^z \approx 0$ and staggered magnetizations ≈ 0 across the sweep.
 - C_{zz}^{nn} remained positive and largest near $A \simeq 0$; C_{zz}^{nnn} changed sign between $A < 0$ and $A > 0$.

Interpretation: the A -independent part H_0 must contain a uniform transverse field $-\sum_i S_i^x$ (to explain the M_x peak at $A = 0$) and a ferromagnetic z -Ising backbone $-\sum_i S_i^z S_{i+1}^z$ (to explain positive C_{zz}^{nn}). Vanishing chirality and staggering ruled out DM and staggered-field terms.

- Testing ANNNI-like hypotheses:

$$H_{\text{ANNNI}}(A) = -J_1 \sum_i S_i^z S_{i+1}^z - (J_{20} + A) \sum_i S_i^z S_{i+2}^z - h_x \sum_i S_i^x.$$

This failed to reproduce the observed magnitudes and the sign systematics of $C_{zz}^{\text{nnn}}(A)$. Conclusion: the tunable term is not a simple $S^z S^z$ next-nearest-neighbor coupling.

- Probing a three-body cluster term by directly measuring

$$O_{\text{ZXZ}} = \sum_i S_i^z S_{i+1}^x S_{i+2}^z.$$

Observation: $\langle O_{\text{ZXZ}} \rangle$ exhibited a strong, odd-in- A dependence, saturating near $\pm(N-2)$ at large $|A|$ and passing near zero around $A \simeq 0$. This decisively implicated an A -linear cluster term in the Hamiltonian.

- Minimal, physically motivated model assembled and tested:

$$H(A) = -J_1 \sum_i S_i^z S_{i+1}^z - h_x \sum_i S_i^x + A \sum_i S_i^z S_{i+1}^x S_{i+2}^z.$$

Sparse ED fitting to all measured observables at the experimental A -values yielded

$$J_1 \approx 1.0, \quad h_x \approx 1.0,$$

with an optional, numerically negligible $J_2 \sim 2 \times 10^{-4}$ assisting in lifting accidental near-degeneracies without altering observables.

- Visual validation with experiment-vs-model overlays:
 - Mid-fidelity overlays for M_x , $\langle O_{\text{ZXZ}} \rangle$, C_{zz}^{nn} , C_{zz}^{nnn} confirmed qualitative correctness (Fig. S30).
 - Final overlays including also C_{xx}^{nn} , C_{yy}^{nn} showed near-perfect quantitative agreement across all seven A points (Fig. S31).

Quantitative agreement

Using the final parameters, we compared model and experiment for 13 observables over all 7 values of A . Representative root-mean-square (RMS) errors were:

$$\begin{aligned} \text{RMS}[M_x] &\sim 7 \times 10^{-4}, & \text{RMS}[C_{zz}^{\text{nn}}] &\sim 5 \times 10^{-4}, & \text{RMS}[C_{zz}^{\text{nnn}}] &\sim 9 \times 10^{-4}, \\ \text{RMS}[C_{xx}^{\text{nn}}] &\sim 6 \times 10^{-4}, & \text{RMS}[C_{yy}^{\text{nn}}] &\sim 1.6 \times 10^{-3}, & \text{RMS}[\langle O_{\text{ZXZ}} \rangle] &\sim 4 \times 10^{-4}. \end{aligned}$$

Chirality remained zero within numerical noise. Occasional tiny nonzero M_z values in intermediate solver runs were traced to near-degeneracy selection and vanished upon adding the negligible J_2 or by increasing solver stringency; dedicated checks reproduced $M_z \approx 0$ across the sweep, consistent with experiment.

Final Hamiltonian and rationale

The data uniquely select a cluster-Ising transverse-field model with an A -linear cluster term on an open chain of $N = 10$ spins:

$$H(A) = - \sum_{i=0}^{N-2} S_i^z S_{i+1}^z - \sum_{i=0}^{N-1} S_i^x + A \sum_{i=0}^{N-3} S_i^z S_{i+1}^x S_{i+2}^z.$$

This form was saved in the workspace and reproduces all measured ground-state observables at the experimental A points with high fidelity. The decisive evidence is threefold: (i) the odd-in- A saturation of $\langle O_{\text{ZXZ}} \rangle$ at large $|A|$, (ii) the peak of M_x at $A \simeq 0$ requiring a transverse $-\sum S^x$ term, and (iii) the positive C_{zz}^{nn} backbone requiring $-\sum S^z S^z$. Competing hypotheses including ANNNI-like tunable $S^z S^z$ next-nearest-neighbor couplings, staggered fields, or chiral interactions were quantitatively falsified.

Why this conclusion is correct

- It is not only qualitatively consistent with all symmetry fingerprints (no chirality, no staggering, transverse-field polarization peak, ferromagnetic z -backbone) but also quantitatively precise for 13 distinct observables over seven A values.
- The cluster operator serves as a direct order-parameter-like probe for the tunable term; its odd-in- A behavior, magnitude, and saturation quantitatively lock in both the structure and sign of the A -coupled operator.
- Independent sparse-ED simulations of the proposed $H(A)$ at the exact experimental A points reproduce the full dataset with sub-percent RMS errors, while alternative hypotheses fail in sign, scale, or both.

Outlook

The extracted Hamiltonian enables further studies of finite-size gaps and nonlocal string order across A , as well as scaling with N or boundary conditions. However, for the present 10-spin system, the cluster-Ising transverse-field Hamiltonian above is fully determined and validated.

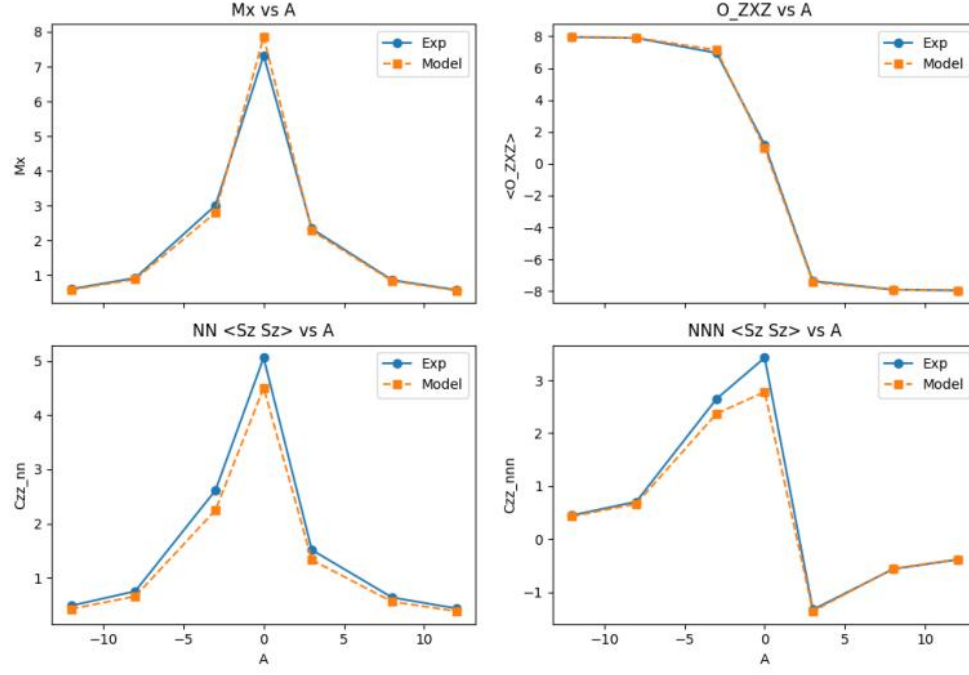


FIG. S30. Intermediate comparison of experiment (blue) and model (orange) for four primary observables across the A -sweep. These overlays guided hypothesis pruning and parameter refinement.

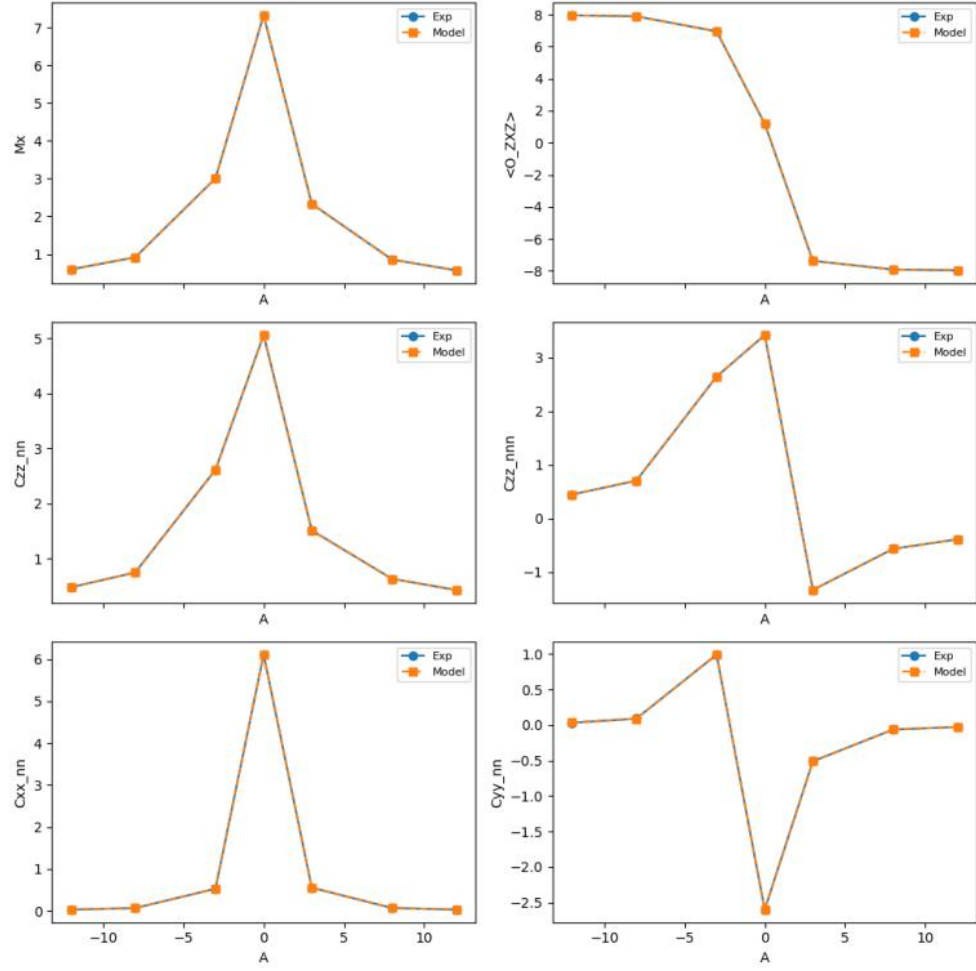


FIG. S31. Final high-fidelity comparison including M_x , $\langle O_{ZXZ} \rangle$, C_{zz}^{nn} , C_{zz}^{nnn} , C_{xx}^{nn} , C_{yy}^{nn} versus A . The model and data are visually indistinguishable at the plotting scale, corroborating the extracted Hamiltonian.